

Security!

Michael A. Specter

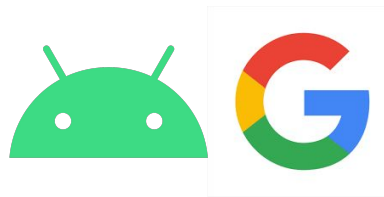
CS3210
Spring 2026

Hi, I'm Michael A. Specter

PhD, EECS

Senior Research Scientist

Assistant Professor, SCP & SCS



Things I hack on:

Build & break systems relevant to society

- Found security flaws in every Internet Voting system used in U.S. Federal Elections
- Written briefs to SCOTUS (via EFF)
- Large-scale reverse engineering
 - Age verification
 - In-App Advertising Privacy

What is computer security?

Two definitions

1. Security as systems design (akin to medicine)
2. Security as ecosystem analysis (akin to public health)

Two definitions (For this class)

1. Security as systems design (akin to medicine)

2. Security as ecosystem analysis (akin to public health)

Security as *systems* design.

Security Defined

Security is the design of systems that achieve some property against an active adversary.

Process:

define a system,
provide a set of security goals,
and prove that the system actually follows those goals.



Components: Goals & Threat Model

Goals:

- You shouldn't be able to read my files
- No process should be able to read another process's memory
- Could also be *categories* (CIA triad)

Threat model:

- Assumptions about what the attacker can do
- E.g. can one guess passwords?

Threat model specifies what's in/out of scope.

Goal defines what you want to achieve.

Components: Implementation

How you actually achieved your goals

1. Policy:

A set of *rules*

Note: this is *technical* policy

E.g. file permissions, requiring passwords/2FA

2. Mechanism:

Hardware/Software that actually enforces your policies

User accounts, encryption, etc

Let's talk about an example

- Your TA controls Canvas, which, ultimately, controls your grade.
- Policy: Only your TA should be able to change grades in Canvas.
- How would you cheat to get an A?
 - You might steal her computer
 - Break into her GT account
 - Break into Canvas's servers
 - Break into your professor's account
 - Bribe her
- There are a ton of attacks!
- Security is a negative goal!
 - Allowing badness involves a failure in any number of ways

What could go wrong?

What could go wrong?

1. Bad Policies / Goals
2. Bad Threat Models
3. Bad Mechanisms

What could go wrong?

1. Incorrect Goals / Policies.

Incorrect goals / policies

- Fairfax County Virginia, a 9-year-old boy caught accessing grade mgmt account owned by the superintendent
- System Policies:
 - Student can access only his/her own files
 - Superintendent has access to everyone's grades, files, etc
 - Teachers can add new students to their class
 - Teachers can change password of any student in their class
- What's the worst that could happen if a student gets a teacher's password?
 - Student adds the superintendent to the compromised teacher's class.
 - Changes the superintendent's password.
 - Logs in as superintendent and gets access to all files.
- Policy amounts to: **teachers can do anything.**

Where policies fail

Policies typically go wrong in "management" or "maintenance" cases.

- Who can change permissions or passwords?
- Who can access audit logs?
- Who can access the backups?
- Who can upgrade the software or change the configuration?
- Who can manage the servers?
- Who revokes privileges of former admins?

Getting a threat model right can be
subtle.

What could go wrong?
2. Bad Threat Models

For example:
I assume the adversary does
not know how my system works.

Kerckhoff's Principle

A system is only secure if the adversary knows everything about the system, and it is still secure.

The enemy knows the system.

Inverse: “Security through obscurity”
Does. Not. Work.



Bad Threat Model Examples:

- Stuxnet
 - Iranian nuclear facility centrifuges were destroyed
 - Used an airgapped network
 - Assumed that malware couldn't jump the airgap
 - ...This assumption was bad, malware was installed via USB
- Computational assumptions
 - Cryptographic primitives may not survive as computation gets better – e.g. DES
 - Cryptography & Quantum Computers: Shor's algorithm breaks RSA

Bad Threat Model Examples: Just Trust Your Hardware

- Farewell dossier

- During the cold war, Soviets tended to steal US plans / tech. Often lacked technical knowhow
- Problem: the US had a defector, “Farewell”, in that group.

Gus Weiss, WH’s NSC Director of International Econ:

“Contrived computer chips found their way into Soviet military equipment, flawed turbines were installed on a gas pipeline, and defective plans disrupted the output of chemical plants and a tractor factory. The Pentagon introduced misleading information pertinent to stealth aircraft, space defense, and tactical aircraft. The Soviet Space Shuttle was a rejected NASA design.”

Bad Threat Model Examples: Just Trust Your Software

- 2015: Chinese iOS Developers Mirrored xcode
 - Took too long to download from the U.S.
 - Some of these mirrors had malware'd versions of xcode
 - Injected malware into apps
 - (look up: xcodeGhost)

Reflections on Trusting Trust

To what extent should one trust a statement that a program is free of Trojan horses? Perhaps it is more important to trust the people who wrote the software.

KEN THOMPSON

Bad Threat Model Examples: Just Trust Updates

- Apple v. FBI (now, I guess, the UK)
- NPM's Left-Pad Incident
- Chrome extension bought by malware authors
 - <https://arstechnica.com/security/2014/01/malware-vendors-buy-chrome-extensions-to-send-adware-filled-updates/>

What could go wrong?
3. Bad Mechanisms

Mechanism Failures

- **Rule of Thumb:** Any bug can potentially be a security issue.
 - E.g. buffer overflows, use-after-free, code injection failures
 - E.g. Poor randomness for cryptography
 - E.g. Timing attacks
- Bad when it allows an attacker to cross a *trust boundary* and *elevate privileges*
 - E.g. I trust that a remote attacker supplying a value to my function cannot control the entire program. A bug gave the attacker control of the entire program.
 - E.g. an attacker finds a flaw in a program with more privs to do something

Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

What's wrong with this code?

Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

What happens when you
make a call to `vulnerable_function`?

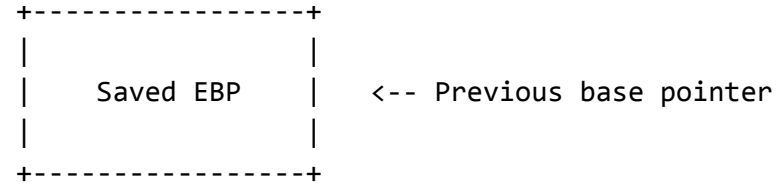
Let's say the input is '1234567890'

Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

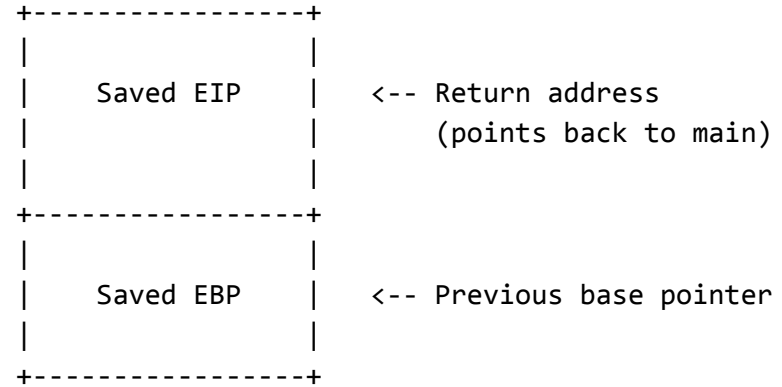


Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

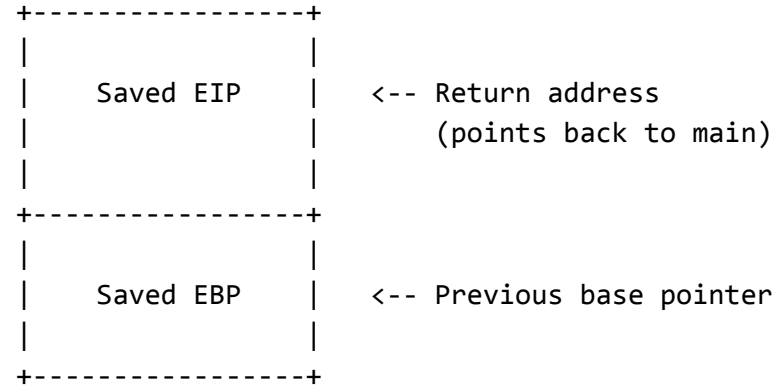


Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

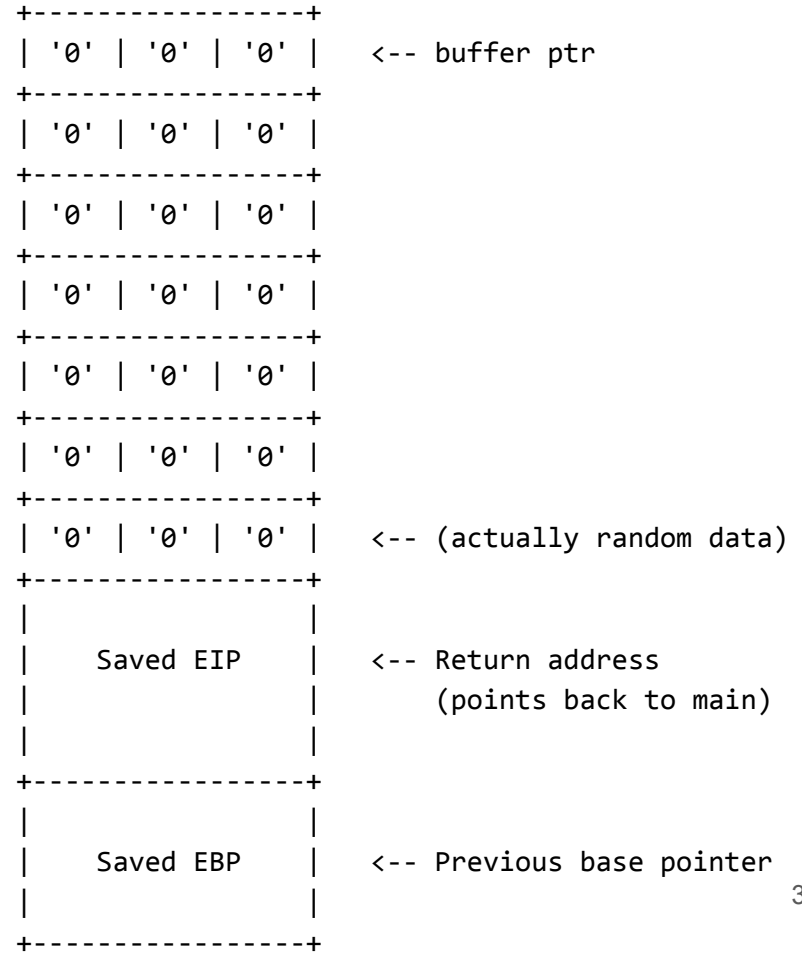


Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

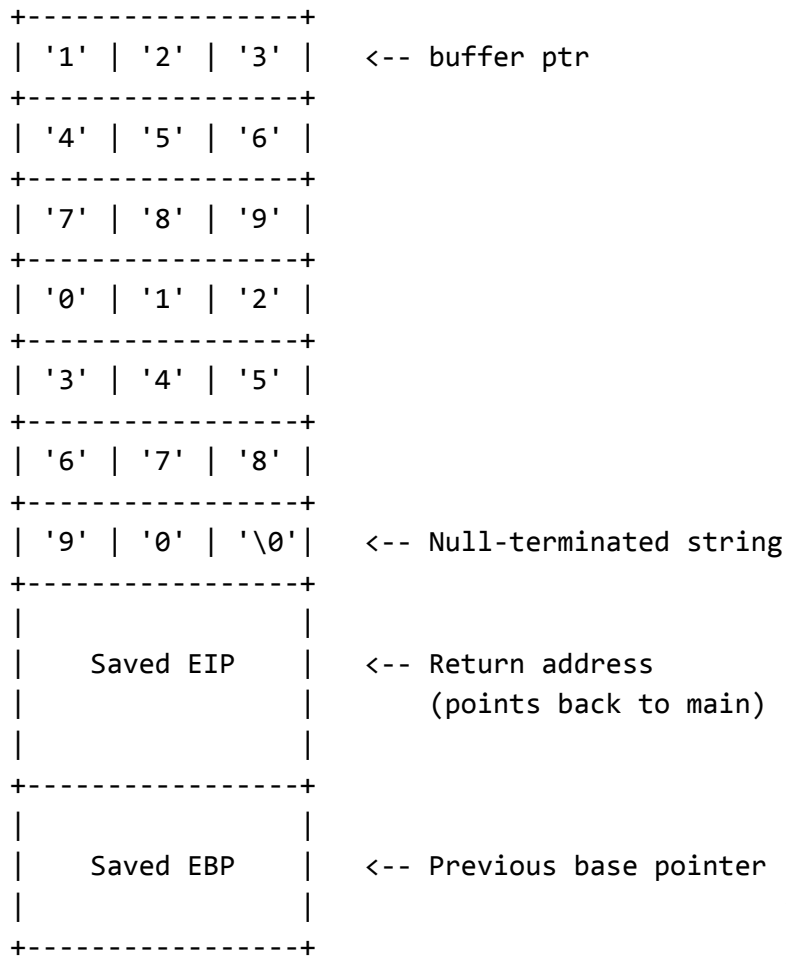


Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```



Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```

What happens when you
make a call to `vulnerable_function`?

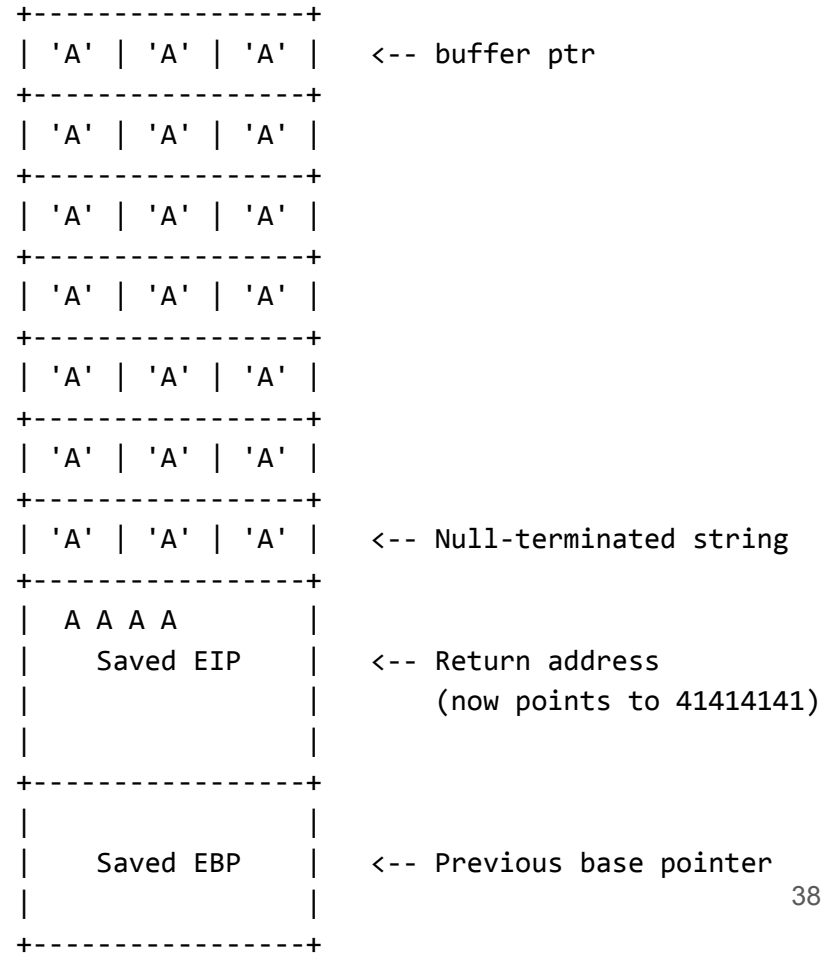
Now let's say the input is 'A' * 100

Let's talk about Memory Safety!

```
> cat test.c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[10];
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    vulnerable_function(argv[1]);
    return 0;
}
```



Once I control EIP (RIP)

In this case, A's will return to hex 41414141, which isn't useful, but:

- Return to code I wrote on the stack
- Return to known functions (e.g. known addresses of things in libc)
- Game is generally over =(

Generally called memory safety issues

- Buffer overreads, writes, etc
- Use-after-free
- Happens both on the stack & heap
- For more (& a history) check out:

Szekeres et al, SoK: Eternal War in Memory, IEEE S&P 2013

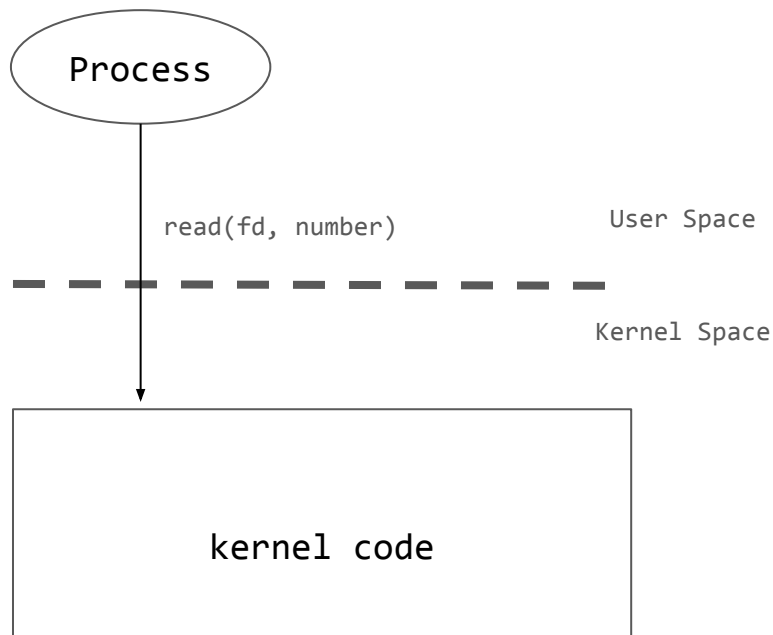
Relevant to Kernel Devs: Syscall security

There are ~456 syscalls in Linux

<https://filippo.io/linux-syscall-table/>

Linux kernel has ~40M lines of code.

Device drivers are written by OEMs
...And, generally, not written well.



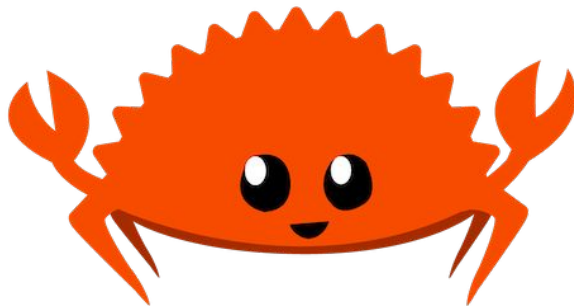
Current, unfixed bugs in linux

open (1402):							
Title	Repro	Cause bisect	Fix bisect	Count	Last	Reported	Discussions
WARNING: bad unlock balance in __rtnl_unlock <code>[net]</code>	C	done		17	12h54m	2h34m	PATCH [2h31m]
WARNING in taprio_get_start_time <code>[net]</code>				1	9h40m	5h29m	
INFO: trying to register non-static key in sock_def_readable <code>[net]</code> ...	syz			1	8h51m	5h29m	
BUG: corrupted list in __sk_destruct(2) <code>[net]</code> <code>[ext4]</code>	syz			1	19h50m	5h29m	
BUG: sleeping function called from invalid context in __bio_release_...				2	4d08h	8h22m	
KASAN: slab-out-of-bounds Read in validate_bset_keys <code>[bcachefs]</code>	C			5	1d16h	1d00h	
WARNING: bad unlock balance in __mm_populate <code>[xfs]</code> <code>[mm]</code>	C			13	1h49m	1d22h	☹ 4 [22m]
KASAN: slab-use-after-free Read in nf_tables_trans_destroy_work(2...	syz			1	2d17h	2d03h	PATCH [3h35m]
KASAN: global-out-of-bounds Read in get_mem_cgroup_from_mm ...	C			1	2d16h	2d03h	
kernel BUG in try_to_unmap_one <code>[mm]</code>				1	6d16h	2d16h	☹ 3 [22h37m]
WARNING in exit_itimers <code>[kernel]</code>				1	7d02h	3d02h	
BUG: unable to handle kernel paging request in io_schedule <code>[ntfs3]</code>	C			4	7d00h	3d07h	
INFO: task hung in invalidate_inode_pages2_range(3) <code>[bcachefs]</code>	C	done		14	21h53m	4d07h	
kernel BUG in filemap_fault <code>[mm]</code>				1	8d08h	4d08h	
INFO: task hung in pipe_release(6) <code>[bcachefs]</code>	syz	done		4	6d22h	4d19h	
KASAN: slab-use-after-free Read in task_work_run <code>[kernel]</code>	syz	error		2	8d19h	4d22h	☹ 1 [3d15h]
INFO: reu detected stall in call_timer_fn(5) <code>[net]</code>	C			1	9d11h	5d11h	
WARNING: bad unlock balance in traverse <code>[fs]</code>	C			13	1d01h	5d17h	
possible deadlock in ipv6_sock_ac_close(4) <code>[net]</code>				2	1d23h	5d23h	
KMSAN: uninit-value in cfg80211_tx_mlme_mgmt <code>[wireless]</code>				1	9d23h	5d23h	PATCH [5d06h]
WARNING in ieee80211_free_keys <code>[wireless]</code>				2	7d17h	6d02h	
WARNING in ieee80211_set_disassoc <code>[wireless]</code>				1	10d	6d04h	
WARNING in get_dump_page <code>[mm]</code> <code>[ext4]</code>	C	done		5	6d23h	6d04h	
WARNING in hcj_send_cmd(2) <code>[bluetooth]</code>				1	10d	6d09h	
general protection fault in native_tss_update_io_bitmap <code>[kernel]</code>	C	error		7	6d23h	6d09h	PATCH [6d00h]
possible deadlock in l2cap_info_timeout <code>[bluetooth]</code>				1	10d	6d19h	
KCSAN: data-race in netfs_advance_write/netfs_write_collection_w...				7	3d00h	7d04h	
KASAN: slab-out-of-bounds Read in fbcon_prepare_logo <code>[fbdev]</code>	syz			7	3d02h	7d08h	
possible deadlock in crypto_exit_scomp_ops_async <code>[crypto]</code>				1	11d	7d17h	PATCH [5d13h]
KASAN: slab-out-of-bounds Read in hfsplus_bmap_alloc <code>[hfs]</code>	C			1	7d22h	7d21h	
INFO: task hung in ...	syz	done		1	8d07h	7d23h	

<https://syzkaller.appspot.com/upstream>

Solutions:

- Bounds checking (e.g. strncpy vs strcpy)
- compiler safety helpers ASLR, DEP, and other
- Some OS Primitives can be helpful
- The real solution is memory safety (e.g. rust, not c)





FEBRUARY 26, 2024

PRESS RELEASE: Future Software Should Be Memory Safe



► ONCD ► BRIEFING ROOM ► PRESS RELEASE

Leaders in Industry Support White House Call to Address Root Cause of Many of the Worst Cyber Attacks

Read the full report [here](#)

WASHINGTON – Today, the White House Office of the National Cyber Director (ONCD) released a report calling on the technical community to proactively reduce the attack surface in cyberspace. ONCD makes the case that technology manufacturers can prevent entire classes of vulnerabilities from entering the digital ecosystem by adopting memory safe programming languages. ONCD is also encouraging the research community to address the problem of software measurability to enable the development of better diagnostics that measure cybersecurity quality.

Passwords!

How should you store a password on-disk?

Solution #1:

Just store it in plaintext!

What could go wrong?

How should you store a password on-disk?

Solution #2: Let's talk about hashes!

$$h: \{0,1\}^* \rightarrow \{0,1\}^d$$

In words: Hash function h maps a bit string of any size to a bit string of length d .

Goals: Efficient, deterministic, random, public

Examples:

MD5	$d=128$
SHA 256	$d=256$
SHA3-256	$d=256$
SHAKE	$d=\text{variable}$

How should you store a password on-disk?

Solution #2: Let's talk about hashes!

$$h: \{0,1\}^* \rightarrow \{0,1\}^d$$

Other Goals:

Collision Resistance:

Given x , cannot find x' s.t. $h(x) = h(x')$ (assuming $x \neq x'$)

One-wayness:

Given $y \leftarrow h(x)$, where x is random, cannot find x' s.t. $h(x') = y$

How should you store a password on-disk?

Just store the password as $h(\text{password})$!

What could go wrong?

Brute force attack:

1. Create a database of likely passwords $P=\{p_1 \dots p_n\}$
2. Calculate $\{(h(p_1), p_1) \dots (h(p_n), p_n)\}$
3. You can now recover most “likely” passwords.
4. Works **even if $|P|$ is prohibitively large**: **Rainbow Tables**
 - a. Exploits time/space tradeoff to limit size of $|P|$

WEB / POLICY / BREAKING

Adobe suffers major cyber attack, says data on 2.9 million customers was compromised

By [Chris Welch](#), a reviewer specializing in personal audio and home theater. Since 2011, he has published nearly 6,000 articles, from breaking news and reviews to useful how-tos.

Source [Adobe](#)

Oct 3, 2013, 5:09 PM EDT



Comments (0 New)

TECH

Over 150 million breached records from Adobe hack have surfaced online

By [Chris Welch](#), a reviewer specializing in personal audio and home theater. Since 2011, he has published nearly 6,000 articles, from breaking news and reviews to useful how-tos.

Source [Naked Security \(Sophos blog\)](#)

Nov 7, 2013, 6:08 PM EST



0 Comments (0 New)

ADOBE ENCRYPTED THE PASSWORDS IMPROPERLY, MISUSING BLOCK-MODE 3DES. THE RESULT IS SOMETHING WONDERFUL:

THE GREATEST CROSSWORD PUZZLE
IN THE HISTORY OF THE WORLD

This is now a real thing.

<https://zed0.co.uk/crossword/>

A crossword based on the Adobe password leak.
Inspired by [xkcd #1286: Encryptic](#)

Password popularity:

1-100

101-200

201-300

301-400

401-500

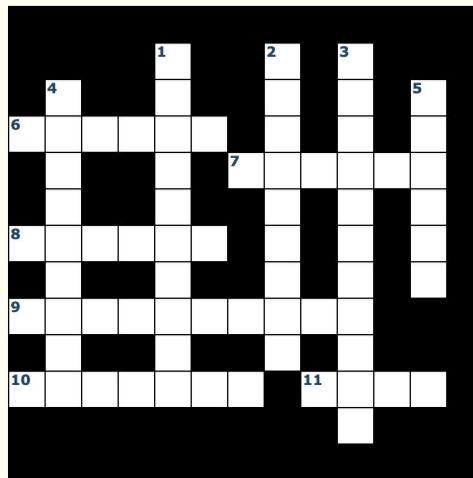
501-600

601-700

701-800

801-900

901-1000



Reveal

Check

Hide

Across

▼ 6: zk8NJgAOqc4=

dog; cat; pet;
dark; dogs name;
Dog; my dog;
black dog; dog
name; dog's
name; darkness;
black cat; sonic;
black; Kitty; horse;
Cat; pets name;
sombra; puppy;
cats name; old
dog; shade; first
dog; pet name;
doggy; hedgehog;
cat's name; bike;
my cat; nickname;
Pet; me; light;
favorite pet;
usual; sha;
doggie; pet's
name; first pet;
animal; sh; shad;
s; car; first cat;
Dog's name;
chien; favorite
dog; ombre

Down

▼ 1:
2aZl4Ouarwm52NYYl9c

adobe; adobe2;
adobe2; adobe
twice; twice;
adobetwice;
adobe2x; site;
??????; name;
software; 2x;
company;
2xadobe;
programa; adobe
x 2; program;
adobe x2; ???;
ad; adobe*2;
????; Adobe;
double;
namenname;
2adobe; ?????;
x2; a; name twice;
photoshop;
company name;
adobe adobe;
adobe?; ado; aa;
company twice; 2;
marca; website;
none; adobe 2x;

How should you store a password on-disk?

Real solution: Hash and Add Salt!

Get a random number r :

$$r \leftarrow \$ \{0,1\}^d$$

Store:

$$h(\text{password} \parallel r), r$$

Attacker cannot easily precompute a table!

Modern techniques also include using intentionally slow hash functions

See bcrypt

This time:

Security Defined

Goals & Threat Model

Implementation

- Policies, mechanisms

How things can go wrong:

- Bad Policies

- Bad Threat Models

- Bad Mechanisms

Mechanism failures

Next time:

OS Design for Security

Isolation, containers, fuzzing & more!

Everything is not hopeless.

Security (2)!

Michael A. Specter

CS3210
Spring 2026

Lecture #2 – Solutions

Last Time:

Security Defined

Goals & Threat Model

Implementation

- Policies, mechanisms

How things can go wrong:

- Bad Policies

- Bad Threat Models

- Bad Mechanisms

Mechanism failures

This Time:

OS Design for Security

Isolation, containers, fuzzing & more!

Security in the real world: Elections

Everything is not hopeless.

Access Control!

On-device Access Control Problem

Once a program is running on a system, how do we limit what it can do?

- E.g. multiple users ssh'd in to one system
- E.g. untrusted app running locally
- E.g. untrusted web renderer

Aside:

If someone proposed running random untrusted code from the Internet...



Strategies: DAC & MAC

Discretionary Access Control (DAC)

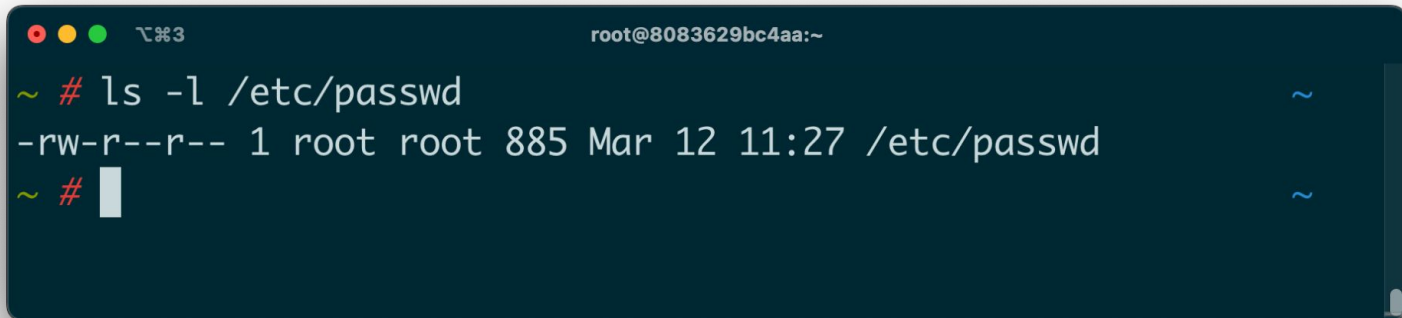
- Owner of a resource defines its access properties
- Can transfer ownership, control, etc

Mandatory Access Control (MAC)

- Policy for all resources is set at the organizational level / e.g. the computer

DAC Example: Unix Files

- Each user has:
 - A User ID (UID)
 - Any number of Group ID's (GID)
- All files have associated ACL metadata:
 - User owner
 - Group owner
 - Read, Write, eXecute bits for {user, group, other}
- Stored in inode metadata

A terminal window with a dark blue background and light green text. The window title bar shows three colored circles (red, yellow, green) and the text "root@8083629bc4aa:~". The terminal content shows a command prompt "~ # " followed by the command "ls -l /etc/passwd". The output is "-rw-r--r-- 1 root root 885 Mar 12 11:27 /etc/passwd". Below the output, the prompt "~ # " is shown again with a white cursor block.

```
root@8083629bc4aa:~  
~ # ls -l /etc/passwd  
-rw-r--r-- 1 root root 885 Mar 12 11:27 /etc/passwd  
~ #
```

DAC Example: Unix Files

```
root@8083629bc4aa:~  
~ # ls -l /etc/passwd  
-rw-r--r-- 1 root root 885 Mar 12 11:27 /etc/passwd  
~ #
```

User

Group

Other

User Owner

Group Owner

DAC Problem: SetUID

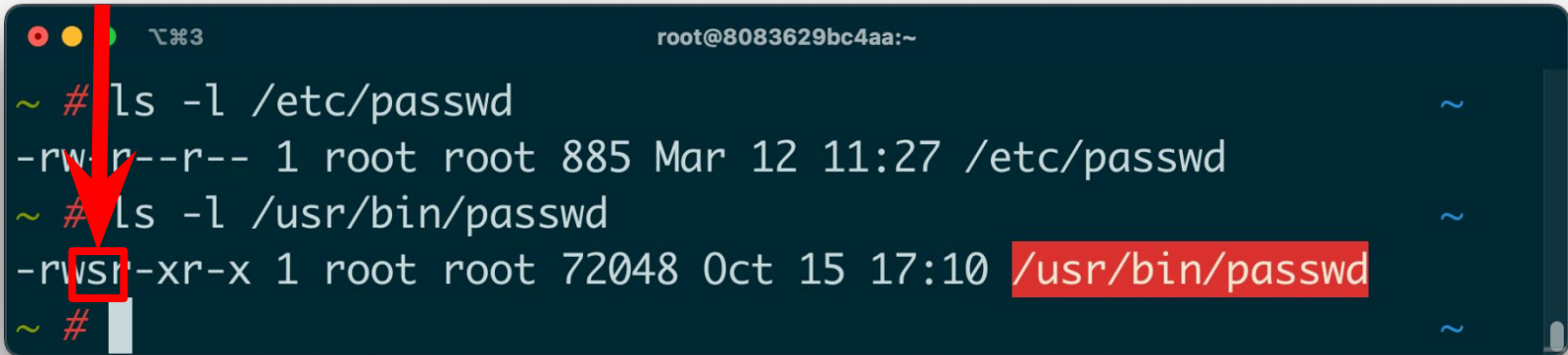
What if I want to change my password?

On linux, need to change /etc/passwd and /etc/shadow files

But, if I can edit these, I can change other folks' passwords, too
How do I change my own password?

Solution:

setuid binaries can be executed by a lower perm user



```
root@8083629bc4aa:~  
~ # ls -l /etc/passwd  
-rw-r--r-- 1 root root 885 Mar 12 11:27 /etc/passwd  
~ # ls -l /usr/bin/passwd  
-rwsr-xr-x 1 root root 72048 Oct 15 17:10 /usr/bin/passwd  
~ #
```

What could go wrong?

What happens if you exploit the setuid program?

Confused Deputy Problem

What happens if you exploit passwd?

Confused Deputy

Tricks a priv process into doing something

Inherent problem w/ DAC-only systems



Real world:
We combine MAC & DAC

Ex: Android!



Android Application Sandboxing: Goals

- Maximally Limit Untrusted Applications
- Apps can be written in any language
 - Cannot rely on PL primitives
- Apps *can* be malicious
- Also, System Decomposition:
 - E.g. Separating out media processing, wifi, bluetooth

Goal: Getting EIP on *one* component != Owning the OS

Android Application Sandboxing

DAC

- Standard Linux file permissions
- Each App has its own UID

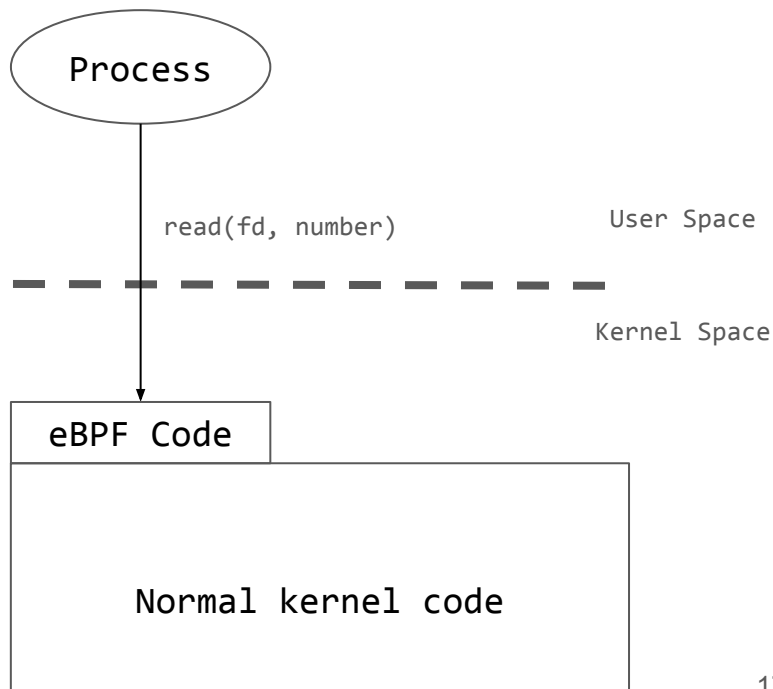
MAC == SELinux

- All processes on Android get a *context tag*
- At compile time, the kernel gets a configuration describing:
 - What files a process with each *context* has access to
 - What operations are allowed (e.g. read/write/execute)
- Everything else is **disallowed**
 - E.g. one could have a “root” process w/ access to nothing.

Android Application Sandboxing: Seccomp

Beyond MAC & DAC, there's SECCOMP!

- Goal: filter *specific* kernel calls
 - R/W/Open/IOCTL are too coarse
 - E.g. allow read but only for specific sections of a file
- eBPF
 - Non-turing complete language
 - Can “filter” specific inputs
 - Also, block
 - Injected in per-process basis





System Decomposition

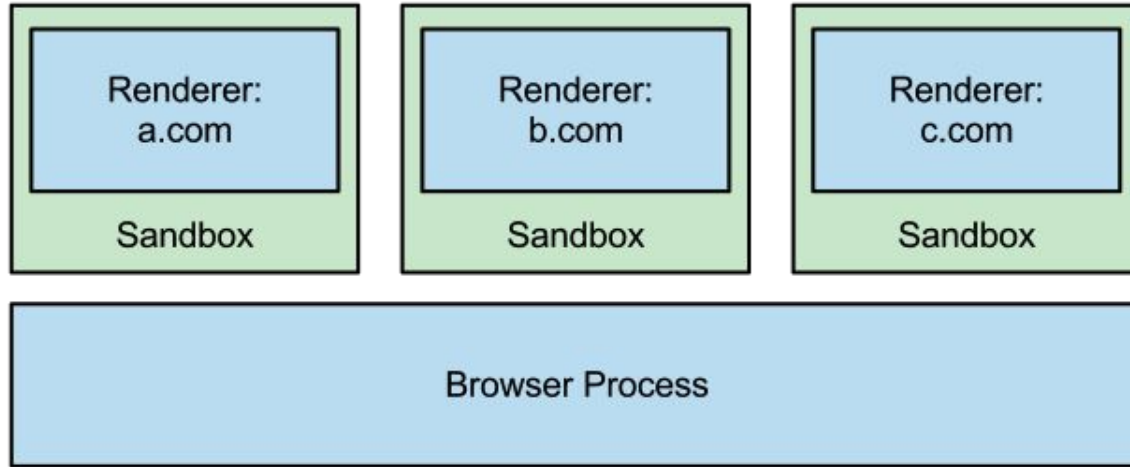
Theory: Make the attacker find multiple vulns

Example: Chrome:

- Renderer deals with a ton of untrusted code
 - Javascript
- For every new page:
 - Fork
 - Drop to a new limited UID
 - Apply new SEPolicy Context
 - Apply seccomp filter
 - Only now we render.



System Decomposition Result



Another Take: Capabilities

Theory:

- MAC system
- OS provides explicit access *keys*
- Keys give an invoked process the ability to perform an *action* on some specific *object*

The Confused Deputy

(or why capabilities might have been invented)

Norm Hardy
Senior Architect

Key Logic
5200 Great America Parkway
Santa Clara, CA 95054-1108

Compiler allows a flag for output

Wrote to things

A compiler would get {output_fd, RW},
the OS checks if the invoker (e.g. the user) has the right to give those capabilities

Aside: Finding bugs

- Fuzzing
 - Send random data to a program to cause it to crash
 - Many strategies (e.g. maximizing coverage of input, novelty, etc)
- Many systems to play with:
 - AFL, libfuzzer
- OS-level Fuzzing from userspace -> kernel space:
 - Syzkaller!
 - Current bugs: <https://syzkaller.appspot.com/>
- Learn how to fuzz:
 - <https://github.com/google/oss-fuzz>
- Smashing the Stack exercises:
 - <https://microcorruption.com/>
 - CSAW CTF
 - GT Club: Grey Hat, WREKCTF

A worked example:
Let's talk about elections!

What do we **want** from a voting system?

- Correctness & Usability
 - Counted as cast, cast as intended, (only) accessible to all eligible voters
- Privacy
 - An attacker cannot learn a voter's selections
- Coercion Resistance
 - Voter cannot cooperate with an attacker to prove the way they voted
- Receipt Freeness
 - No voter can prove the way they voted after the fact
- Verifiability
 - Voters have proof that their vote was counted correctly
 - End-to-End Verifiable (E2E-V) == crypto version





can my hus



can my husband find out who i voted for

How is voting done* today?

*ideally











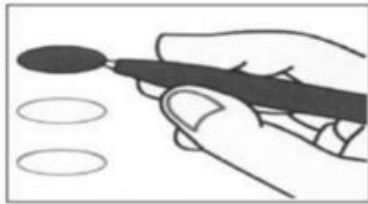






General Elections Process:

1. Print ballots; setup & testing
2. Casting & tallying
3. Optical scanners give initial (“reported”) outcome
4. **Statistical audit** of cast paper ballots
by hand to confirm/disprove reported outcome
5. Certification ceremony



CORRECT



INCORRECT



Cool, what can go wrong?

E.g. 2000 Election
Bush v. Gore

2000: Bush v. Gore

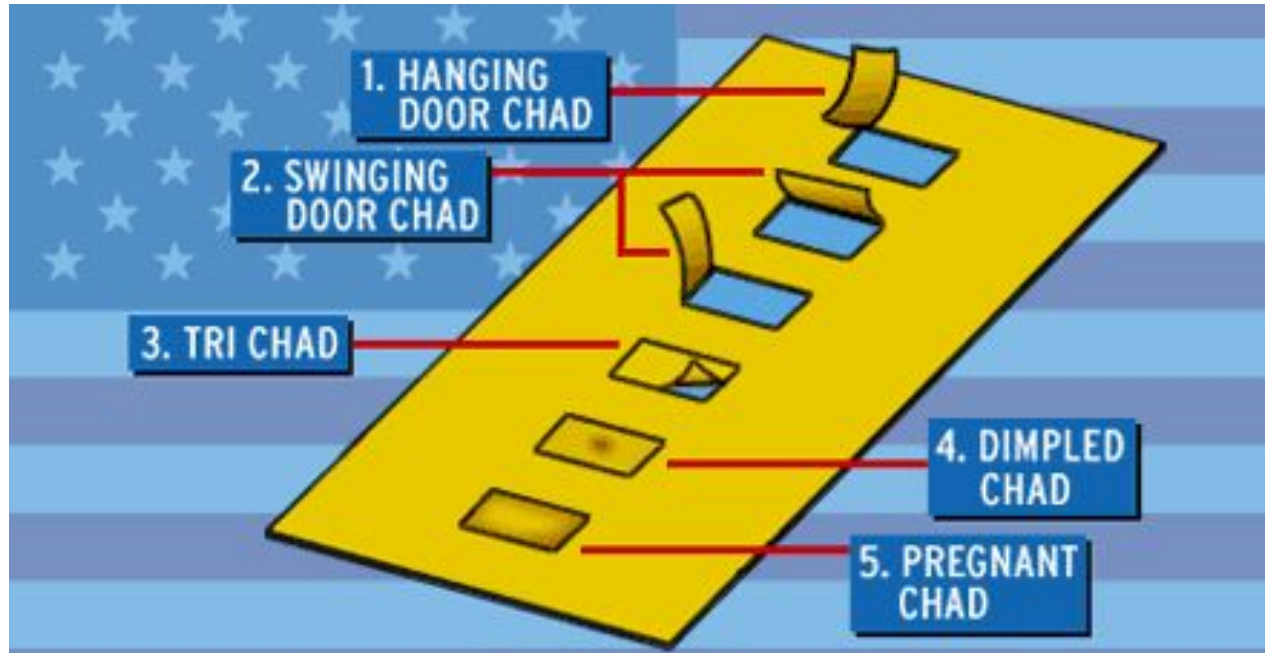
- Incredibly close race
- Came down to Florida
 - ~500 votes, .009 margin
- Many recounts and legal chicanery
- Ultimately wound up at SCOTUS

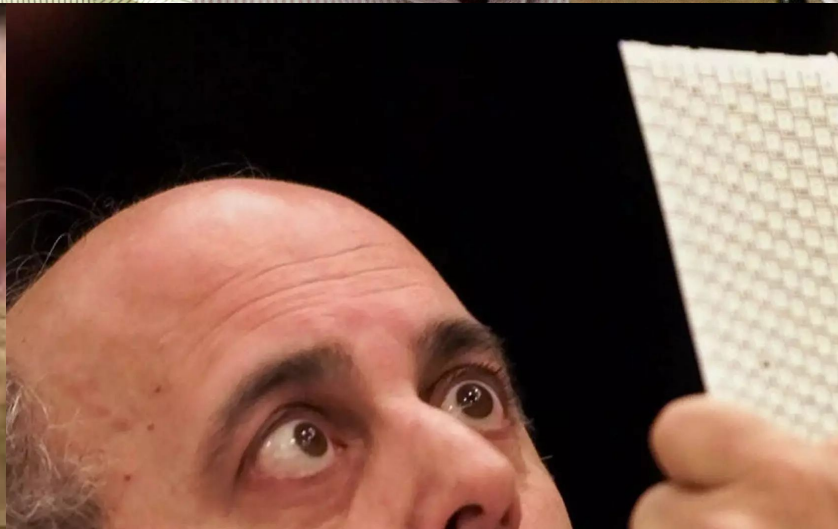
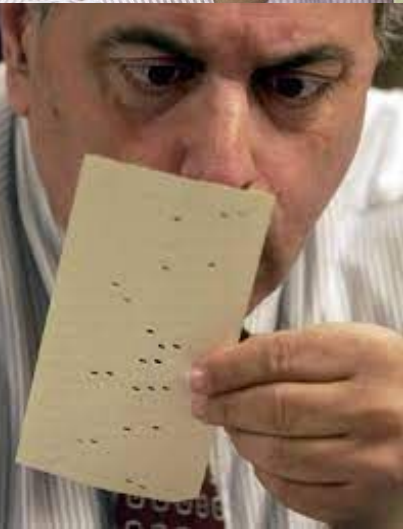
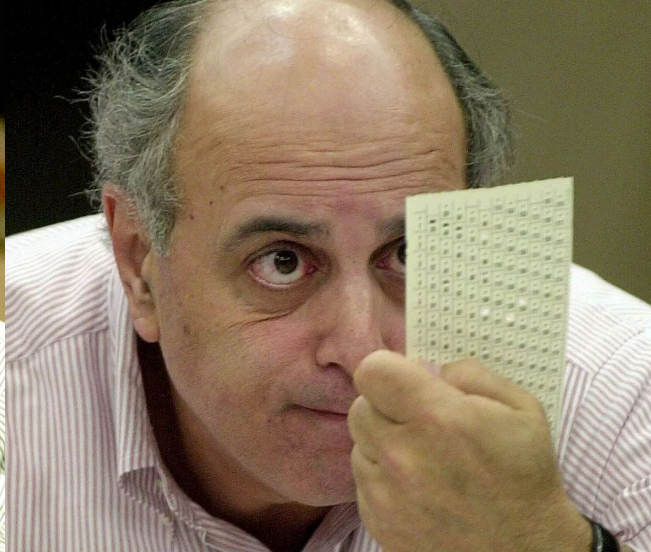
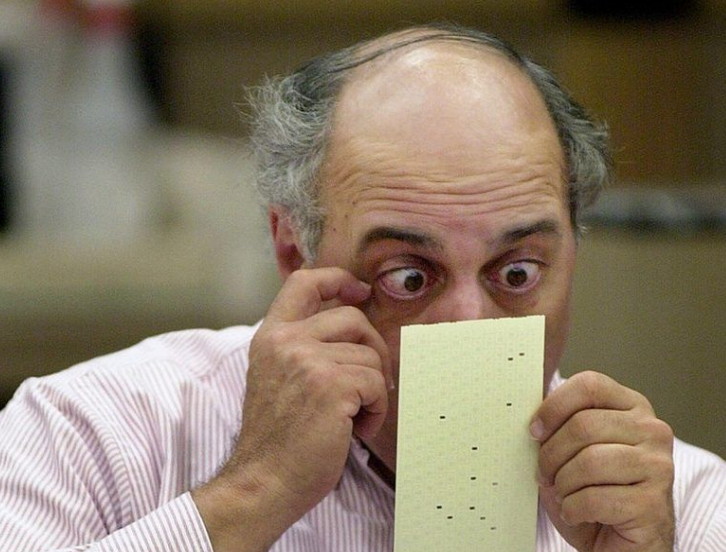
Turnout	70% ▲ ^[1]	
		
Nominee	George W. Bush	Al Gore
Party	Republican	Democratic
Home state	Texas	Tennessee
Running mate	Dick Cheney	Joe Lieberman
Electoral vote	25	0
Popular vote	2,912,790	2,912,253
Percentage	48.847%	48.838%

Usability Failure – ES&S Votmatic



Usability Failure – ES&S Votmatic (Result)







“I asked,
‘Can’t you find somebody else?’
They said ‘no.’”
– Judge Robert Rosenberg

Usability Failure – Who are you voting for?

1

OFFICIAL BALLOT, GENERAL ELECTION
PALM BEACH COUNTY, FLORIDA
NOVEMBER 7, 2000

**ELECTORS
FOR PRESIDENT
AND
VICE PRESIDENT**

(A vote for the candidates will
actually be a vote for their electors.)

(Vote for Group)

(REPUBLICAN)	3 ➔
GEORGE W. BUSH - PRESIDENT DICK CHENEY - VICE PRESIDENT	
(DEMOCRATIC)	5 ➔
AL GORE - PRESIDENT JOE LIEBERMAN - VICE PRESIDENT	
(LIBERTARIAN)	7 ➔
HARRY BROWNE - PRESIDENT ART OLIVIER - VICE PRESIDENT	
(GREEN)	9 ➔
RALPH NADER - PRESIDENT WINONA LaDUKE - VICE PRESIDENT	
(SOCIALIST WORKERS)	11 ➔
JAMES HARRIS - PRESIDENT MARGARET TROWE - VICE PRESIDENT	
(NATURAL LAW)	13 ➔
JOHN HAGELIN - PRESIDENT NAT GOLDHABER - VICE PRESIDENT	

A

OFFICIAL BALLOT, GENERAL ELECTION
PALM BEACH COUNTY, FLORIDA
NOVEMBER 7, 2000

4 ←	(REFORM) PAT BUCHANAN - PRESIDENT EZOLA FOSTER - VICE PRESIDENT
6 ←	(SOCIALIST) DAVID McREYNOLDS - PRESIDENT MARY CAL HOLLIS - VICE PRESIDENT
8 ←	(CONSTITUTION) HOWARD PHILLIPS - PRESIDENT J. CURTIS FRAZIER - VICE PRESIDENT
10 ←	(WORKERS WORLD) MONICA MOOREHEAD - PRESIDENT GLORIA La RIVA - VICE PRESIDENT
WRITE-IN CANDIDATE To vote for a write-in candidate, follow the directions on the long stub of your ballot card.	

Readings

Intro to Security

- Original Paper Intro:
Smashing the Stack for Fun and Profit
- Memory Corruption History:
SOK: Eternal War in Memory

OS Primitives for Security

- System Decomp.:
Building Secure High-Performance Web Services with OKWS
- Android:
The Android Platform Security Model (2023)
- Capabilities:
Security in KeyKos (1986)

Final thoughts

1. Security (as a discipline) can be fun and impactful!
2. Goals, threat model, policies
 - a. Kirchoff's principle
3. Designing for security can be subtle
4. Mechanisms can and will fail, learn to design around them

Many more courses in security @ GT! New thread!

specter@gatech.edu // mikespecter.com // spdr.cc.gatech.edu

Fall 2026: gatech.fail

Spring 2027: gatech.vote

THE OS AND NETWORKING

ALEXEY TUMANOV

School of Computer Science
faculty.cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

ADMINISTRATIVA



- **Lab3 – Checkpoint**
 - Due: THIS Fri, 3/20
 - You will be 3/5 weeks in
 - 1 week of Spring Break, 2 weeks to lab3 deadline remaining
 - Full lab3 due date: Fri Apr 3
- **Lab2 grades** → will be released by tomorrow, Wed 3/18
- **Exam 1 grades** → will be released by tomorrow, Wed 3/18
- **Withdrawal deadline:** Wed 3/18

March 18 (Wed)

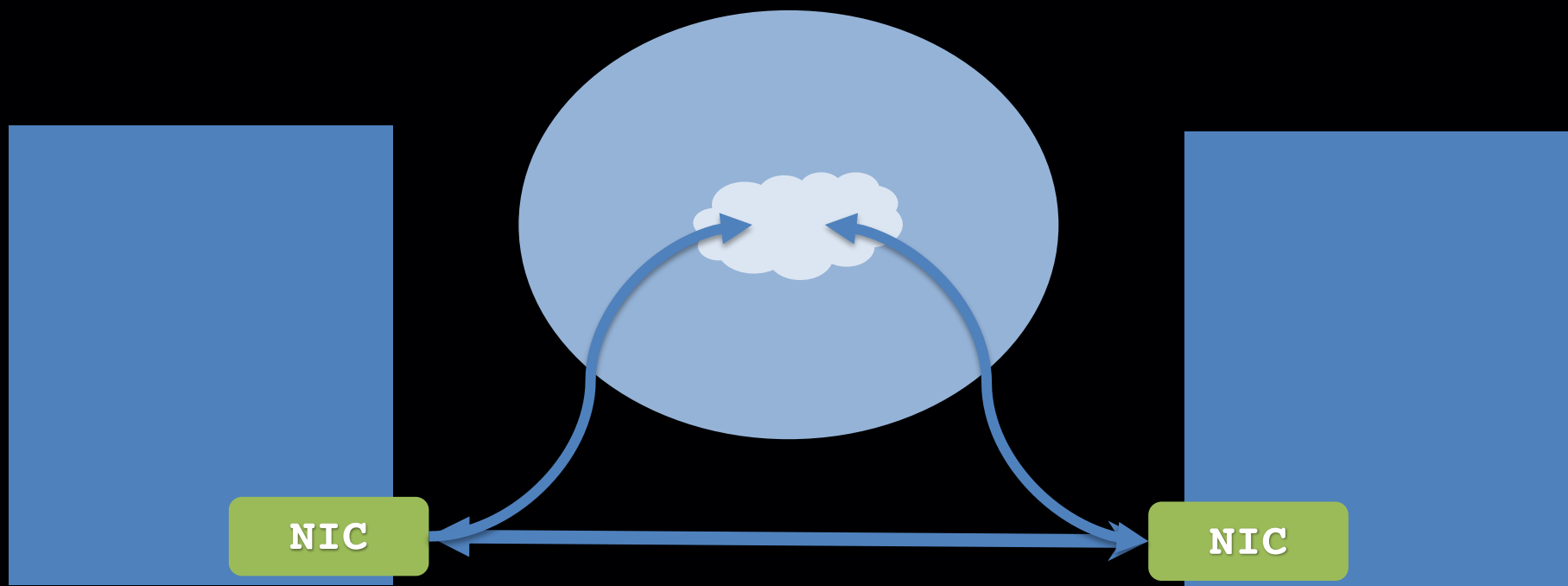
Spring

Registration

Withdrawal deadline. Last day to withdraw from a single course or from school with W grades for Spring Semester 2026 by 11:59 PM Eastern Time. Students must withdraw from all classes in order to receive a refund.

- **Lab4 networking option!**
 - network driver, network enabled xv6 apps! (stay tuned for more)
 - Possibly eBPF (end of this lecture)

What does our networking hardware look like to the OS?



Description of the basic network card

ORTHOGONALITY AND COMPOSABILITY



- How do we go from having NIC $\leftrightarrow$ NIC point to point communication mechanism
- And build a world wide anywhere to anywhere communication mechanism?
- It is essential for the building blocks to have two fundamental properties:
 - **Orthogonality** : *Orthogonality in a programming language means that a relatively small set of primitive constructs can be combined in a relatively small number of ways to build ...*
 - **Composability**: *a system design principle that deals with the inter-relationships of components. A highly composable system provides components that can be selected and assembled in various combinations to satisfy specific user requirements*

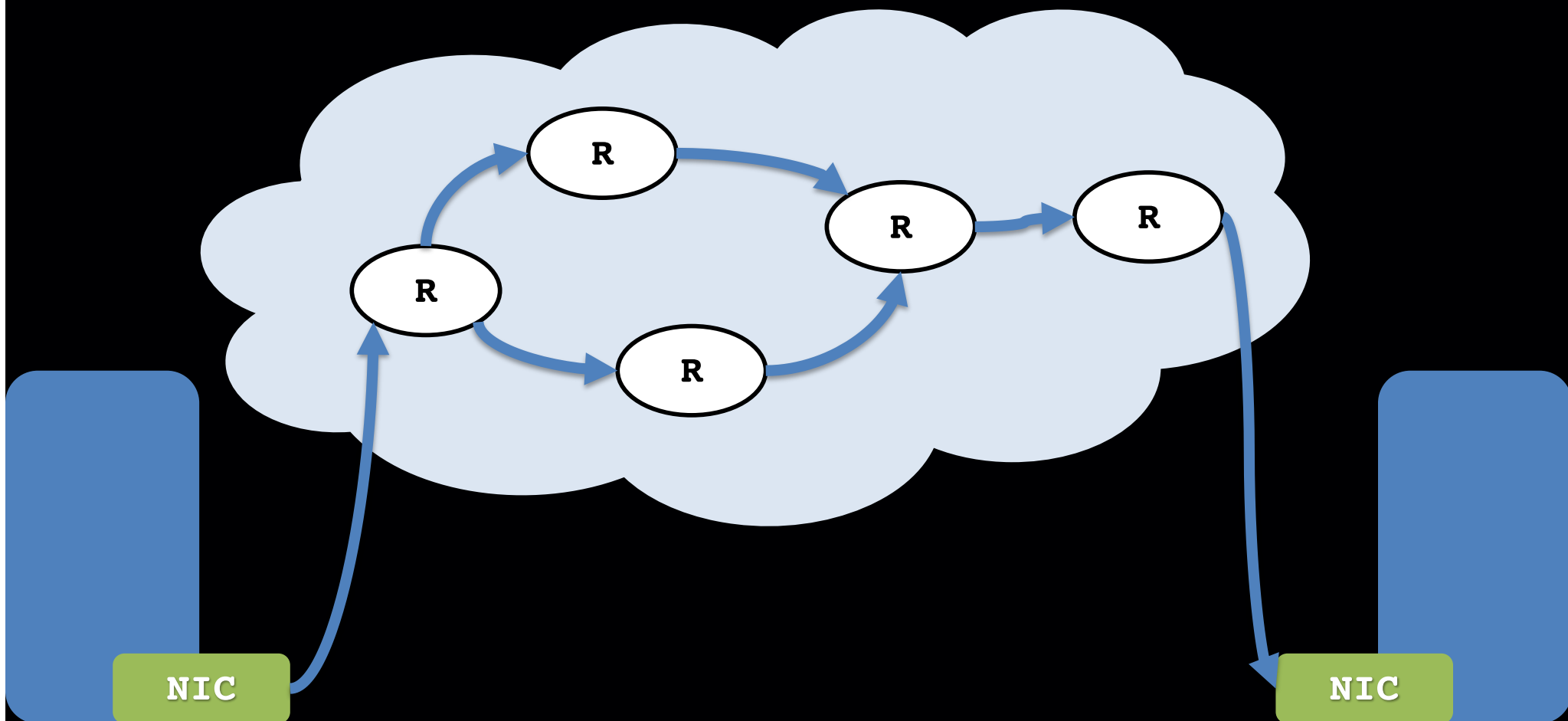
Source: [https://en.wikipedia.org/wiki/Orthogonality_\(programming\)](https://en.wikipedia.org/wiki/Orthogonality_(programming))
<https://en.wikipedia.org/wiki/Composability>

What does our networking hardware look like to the OS?

Point to point

How does this differ from the “internet” abstraction we provide to OS users/applications?

NETWORKING ABSTRACTIONS



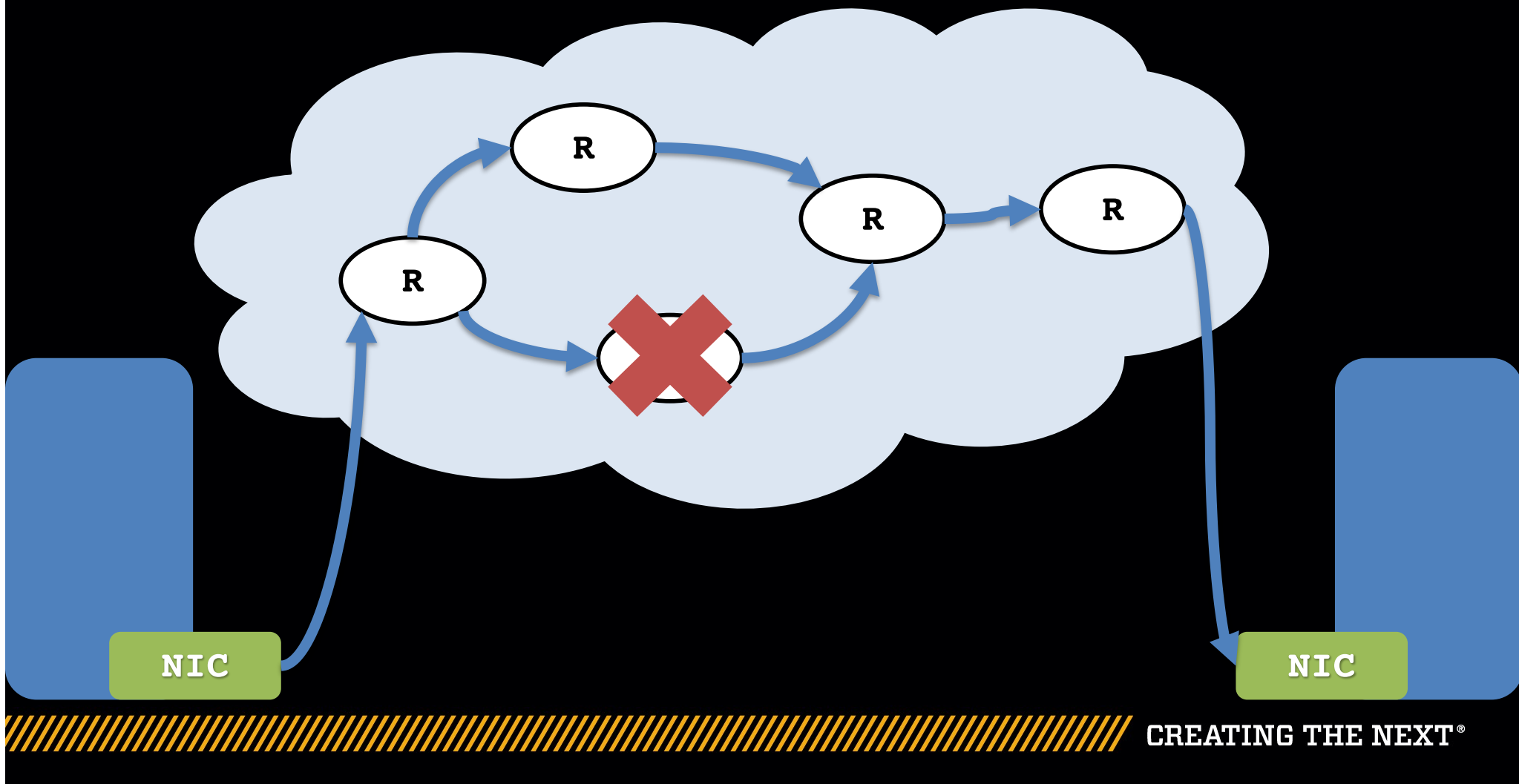
What does our networking hardware look like to the OS?

Point to point

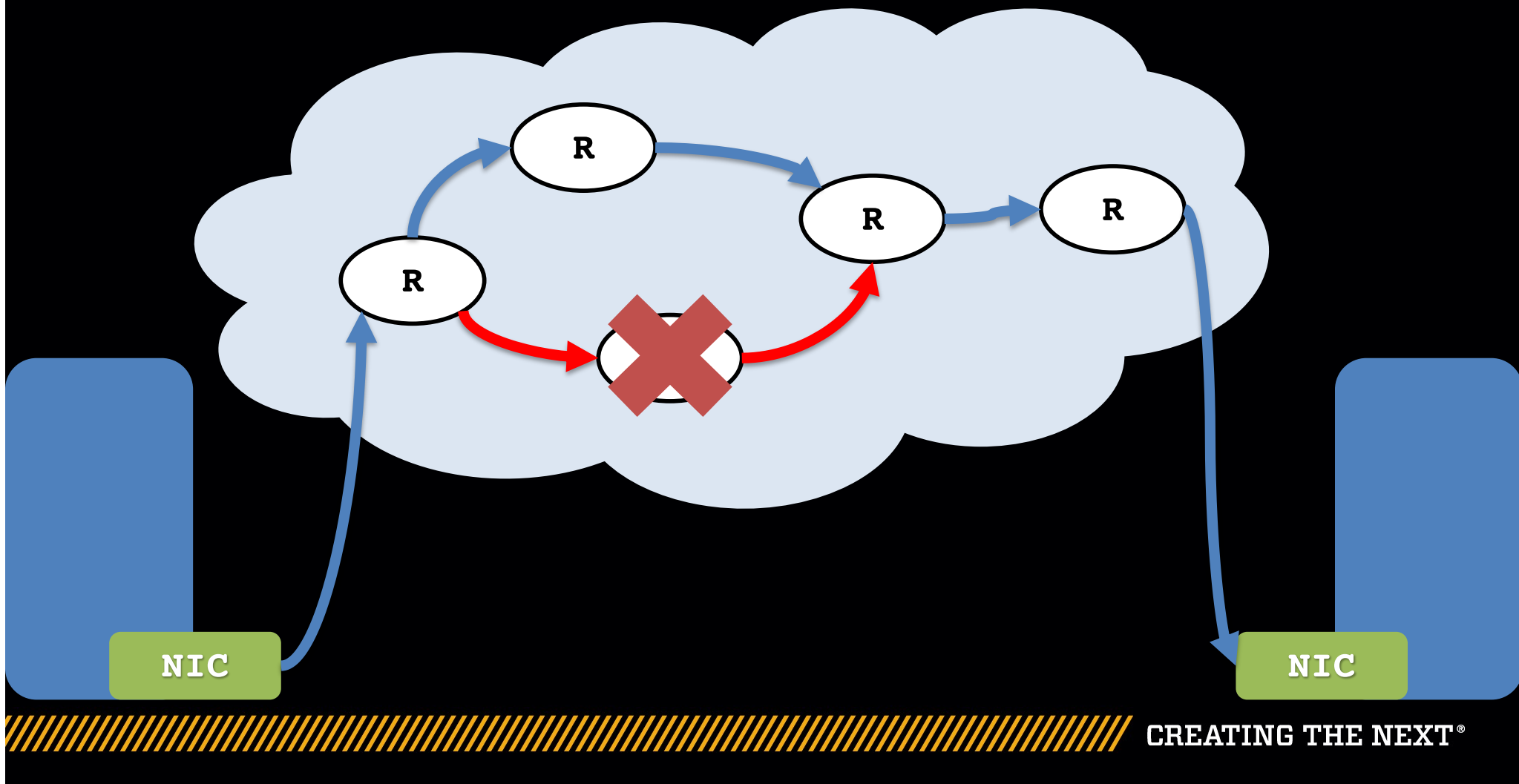
How does this differ from the “internet” abstraction we provide to users?

- url/domain name (Global addressing, naming)
- Point-to-point communication
- Multiplexing (1 NIC, many connections)
 - 1 NIC, many ports
 - Many connections on each port
- Streaming (semantically relevant data, concept of flow/stream)
- Reliability (surviving failures) (not trivial)

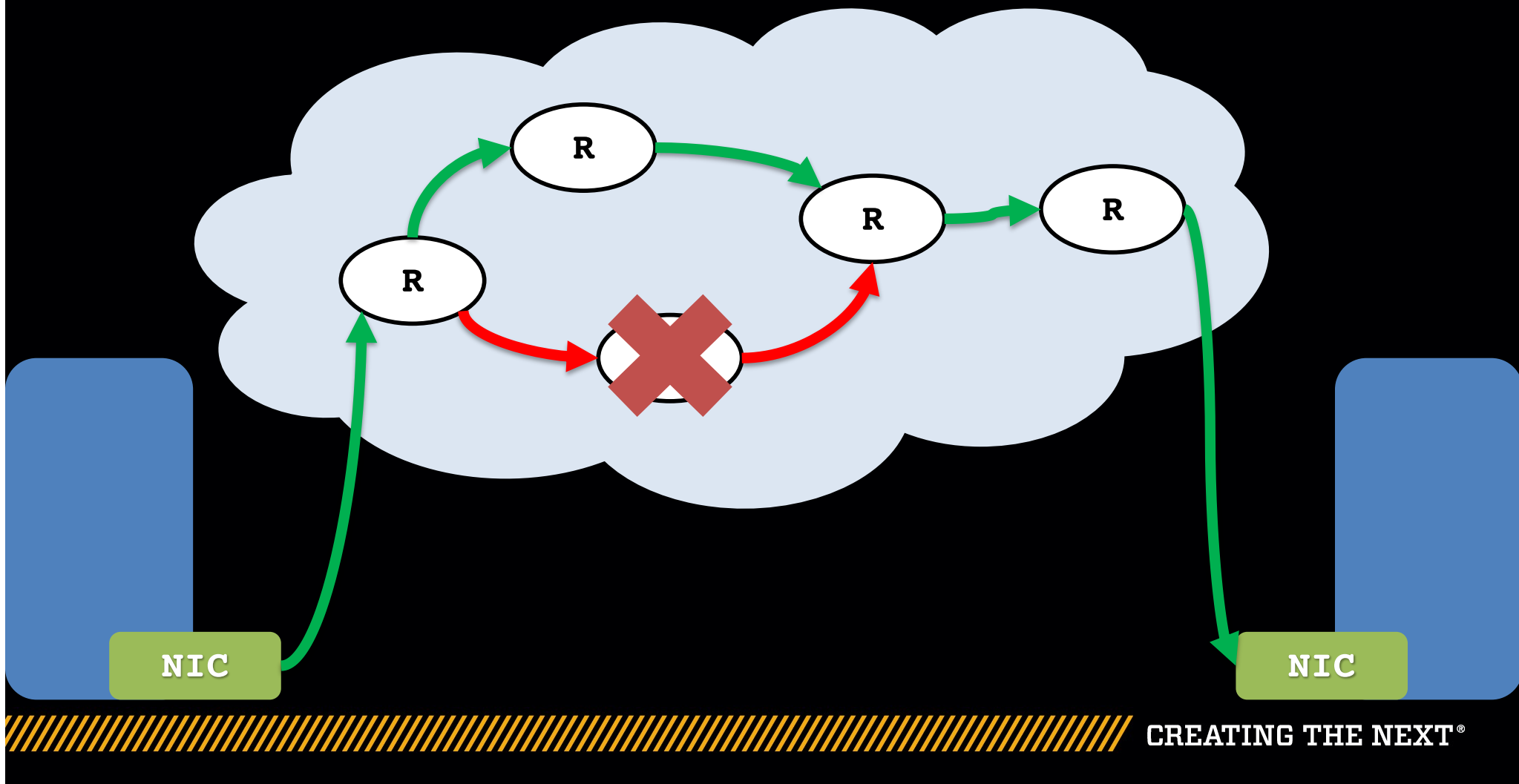
NETWORKING ABSTRACTIONS: FAULTS



NETWORKING ABSTRACTIONS: FAULTS



NETWORKING ABSTRACTIONS: FAULTS



What does our networking hardware look like to the OS?

Point to point

How does this differ from the “internet” abstraction we provide to users?

- url/domain name (Global addressing, naming)
- Point-to-point communication
- Multiplexing (1 NIC, many connections)
 - 1 NIC, many ports
 - Many connections on each port
- Streaming (semantically relevant data, concept of flow/stream)
- Reliability (surviving failures) (not trivial)

naming

reliability

isolation

multiplexing

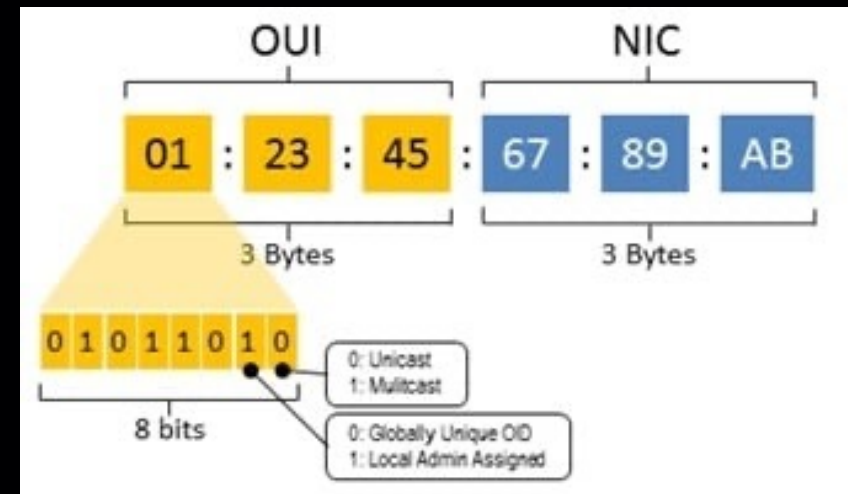
protection

NETWORKING: HARDWARE PROVIDES



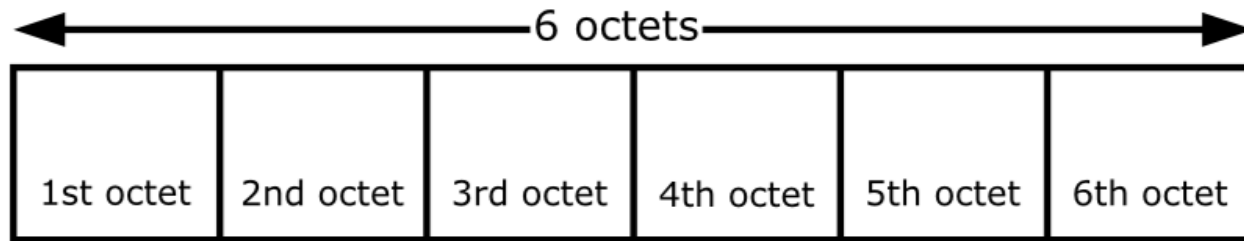
Networking Hardware provides:

- Provides a single network interface
- Unreliable transport (reorders packets, drops data)
- Communicate to MAC address
- No Security (TBD Later)

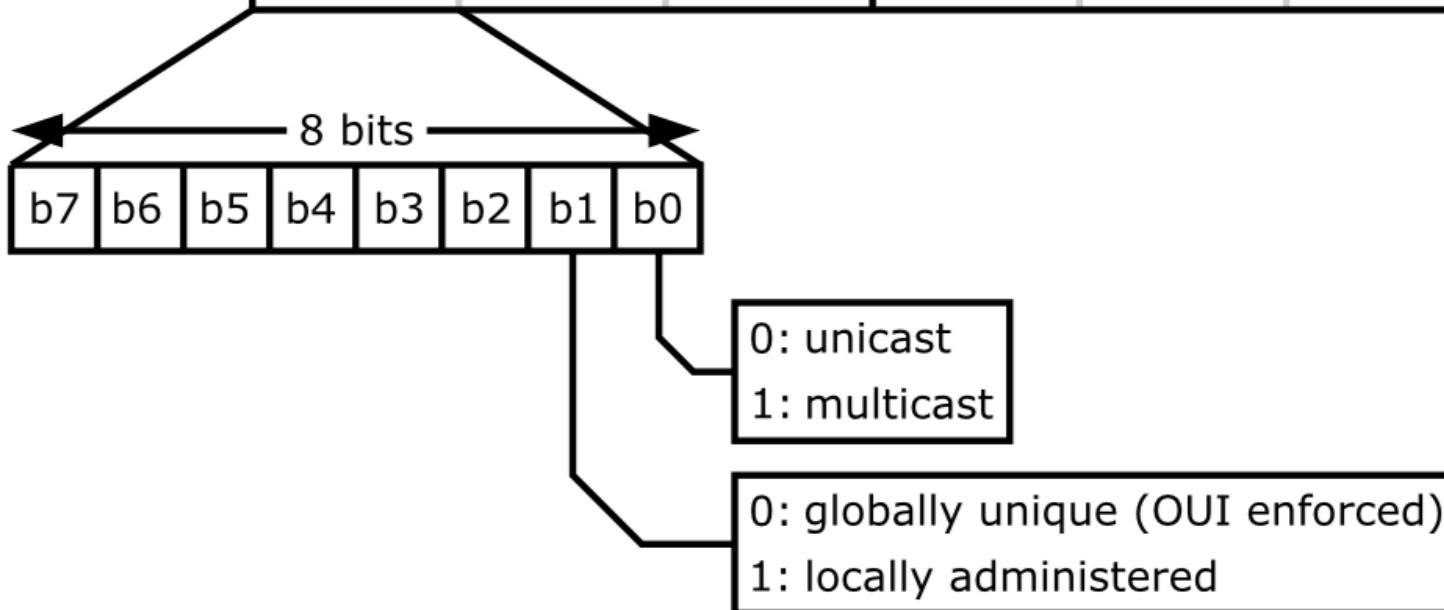
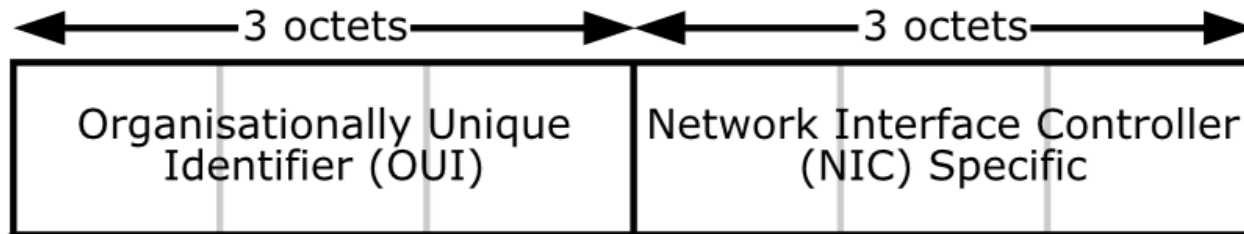


Bare bones, frame based, unreliable p2p comm system

DATA LINK



or



NETWORKING: OS PROVIDES



OS provides:

- *Reliable* transport
- *Naming*: Hostname communication (e.g. gatech.edu)
- *Multiplexing*: Many “ports” per network card
- Fine-grain *multiplexing*: Many connections per port
- *Protection*: Secure transport layers (kind of)

GAP between the network and OS provided abstractions:

- This lecture/class
- Networking stack

How do we build these abstractions?

Networking abstractions are built in layers.

(7 by OSI standards)

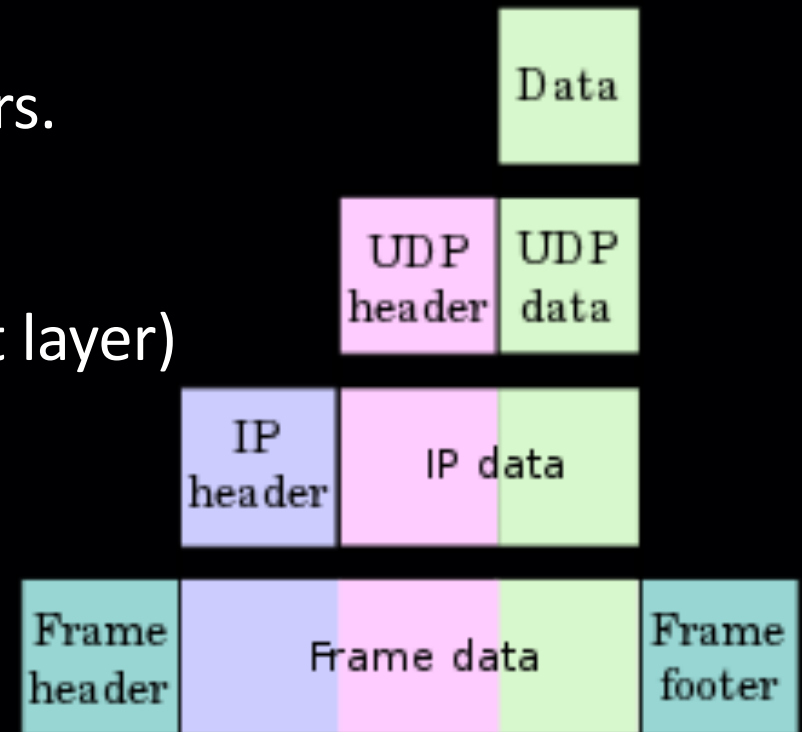
Physical – (e.g. ethernet/fiber, lowest layer)

Data Link – (e.g. IEEE 802.3)

Network – (e.g. IP)

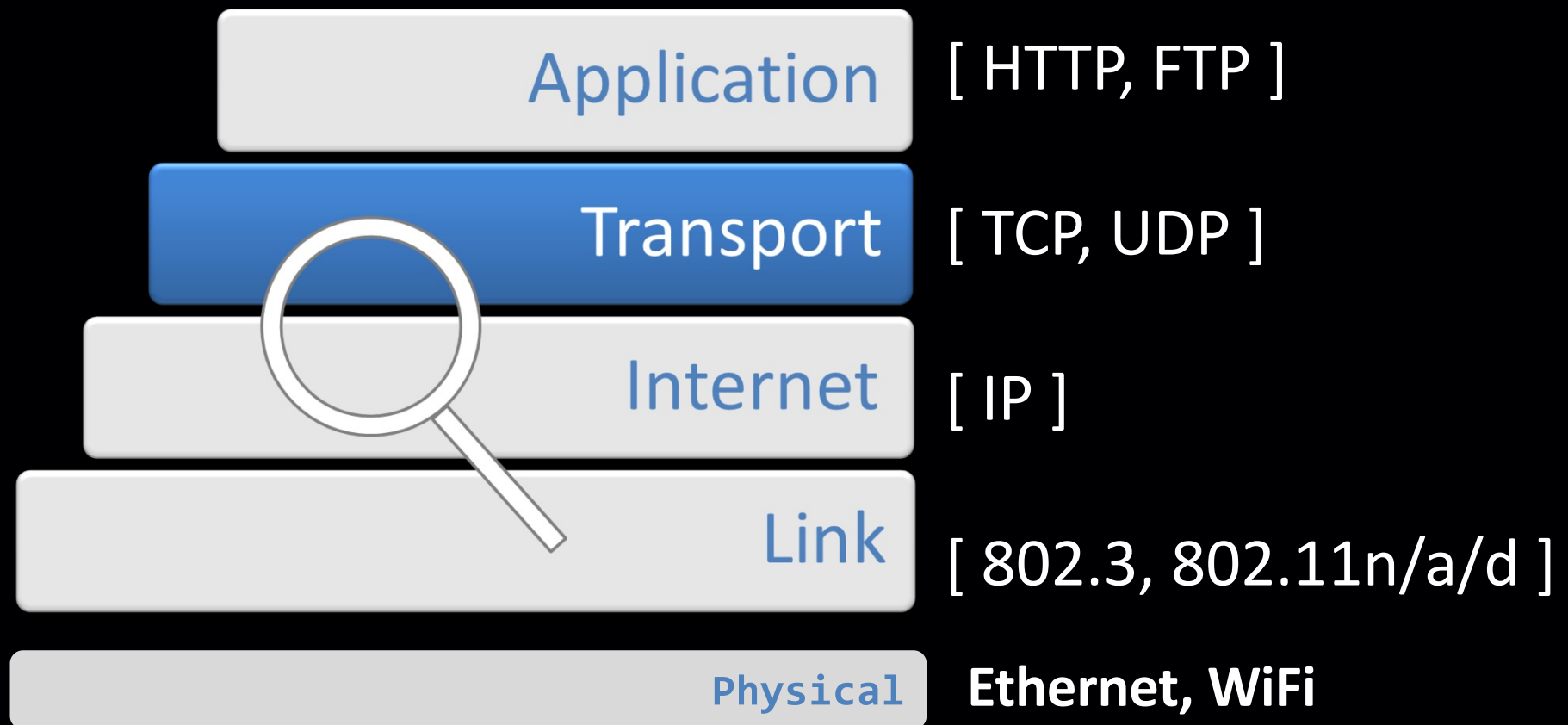
Transport – (e.g. UDP, TCP)

& Higher levels (we won't worry about now)



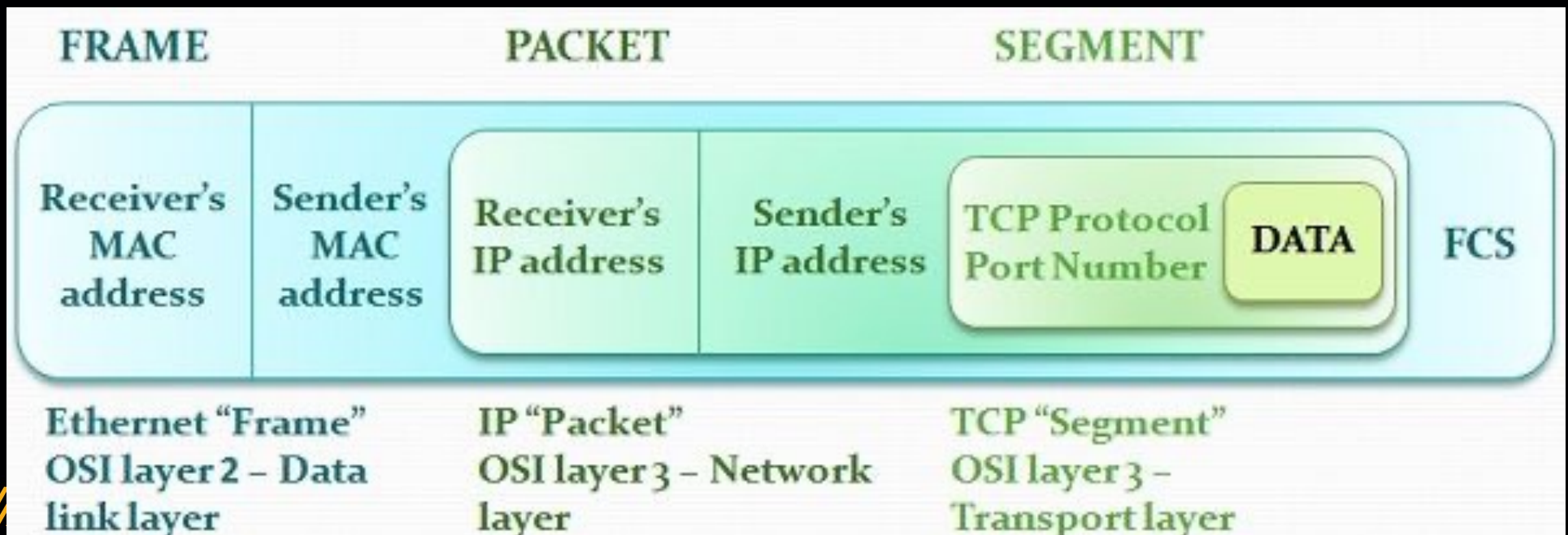
NETWORKING STACK

OSI 7 layer model / 3 network layer + 1 application layer



HOW DO THESE LAYERS WORK

- (Not fully accurate) – Protocol == Header
- NICs process PHY/MAC
- OSes process Network & Transport [typically]



End-to-end network stack propagation

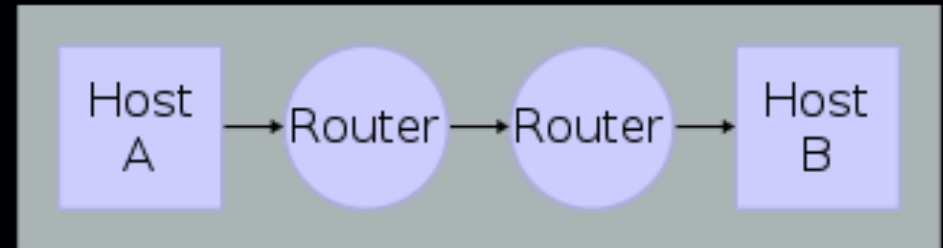
Link layer communicates
on a single link
(e.g. Ethernet cable)

Internet layer manages
multiple hops across links

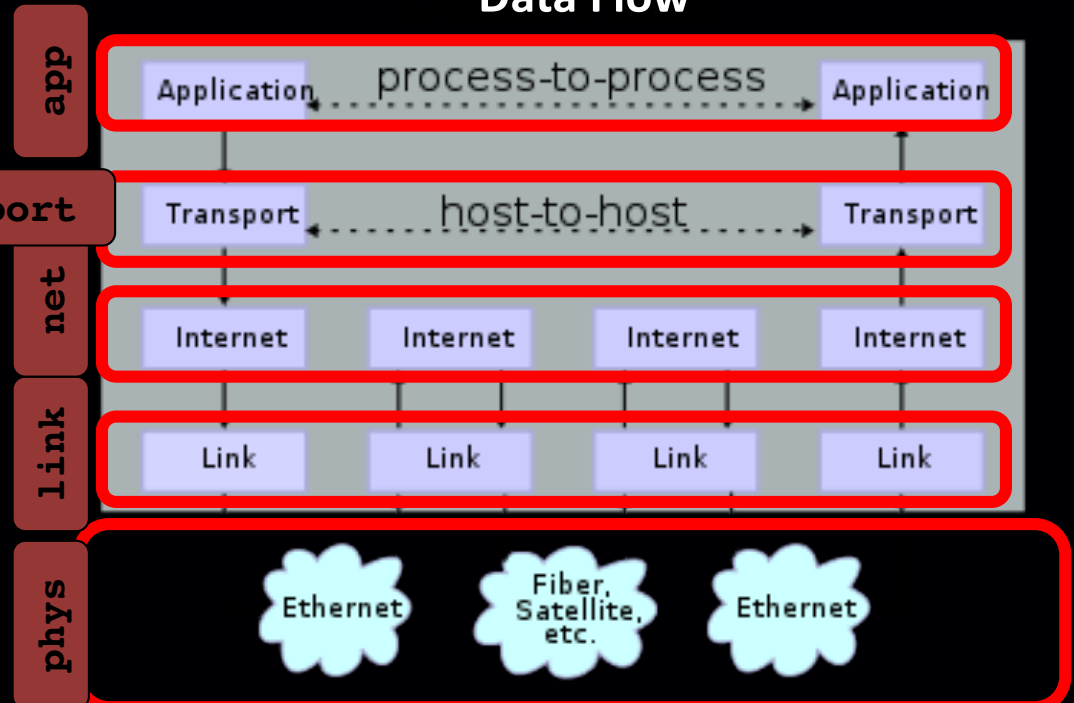
- Host addressing (IP)
- Routing

Transport layer manages
data reliability

Network Topology



Data Flow



End-to-end network stack propagation

Link layer communicates
on a single link
(e.g. Ethernet cable)

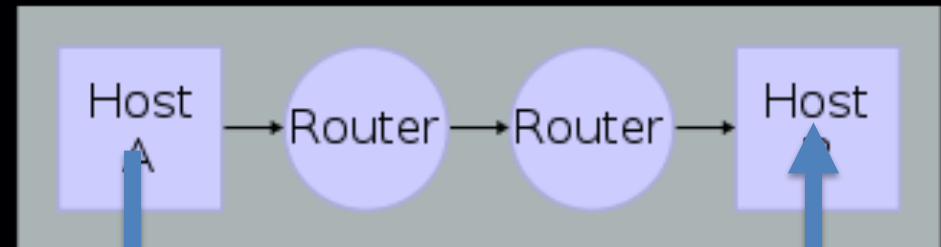
Internet layer manages
multiple hops across links

- Host addressing (IP)
- Routing

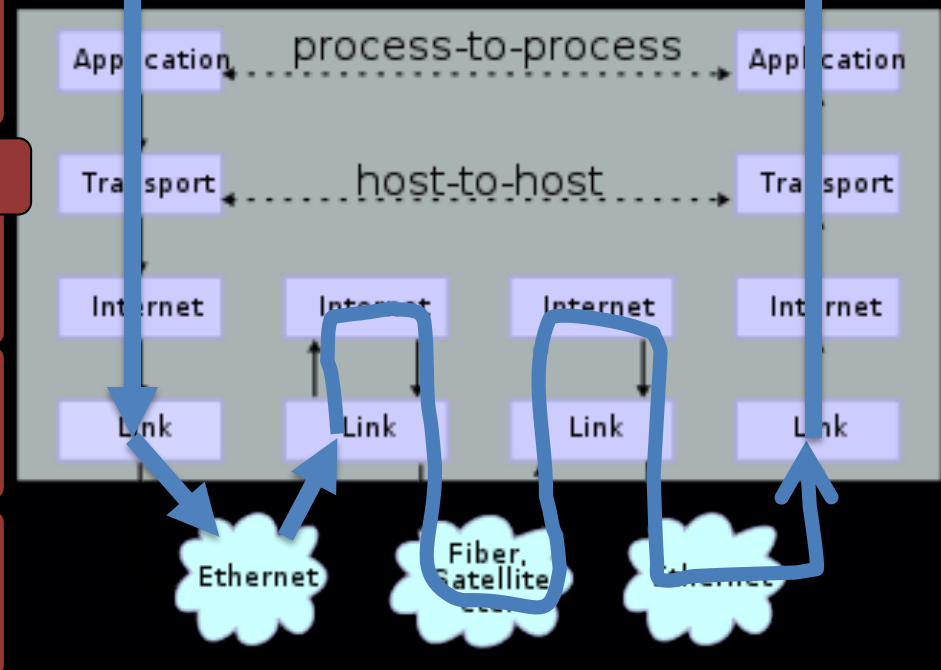
Transport layer manages
data reliability

app
transport
net
link
phys

Network Topology



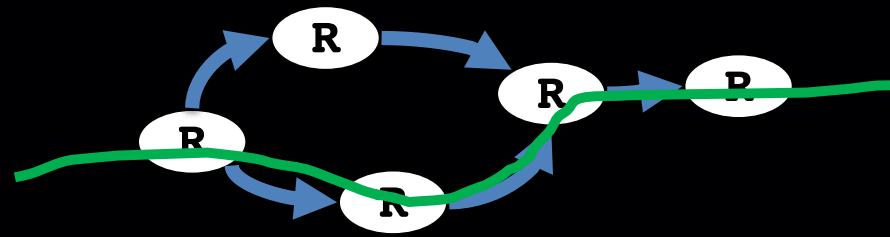
Data Flow



Internet Layer (L3):

Internet layer manages multiple hops across links

- **Host addressing (IP)**
- **Routing**



How do we construct the *mechanism* of IP addresses?

How do we route?

**Essentially a large-scale distributed system,
take Networking to cover in-depth**



Transport Layer (L4):

Layer of particular interest to OS!

Let's look at TCP:

Abstractions Provided (Goals):

Stream interface (data comes in a stream, not packets)

Reliability (packets aren't lost, or reordered)

Point-to-point (Direct point-to-point communication)

Transport Layer (L4):

Layer of particular interest to OS!

How do we provide a *stream* interface??

We're given packets– (typically 1500 bytes)

How do we make this look like a *stream* of arbitrary data writes?

- Use multiple packets per write, or put multiple writes in a packet
- But, packets can be reordered or dropped, how do we handle?
What happens if a packet is dropped?

How do we make packets look like stream of arbitrary data writes?

From packets to streams

- multiple writes per packet / multiple packets per write

Packets can be **out of order**

- Sequence number

Packets can be **dropped**

- Retransmission
- Timeout mechanism

- *Detection* mechanism
 - Ability to detect that a certain failure modality has occurred
- *Correction* mechanism
 - Ability to correct detected failure modality:
 - Handling duplicates (slow collection, router died, resend)
 - Handling reordering
 - Handling dropped packets
- *Timeout* mechanism

TCP/UDP: TRANSPORT TRADEOFFS (L4)



TCP Provides data *reliability*

- Byte-stream interface
- No errors
- No duplicates
- No drops

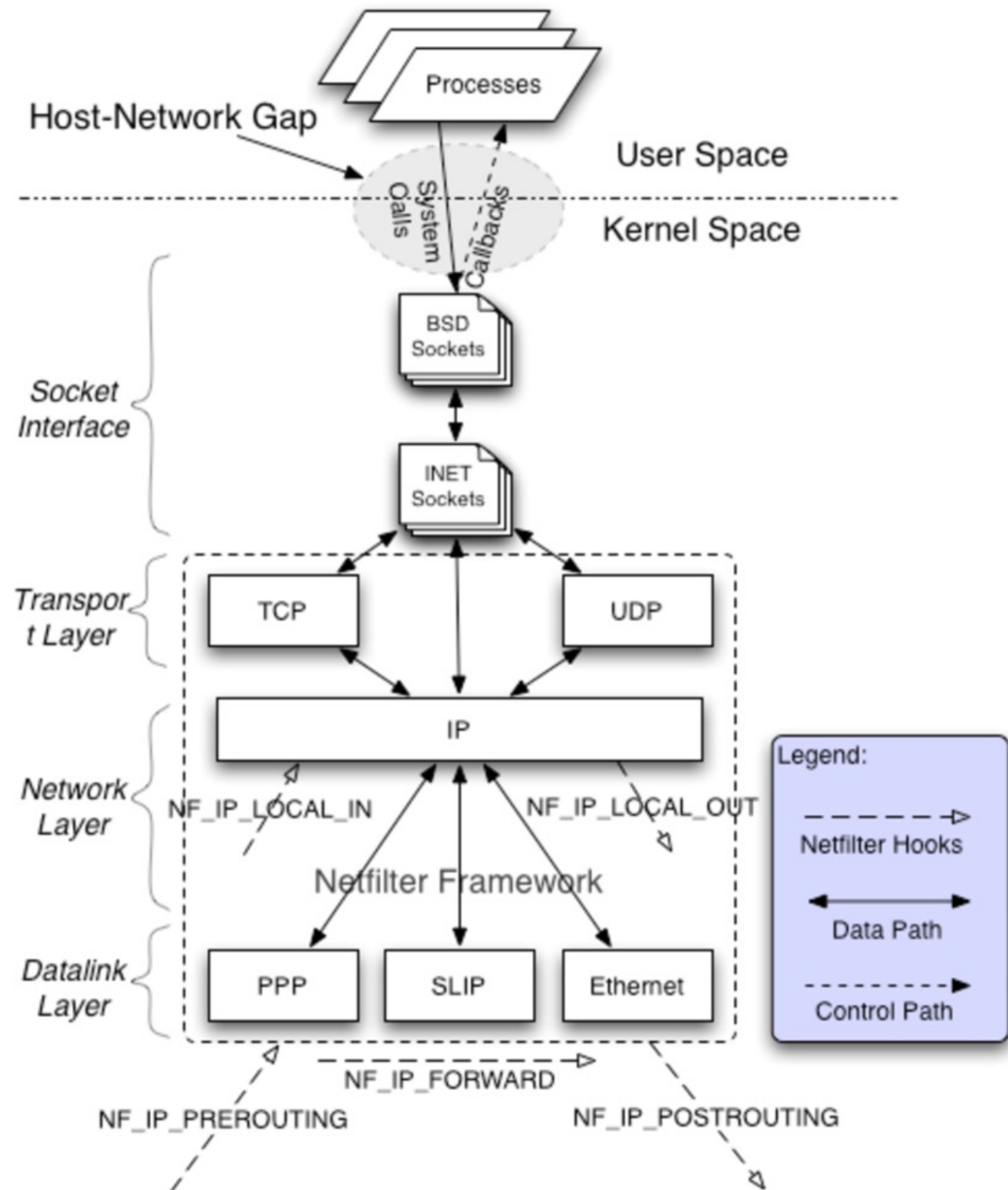
UDP is an alternative to TCP

- Provides only “no data errors” via checksum
- Why/when is UDP useful?

TCP/IP IN LINUX

Socket Interface

Netfilter framework



PACKET RX/TX IN LINUX

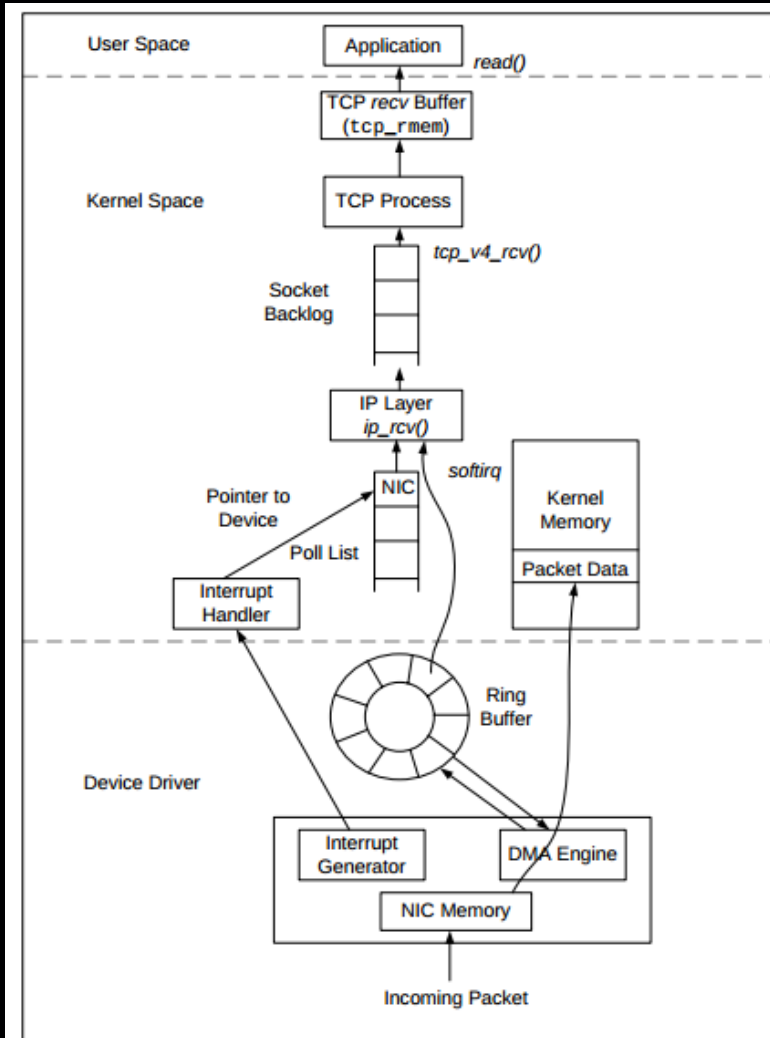


Fig. 1: Packet Reception

PACKET RX/TX IN LINUX

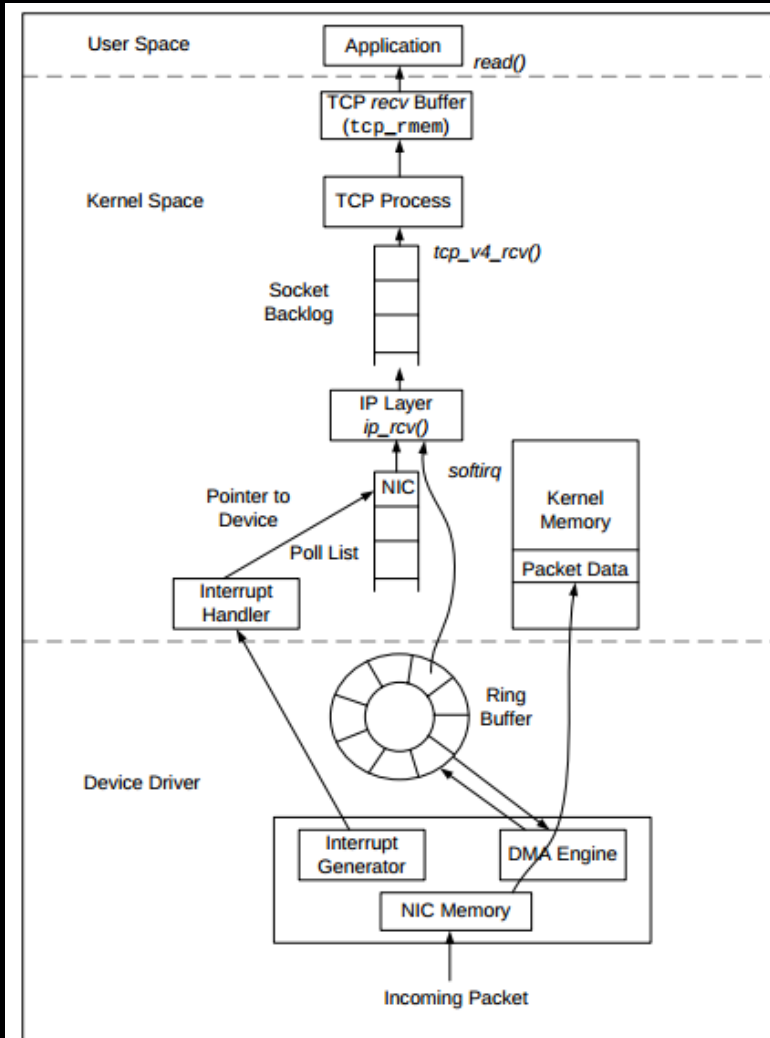


Fig. 1: Packet Reception

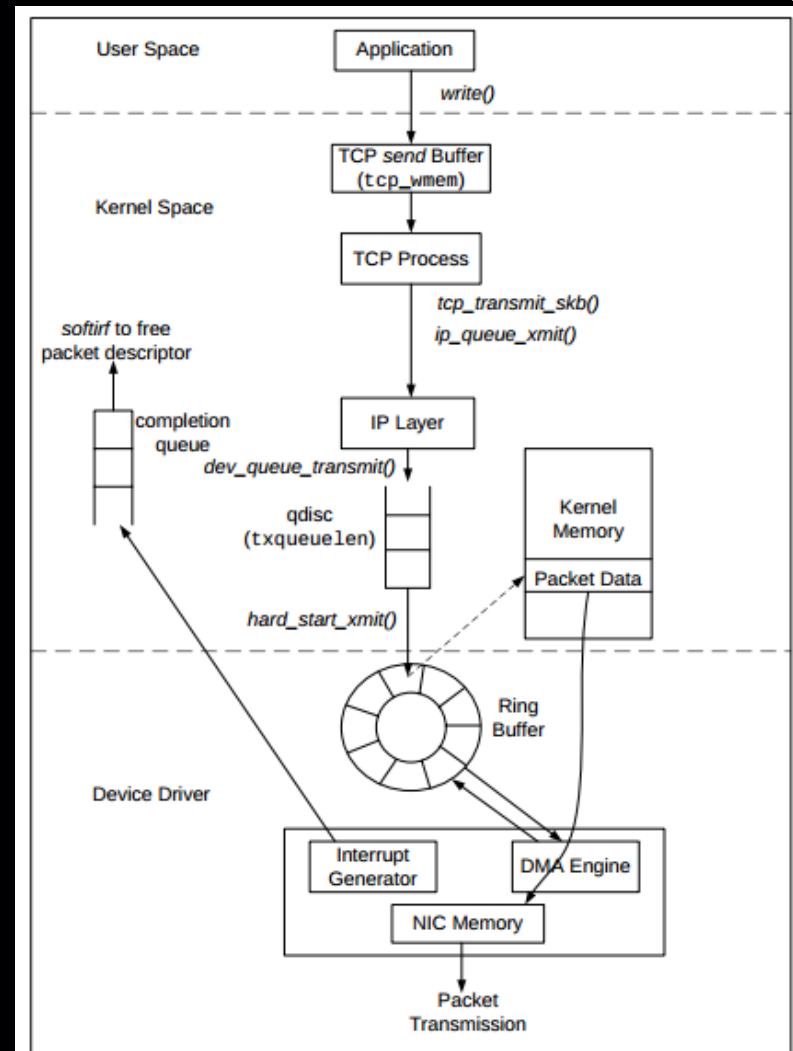


Fig. 2: Packet Transmission

EXTENDED BERKELEY PACKET FILTER (EBPF)

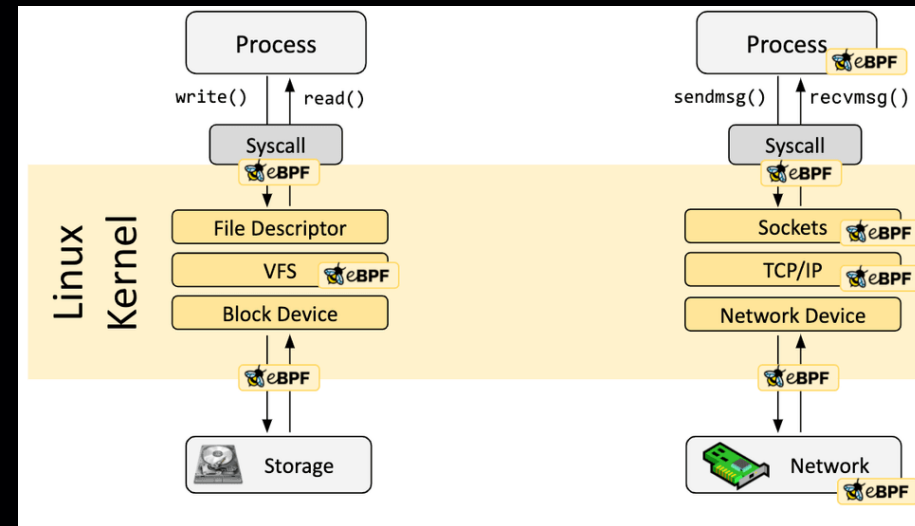


Problem: pain to write custom kernel modules

- Can also have security/robustness issues
- But...want high performance!

Solution: enable sandboxed programs to run within the OS

- User writes program (e.g., network tracing) and compiles into eBPF bytecode
- In-kernel verifier checks for code safety
- Bytecode is JIT'ed and attached to specific kernel events (e.g., system calls)



<https://ebpf.io/what-is-ebpf/#hook-overview>

EXTENDED BERKELEY PACKET FILTER (EBPF)



Example: want to block all IPv6

How it's done today in the kernel:

- Allocate memory for each packet, pass it up the network stack to check the IP
- **Problem:** can be slow – premise of DDoS attacks!

Solution with eBPF:

- Write eBPF app that checks for blocked IPs
- "Hook" into NIC (XDP)
- Trigger upon packet arrival, discard IPv6 packets at the driver level

```
// Attach to network driver
SEC("xdp")
int filter_ipv6(struct xdp_md *ctx) {

    // Get packet header
    void *data      = (void *) (long) ctx->data;
    struct ethhdr *eth = data;

    /*
     * Do OOB and other validation checks
     */

    // Discard IPv6
    if (is_ipv6(eth)) {
        return XDP_DROP;
    }
    else {
        return XDP_PASS;
    }
}
```

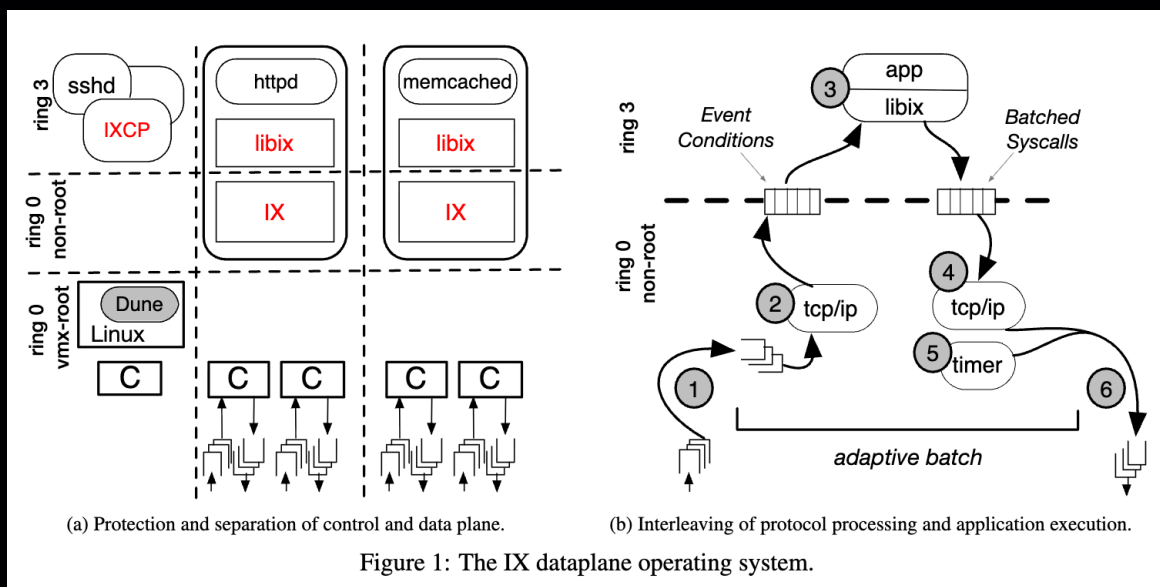

OS AND NETWORKING – ONLY SCRATCHING THE SURFACE!

There is *a lot* more to networking

- In general and OS-specific

Out-of-scope of this class:

- VM/container networking
- Software-defined networks
- Advanced NICs
- ...



IX [OSDI'14]

FILE SYSTEMS

ALEXEY TUMANOV

CREATING THE NEXT®

ADMINISTRATIVA: LABS



- Lab3 due this Friday, Apr 3!
- This is probably the best lab to spend your late days on
 - Lab3: 20% weight vs lab4: 10% weight
- Lab4 to be released this Fri, Apr 3
 - Lab4-fs (default) : 2 weeks and due 4/17
 - Lab4-net will be available: 3 weeks and due 4/24
 - Will announce the ***bonus*** structure for lab4-net
 - Grading scheme will be backward compatible
- Exam 2: Tue, 4/28, 6:15-9pm (supervised lab locations)
 - Only 4 weeks away!
 - Focus: FS, Ordering and atomicity, Distrib. Systems
 - Exam 2 prep lectures : Thu 4/16 and Tue 4/28

Mar 31 LEC 20: File Systems 1 Reading: XV6 Chapter 6	Apr 01	Apr 02 LEC 21: File Systems 2
Apr 07 LEC 22: Ordering and Atomicity 1	Apr 08	Apr 09 LEC 23: Ordering and Atomicity 2
Apr 14 LEC 24: Distributed Systems	Apr 15	Apr 16 LEC 25: Research
Apr 21 LEC 26: Exam 2 Prep	Apr 22	Apr 23 LEC 27: Course conclusion
Apr 28 LEC 28: Exam 2 Prep finale Exam 2 Time: 6:30pm to 9:15pm ET	Apr 29	Apr 30

What is a Filesystem?

Why do we need a file system?

Does a filesystem have to relate to disk?

Does a filesystem need any secondary storage??

Manage user
data

Multiplex
storage
resources

storage
resource
isolation

storage
resource
protection

Data
management

What is a Filesystem?

Why do we need a file system?

Does a filesystem have to relate to disk?

Does a filesystem need any secondary storage??

*Manage user
data*

*Multiplex
data
resources*

*data
resource
isolation*

*data
resource
protection*

*Data
management*

OS services provided by File Systems

- *Isolation!*
- *Protection! (Granularity matters)*
 - Protecting different entities at different levels of the stack (users, files, blocks)
- *Multiplexing (spatio-temporal construct)*
 - Space multiplexing: ability to use different parts of a resource simultaneously
 - Time multiplexing: ability to *order* access to resource over time correctly
- *Naming*
 - Higher level abstractions that create a user-friendly interface to access data

What is a Filesystem?

Wikipedia:

“A filesystem controls how data is stored or retrieved”

- File System (FS) is an abstraction to manage data in general
 - Not tied to persistent data
 - FS controls access to the data (any data)
 - Naming and organizing data
 - Sharing data between multiple users and multiple processes
 - Data can be read-only

Manage State

- Naming, Protection, Isolation:
 - FS : useful mechanism for accessing devices (e.g., camera, /dev)
 - Interface that broadly generalizes to a variety of data I/O scenarios

- File System (FS) is an abstraction to manage state in general
 - Used to export, or make accessible, the data to user space
 - Through a narrow-waist interface! (This is Sparta!!!)
 - FS interface adopted for managing linux control plane (procfs)
 - Performance counters exposed via the file system
 - Tileria TilePro64 mesh NoC with peripheral memory controllers

FS Lecture Overview

File systems

API (Design choices)

Disk layout

Buffer cache

xv6 in action - code walk

What abstractions do *disk-based* filesystems provide?

Naming

Byte-level interfaces (instead of block/sector)

Protection/Isolation (user accesses)

Concurrency (file locks – flock)

Multiplexing – One disk, many files

Data sharing – many users can access one file

IPC?

What abstractions do disk-based filesystems provide?

Naming

Byte-level interfaces (instead of block)

- Façade : translation of dev-native to process-native interfaces
- Ability to hide complexity:
 - Heterogeneity of storage devices
 - Device availability and failure!
- Ability to hide topology (e.g., local vs. remote)
- “Look and feel” of memory

What other than persistent storage are filesystems good for?

Device interfacing (/dev/)

(cameras, ram access, raw disk access, serial interfaces)

System metadata (/proc and /sys)

Information about processes, the cpu, memory, kernel configuration!

Intuitive interface to interface with I/O devices

What other than persistent storage are filesystems good for?

Information about NUMA zones

Information about PCI devices (net lab)

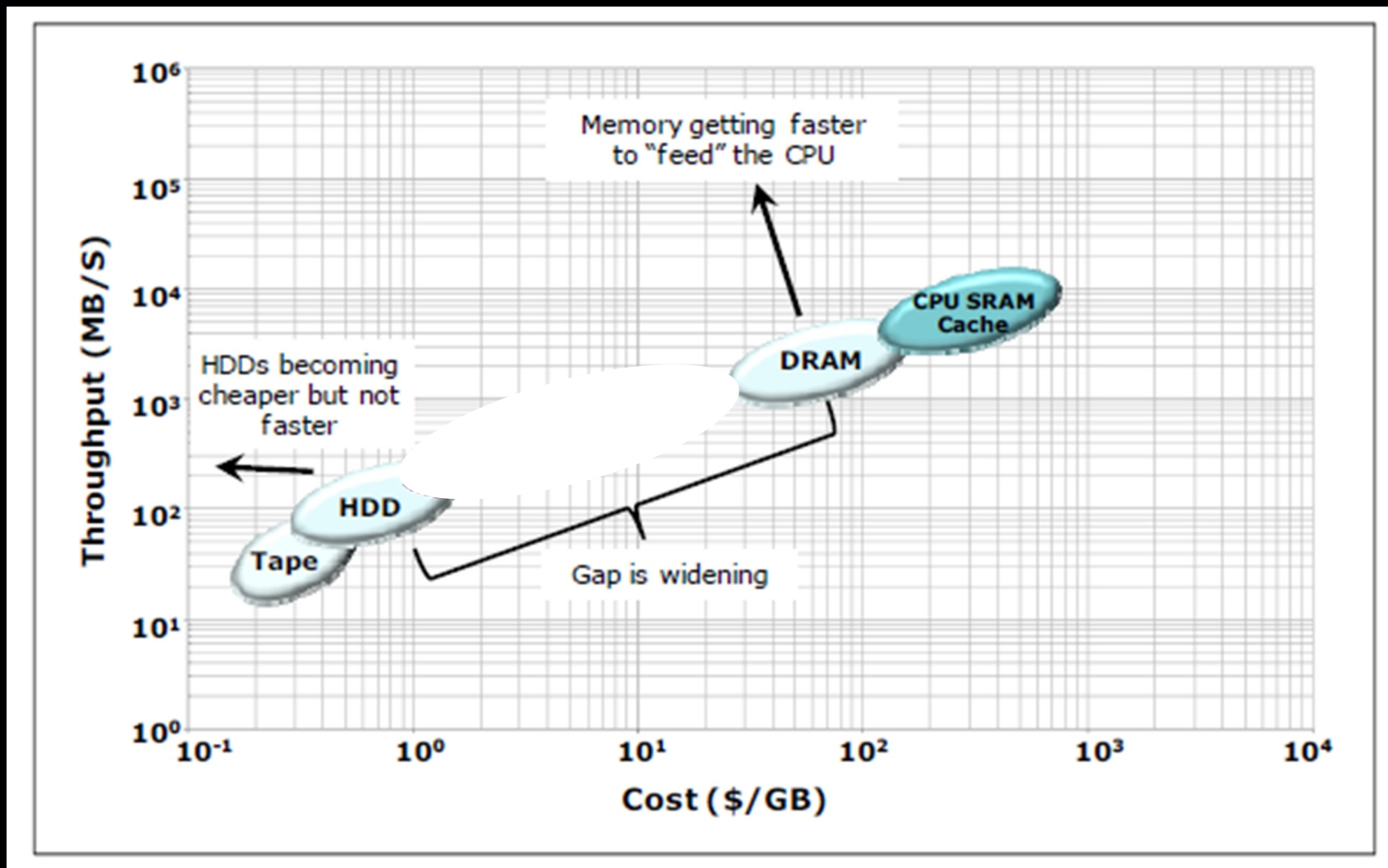
Controlling hugepage allocation

Process information: open file descriptors (lsof)

Takeaway: file system  persistent storage

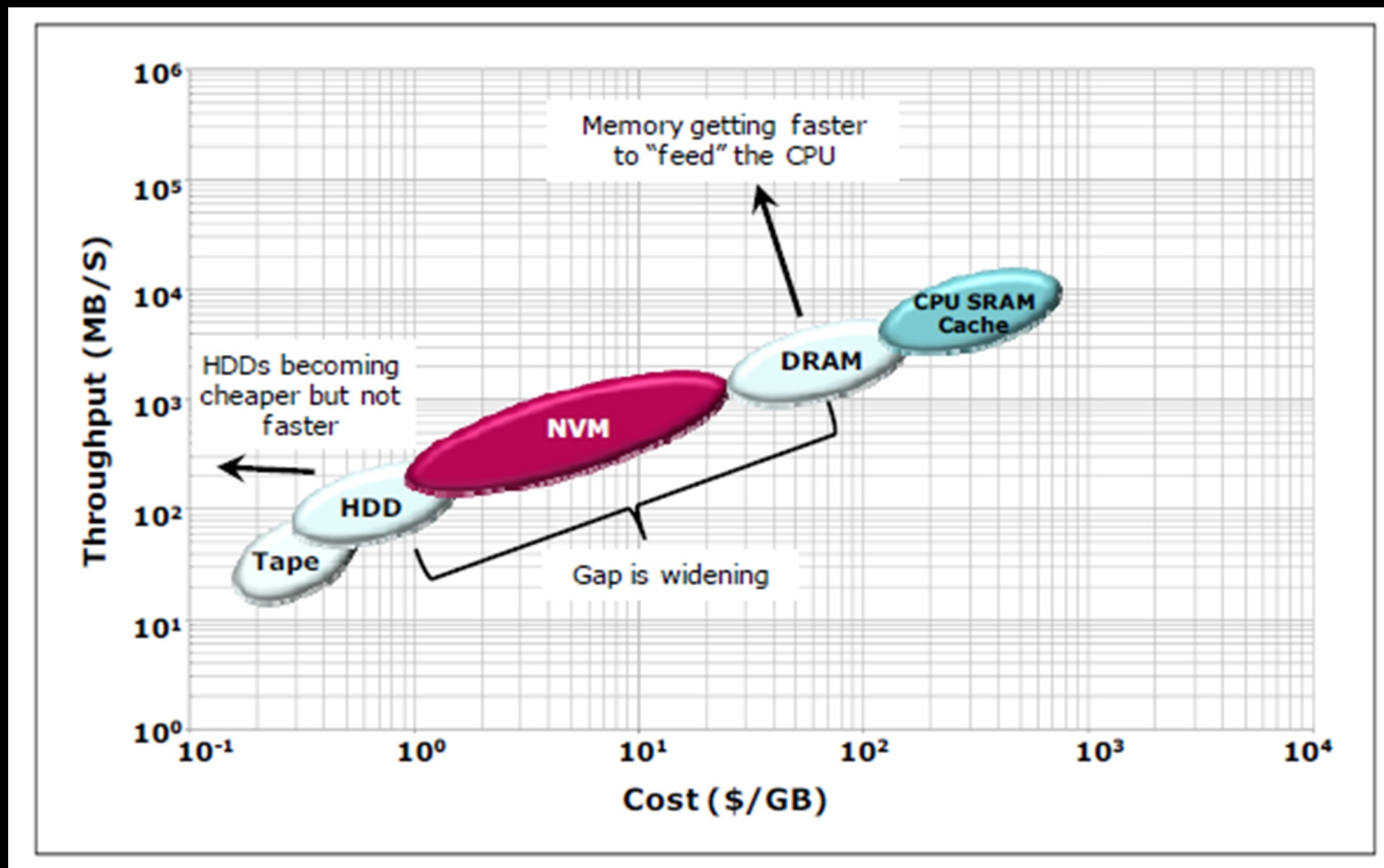
FILE SYSTEMS

Storage Trends

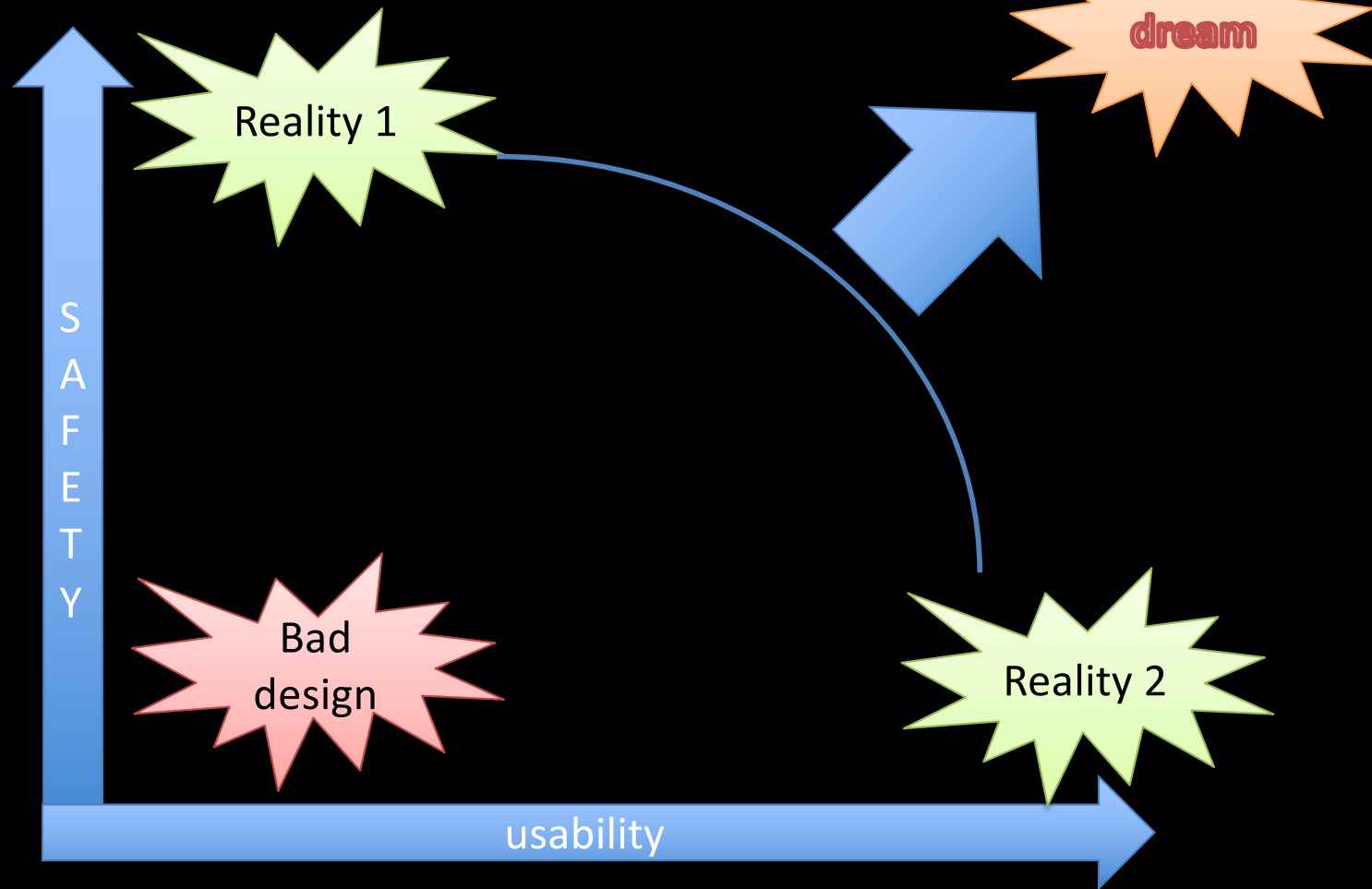


FILE SYSTEMS

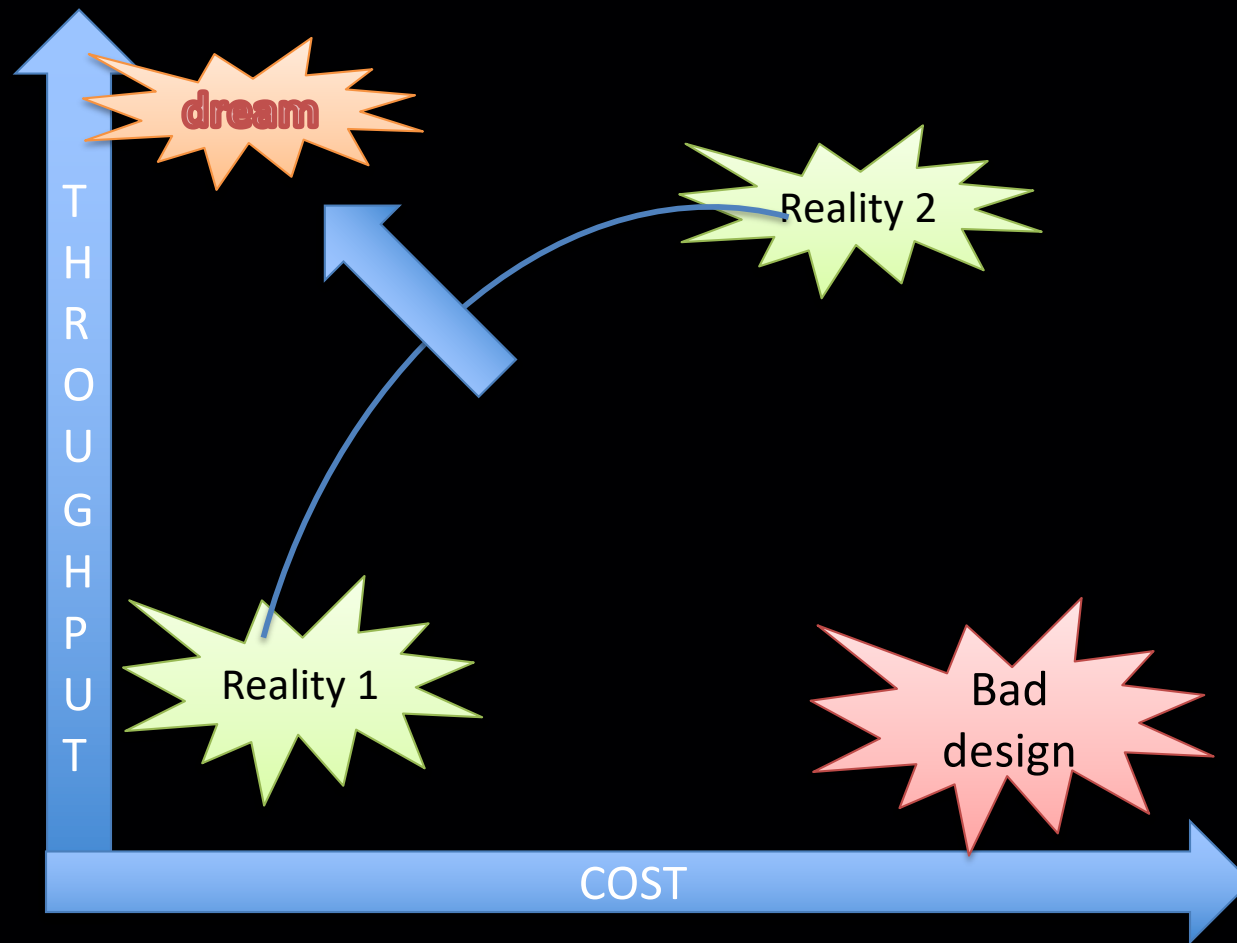
Storage Trends



TRADEOFFS & PARETO OPTIMALITY

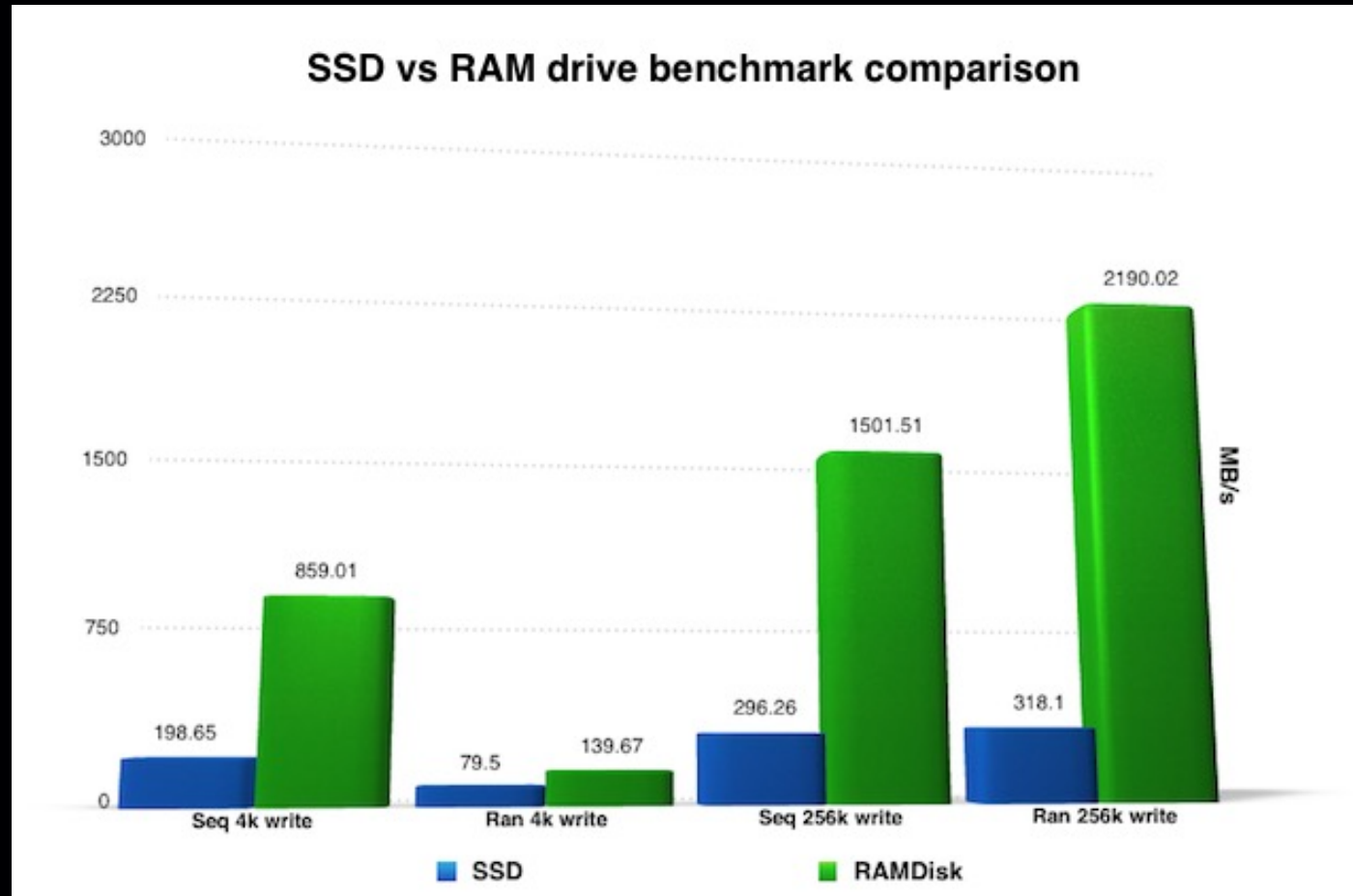


TRADEOFFS & PARETO OPTIMALITY



FILE SYSTEMS

Do SSDs solve the problem?



<http://www.makeuseof.com/tag/ram-drives-faster-ssds-5-things-must-know/>

FILE SYSTEMS

High speed storage in NVM is approaching RAM

- High performance data recorders can approach RAM speeds, e.g., 2.5 GB/sec
- These solutions, however, are far more expensive than DRAM
- Used in applications where reliable persistent storage is required such as real-time sensor (radar, imagery, etc.) data recording.
- None of this is useful, however, without an efficient file system.



StorePak

What makes FS interesting (intellectual merit)?

Challenges we need to think about:

- Crash recovery
- Performance:
 - Data vs control plane separation
- API design for sharing state
- Security for sharing (multi-process, multi-user)
- Abstraction generality: pipes, devices, /proc, /afs, etc.
 - so FS-oriented apps work with many kinds of objects
- Networked/distributed file systems: AFS, NFS, HDFS...

DATA/CONTROL PLANE



Source: <https://ganganichamika.medium.com/separating-data-plane-and-control-plane-9fee0b7f3ef8>

FILE SYSTEMS: DESIGN CONSIDERATIONS



High-level API choices

Granularity

- files, virtual disks, databases (records in the RDBMS)

File content

- byte array, records, b-tree (or key-value stores)

Organization:

- name hierarchy vs flat names (object IDs)

Synchronization

- None vs locks, transaction rollbacks
- Multi-version tracking?

FILE SYSTEMS: DESIGN CONSIDERATIONS



API implications

File descriptor (fd) refers to something

`fd table`

- preserved even if file name changes or deleted

File can have multiple links i.e., multiple directories

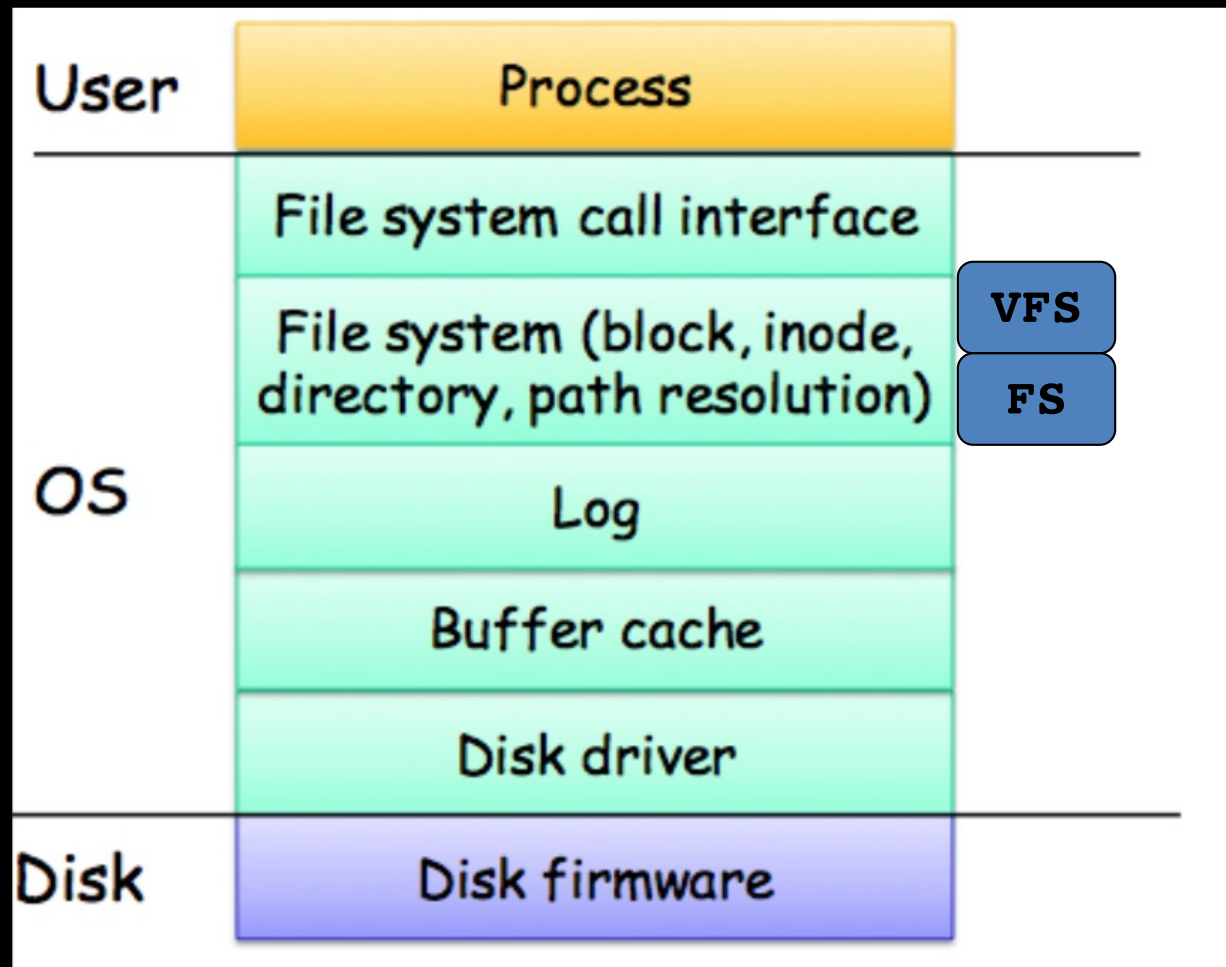
- file info should be stored somewhere other than directory

Thus a file is independent of its names

- it is called an "inode"
- inode must keep link count (tells us when to free)
- inode must have count of open fds'
- inode deallocation deferred until last link, fd removed

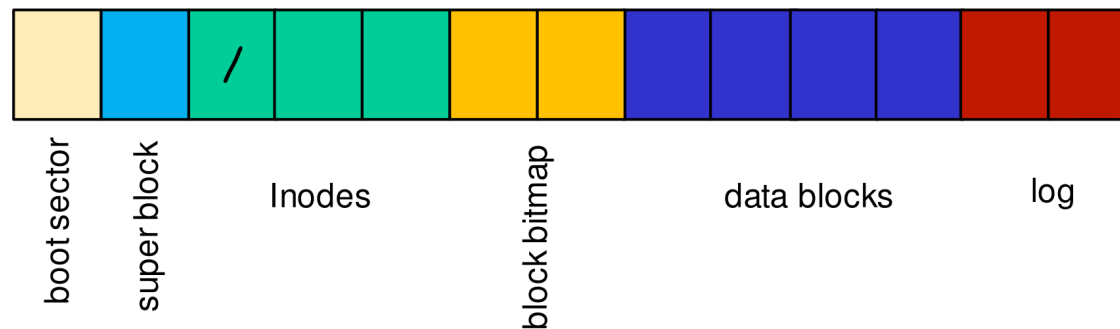
FILE SYSTEMS STACK

FS Software Layers



FILE SYSTEMS

On-Disk Layout



❑ `fs.h, fs.c, mkfs.c`

❑ `struct superblock`

XV6 ON DISK LAYOUT

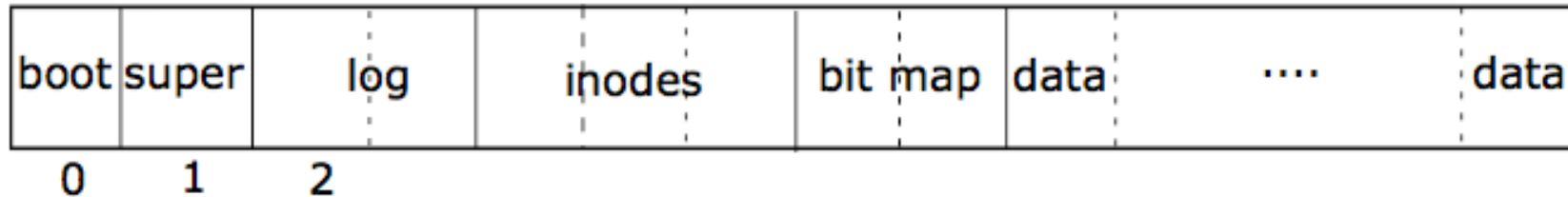


Figure 6-2. Structure of the xv6 file system. The header `fs.h` (4100) contains constants and data structures describing the exact layout of the file system.

Disk Blocks

Most OSs use blocks of multiple sectors

- e.g. 4 KB blocks = 8 sectors
- to reduce book-keeping and seek overheads

xv6 uses single-sector blocks for simplicity

"meta-data"

- everything on disk other than file content
- super block, i-nodes, bitmap, directory content

Inode: how to keep track, what's stored?

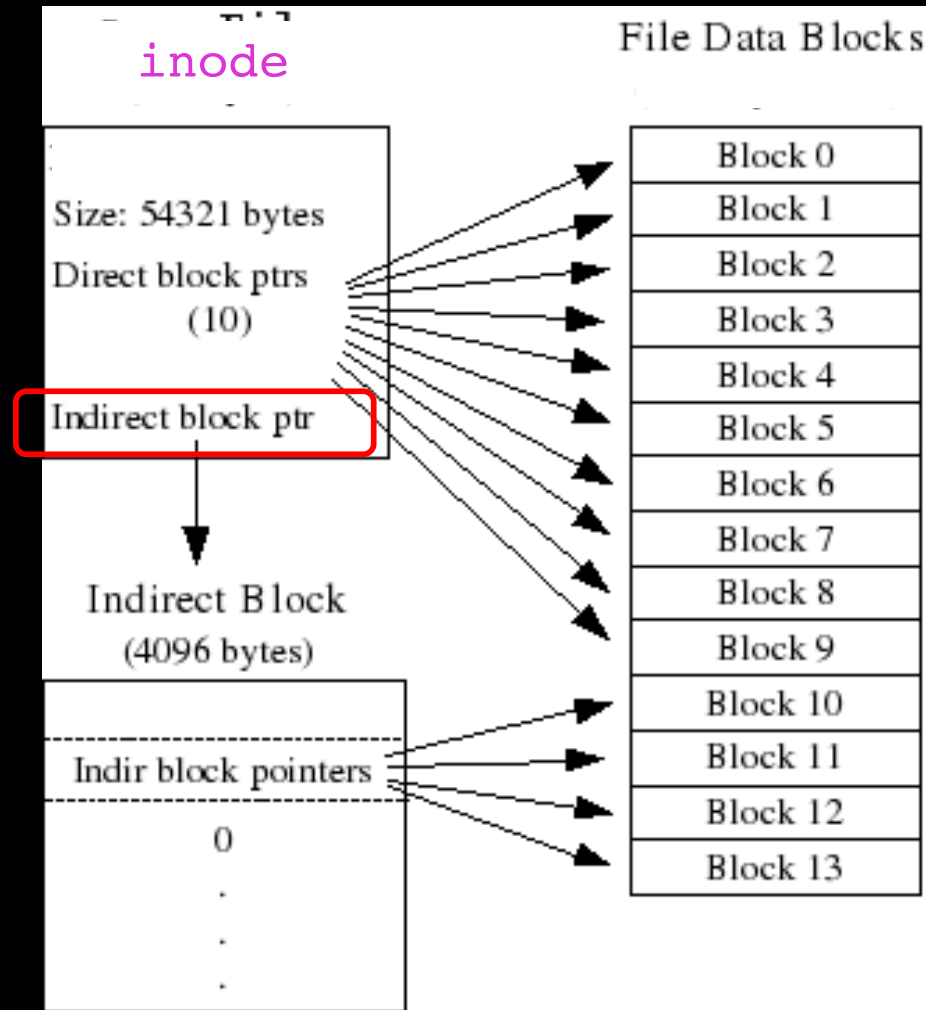
On-disk data struct

- type (free, file, directory, device)
- nlink
- size
- `addrs[12+1]`

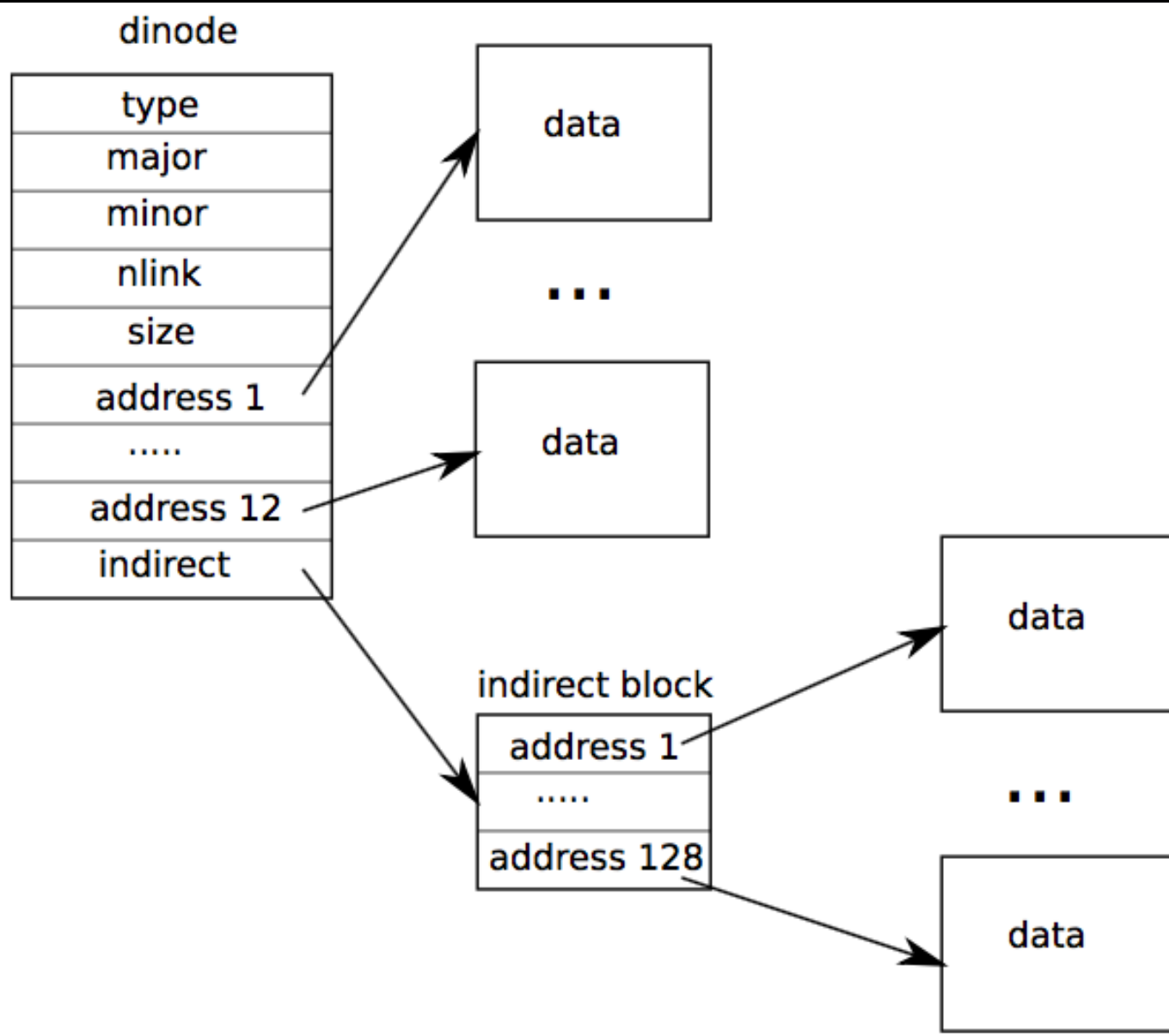
Q: Why 12+1 ?

FILE SYSTEMS

Direct and Indirect Blocks



XV6 INODE CONTENT



XV6 INODE CONTENT



- NDIRECT addrs entries → NDIRECT BSIZED blocks of data
 - $\text{NDIRECT} * \text{BSIZE} = 12 * \frac{1}{2} \text{ KB} = 6\text{KB}$
- Last entry → pointer to an indirect INODE
 - NINDIRECT entries in indirect block map NINDIRECT blocks of data
 - $\text{NINDIRECT} * \text{BSIZE} = 128 * \frac{1}{2} \text{ KB} = 64\text{KB}$

Direct and indirect blocks

How to find file's byte 8000?

- logical block 15 = $\text{floor}(8000 / \text{BLOCK_SIZE}) = 15$
- 3rd entry in the indirect block = $15 - 12 = 3$

i-node structure

- each i-node has an i-number
- easy to turn i-number into inode
- inode is 64 bytes long
- byte address on disk: $2 * 512 + 64 * \text{inum}$
 - E.g., accessing ``/`` = $2 * 512 + 64 * 0 = 1024\text{B}$ offset

TO BE CONTINUED



FILE SYSTEMS



Directory contents

Directory much like a file

- but user can't directly write

Content is array of dirents

Dirent:

- inum
- 14-byte file name
- dirent is free if inum is zero

Example: accessing `/foo/bar/baz`

how many inodes are being accessed?

Inode operations

kernel keeps inode in-memory until reference $\neq 0$

ialloc() - allocate inode

ilock() - and iunlock sync access to inode

iget() - returns the inode struct and inc ref count

iput() - dec the ref count and frees if ref = 0

iupdate() - copy modified inode to the disk

Inode xv6 usage

```
ip = iget(dev, inum)
ilock(ip)
... examine and
modify ip->xxx ...
iunlock(ip)
iput(ip)
```

Concurrent calls to ialloc?

Will they get the same inode?

- note bread / write / brelse in ialloc
- bread locks the block, perhaps waiting, and reads from disk
- brelse unlocks the block

Why do we use iget even after finding an inode?

- Let's see the iget method

Q: Why iget does not hold ilock?

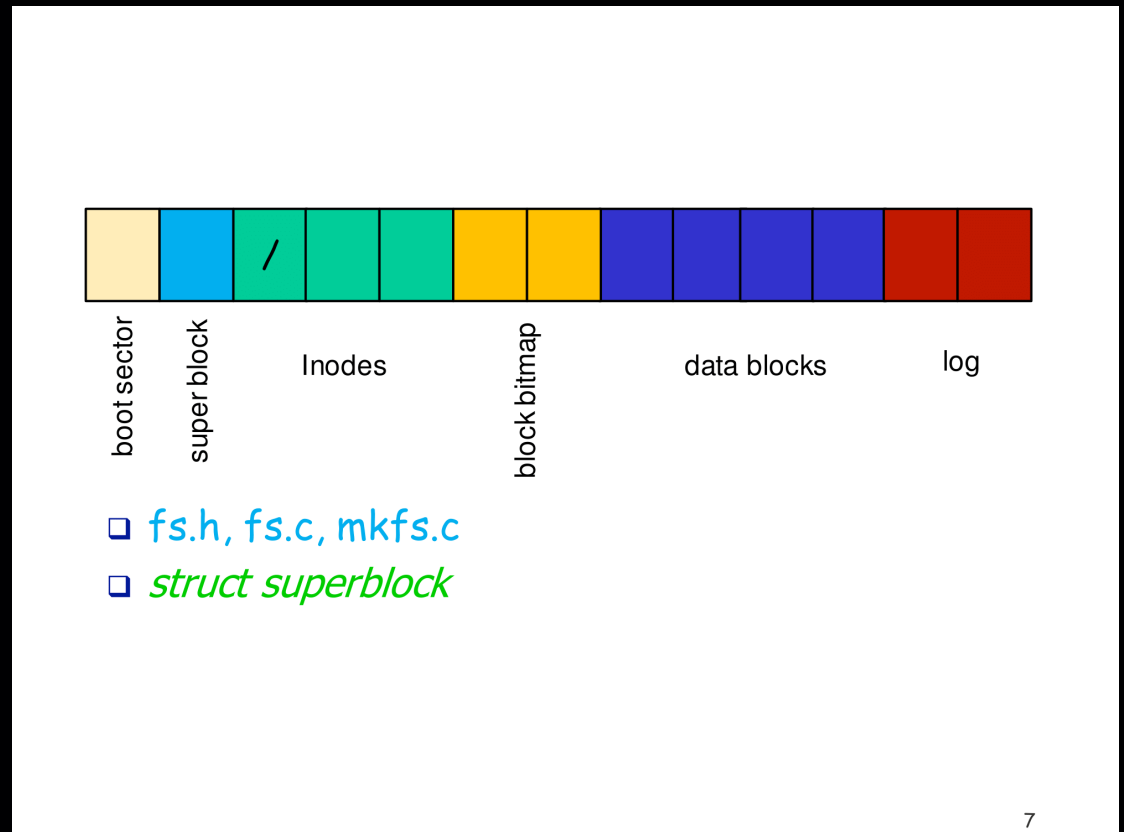
FILE SYSTEMS

xv6 maintain free bitmap
on disk – one bit per block
(sb->bmapstart)

- 0 means block is free, 1
means block in use

Checking if a block is free if
you know block number

- `buf[blockNum/8] & (0x1 <<
(blockNum % 8))`



Block allocation sequence

`balloc()` - allocates new disk block

`readsb()` - into to `sb` struct in memory

Iterate over the bitmap blocks for free block

If block found, update corresponding bit

`bfree()` - clear the relevant bit

Buffer cache layer

A double-linked list of buf structures

Holding cached copies of disk block contents

Two jobs:

- synchronize access to disk blocks
- one block on disk – one block in memory
- one kernel thread at the time use same block

Cache popular blocks in fixed buffers

Buffer cache layer

Flags:

- B_BUSY – buffer locked
- B_VALID – buffer has been read from disk
- B_DIRTY – buffer was modified and should be written to disk

Interface:

- binit() - called by main
- bread() - to read buffer from block on disk
- bwrite()- to write buf to disk
- brelse()- to release buf when done and move it to the head

Buffer cache layer

Let's look at the block cache in bio.c

- block cache holds just a few recently-used blocks

FS calls bread, which calls bget

- bget looks to see if block already cached
- if present and not B_BUSY, return the block
- if present and B_BUSY, wait
- if not present, re-use an existing buffer

Q: why goto loop after sleep()?

Replacement policy

xv6 implements LRU for buffer cache replacement.

Maintain the buffers in a doubly-linked list.

When done accessing a buffer (at the time of clearing the busy bit),

- move the buffer to the front of the buffer cache list
- start replacement at the last entry of the list.

Let's discuss buffer cache and disk driver interaction

Disk driver

Let's look into ide.c

ideinit() initializes the IDE

- Q. What does this line mean
ioapicenable(IRQ_IDE, ncpu - 1)?
- Q. Why do we check if disk 1 is present?

Disk driver

`ide_rw()` - read or write a block from/to the disk

- Q: How to handle multiple `ide_rw()` calls?

Notice just one lock (`ide_lock`) for enforcing multiple invariants

`iderw` and `ideintr` share the request queue using `idelock`

Q: What if we enable interrupts with single processor?

Now, let's look at xv6 in action

- Focus on disk writes
- Illustrate on-disk data structures via how updated

Q: How does xv6 create a file?

xv6 supports logging which we will discuss later

- log_write replaces bwrite()

Q: what's in block 34?

- look at create() in sysfile.c

Q: why *two* writes to block 34?

Q: what is in block 59?

```
$ echo > a
write 34 ialloc (from create
sysfile.c; mark it non-free)
write 34 iupdate (from create;
initialize nlink &c)
write 59 writei (from dirlink
fs.c, from create)
```

Delete a file

```
$ rm a  
write 59 writei (from sys_unlink; directory content)  
write 34 iupdate (from sys_unlink; link count of file)  
write 58 bfree- (from itrunc, from input)  
write 34 iupdate (from itrunc)  
write 34 iupdate (from input)
```

Q: How fast xv6 apps can read big files?

- First reads data from disk to buffer cache
- Then, from buffer cache to user space
- What happens if we pass user buffer to the disk device driver?

Q: How much RAM should we dedicate to disk buffers?

FILE SYSTEMS PART 2

ALEXEY TUMANOV

CREATING THE NEXT®

Administrativa

Lab4 fs: released Fri, Apr 3. Due Fri, 4/17 (2 week assignment)

File Systems: user-isolation and file permissions

- Adding users
- File permissions
- Chown syscall, chmod syscall

Part 1: Adding Users: setuid/getuid

Part 2: File Permissions: chown/chmod

Part 3: Login:

- Current xv6: free for all! Drops you into root

Part 4: Large File Support

Administrativa

Lab4 : spec extension

Part 3: login system for xv6

Goal: leverage:

- Parts 1 and 2 of the lab
- Provided user-space hash and encryption library
- Implement part 3 (login functionality)

Learning objective:

- End-to-end system
- Security design principles

Lab4 Networking

- Lab4 will have a filesystem (lab4-fs) and networking (lab4-net) options
 - Grading structure: $\max(\text{lab4-fs}, \text{lab4-net})$ on total score
 - This implies that you can do both or either
 - **BONUS** points for lab4-net:
 - Separate assignment on canvas
 - Achieved by meeting or exceeding a specified threshold score on lab4-net
 - i.e. if the lab4-net score threshold is NOT met => no bonus
 - Lab4-net will be due 3 weeks from tomorrow: Fri, Apr 24th
 - The level of difficulty is ~ lab2. If you want the bonus, focus on lab4-net.

FILE SYSTEMS

What is a Filesystem?

Why do we need a file system?

Does a filesystem have to relate to disk?

Does a filesystem need any secondary storage??

Manage user
data

Multiplex
data
resources

data
resource
isolation

data
resource
protection

Data
management

FILE SYSTEMS

- File System (FS) is an abstraction to manage data in general
 - Not tied to persistent data
 - FS controls access to the data (any data)
 - Naming and organizing data
 - Sharing data between multiple users and multiple processes
 - Data can be read-only (e.g., procfs)
- Naming, Protection, Isolation:
 - FS : useful mechanism for accessing devices (e.g., camera, /dev)
 - Interface that broadly generalizes to a variety of data I/O scenarios

FILE SYSTEMS



FS Last Lecture Overview

File systems

API (Design choices)

Disk layout

Buffer cache

xv6 in action - code walk

FILE SYSTEMS

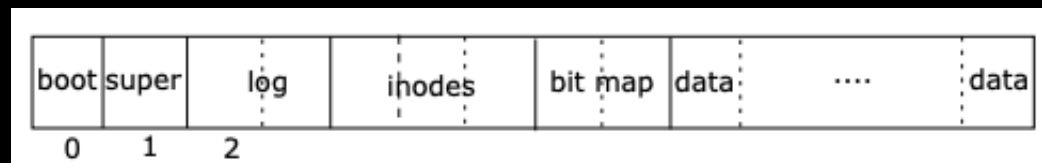
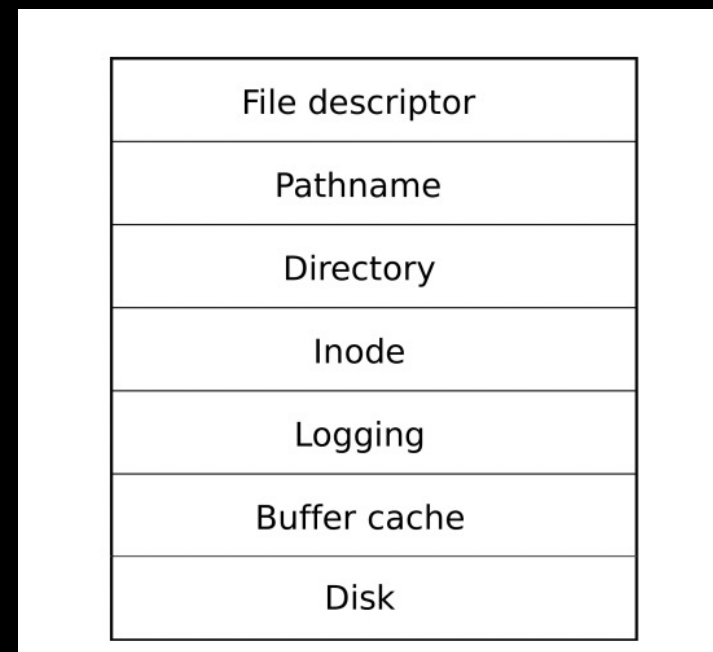


This lecture (File Systems 2):

- The inode layer
- Hand-over-hand locking
- Xv6 FS Code Walkthrough

xv6 FS Layers

- The xv6 file system is layered, which each layer providing abstractions that build on top of each other
- The layers interact with each other in nuanced ways, especially with regards to concurrency



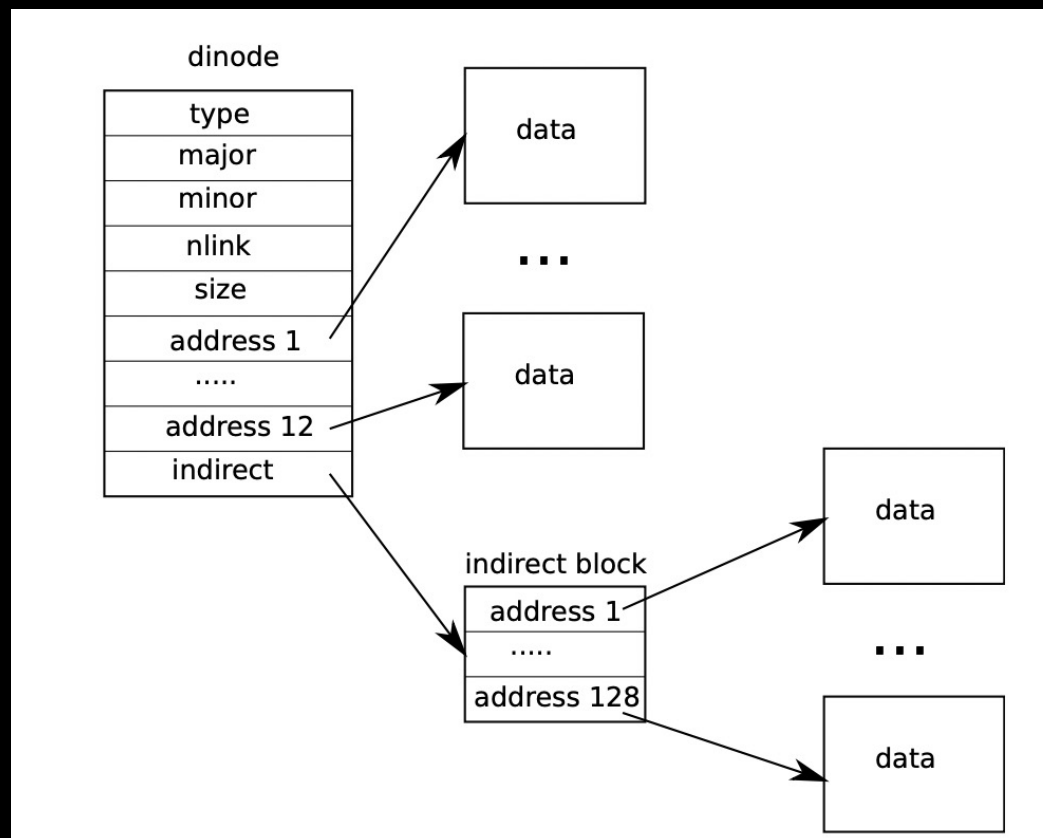
Inodes

Inode (Think of as file/directory *metadata*)

- type (free, file, directory, device)
- nlink (how many directories reference this inode?)
- size (how large in bytes is the data of this inode?)
- addrs[12+1] (12 directly linked block, 1 indirect block)

Does not contain actual data! Only pointers to it

File inode representation



Q: What is the largest possible file size in xv6?

$$(12 + 128) * 512 = 71680 \text{ bytes}$$

Accessing an inode

i-node structure

- Logically, each i-node has an i-number
- Easy to turn i-number into inode
- inode is 64 bytes long
- byte address on disk: $2 * 512 + 64 * \text{inum}$
 - $2 * 512$ because of boot sector and superblock

Tidbit: Lab4fs specific layout

+0x00 - 2bytes - type -- file type

+0x02 - 2bytes - major -- major device number

+0x04 - 2bytes - minor -- minor device number

+0x06 - 2bytes - nlink -- number of links to inode in the file system

+0x08 - 4bytes - size -- size of the file in bytes

+0x0c - 2bytes - owner -- Owner uid of the file

+0x0e - 2bytes - perms -- Permissions of the file

+0x10 - 48bytes - addrs -- Array of data block addresses + indirect block address

Accessing file data

Eg: How to find file's byte 8000?

- Logical data block = $8000 // \text{BLOCK_SIZE} = 15$
- Remember: 12 direct data block addresses + 1 indirect block address
- Logical block 15 corresponds to 3rd entry in the indirect block

Directory inode representation

Content is array of dirents

Dirent:

- inum
- 14-byte file name
- dirent is free if inum is zero

There are 13 dirents in xv6,
represented as an array on disk

type
major
minor
nlink
size
dirent 1
...
dirent 13

Directory Walks

/bar/baz/foo.txt

inum	name
0	/
1	test0
2	bar
...	...
13	



inum	name
0	bar
...	...
4	baz
...	...
13	test1



inum	name
0	baz
...	...
7	foo.txt
...	...
13	



Inode for foo.txt

Directory Walks and Concurrency



Process 0: Walk /bar/baz/foo.txt; currently at baz

Process 1: Delete baz

- We have a data race
- Directory walk needs exclusive access to the inode

Coarse-grained locking

inum	name
0	/
1	test0
2	bar
...	...
13	

inum	name
0	bar
...	...
4	baz
...	...
13	test1

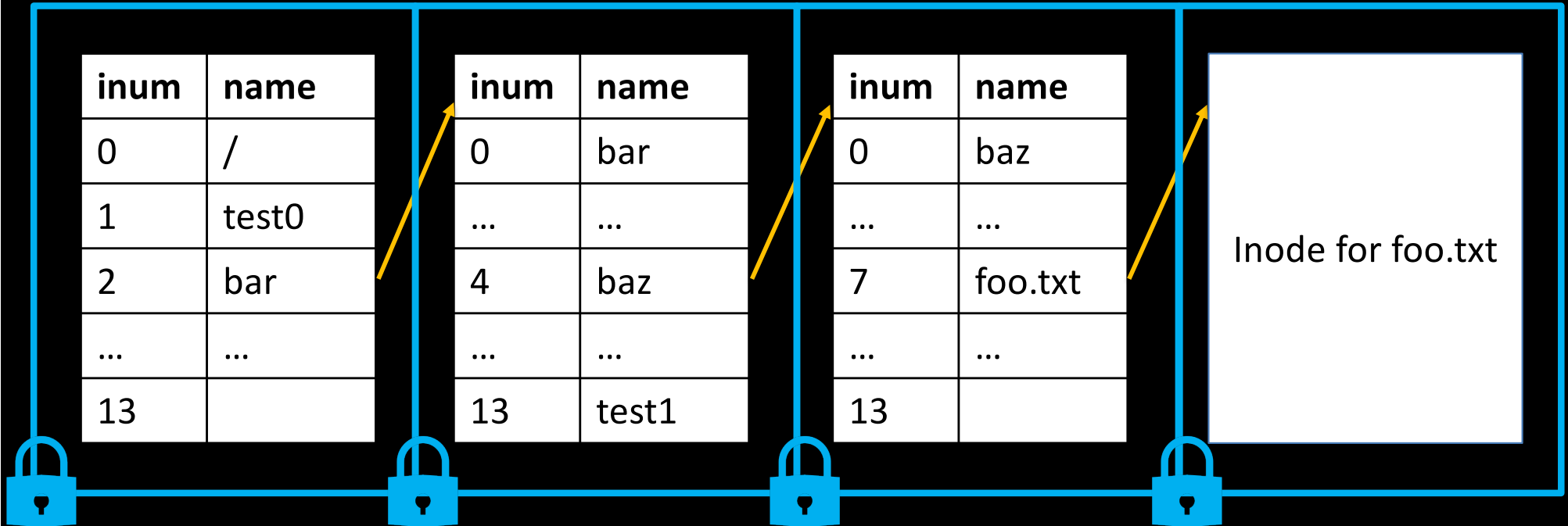
inum	name
0	baz
...	...
7	foo.txt
...	...
13	

Inode for foo.txt



- Lock the inode cache to ensure exclusive access to ALL inodes
- Poor performance; effectively no concurrency

Hand-over-hand locking



- Locking is more fine-grained
- *Actually* supports concurrent path accesses
- Much better performance

Directory Structure

- Why do we use an array of dirents? Why not a binary tree?
- **Modern Linux:** Uses a disk-based tree structure (b-tree or b+tree) in modern filesystems (btrfs, optionally on ext4).
 - However, most systems just use a simple directory array.
 - Why?

XV6 Filesystem Overview

How do the systems in the xv6 filesystem fit together?

- `sysfile.c` – System call interfaces for file operations (like `sysproc.c`)
- `file.c` – Virtual Filesystem operations
- `fs.c` – File, directory, and inode manipulation
- `bio.c` – xv6's block cache
 - Holds copies of disk data to reduce disk operations
- `ilog.c` – xv6's file logging system
 - Provides atomicity and ordering of filesystem operations
- `ide.c` – The ide (disk) interface.

Disk driver

Let's look into ide.c

ideinit() initializes the IDE

- Q. What does this line mean
ioapicenable(IRQ_IDE, ncpu - 1)?

Disk driver

`ide_rw()` - read or write a block from/to the disk

- Q: How to handle multiple `ide_rw()` calls?

Notice just one lock (`ide_lock`) for enforcing multiple invariants

`iderw` and `ideintr` share the request queue using `idelock`

Buffer Cache

A double-linked list of buf structures

Holding cached copies of disk block contents

Two jobs:

- synchronize access to disk blocks
- one block on disk – one block in memory
- one kernel thread at the time use same block

Cache popular blocks in fixed buffers

Buffer Cache Replacement Policy



xv6 implements LRU for buffer cache replacement.

Maintain the buffers in a doubly-linked list.

When done accessing a buffer (at the time of clearing the busy bit),

- move the buffer to the front of the buffer cache list
- start replacement at the last entry of the list.

Let's discuss buffer cache and disk driver interaction

Buffer Cache API

Buffer cache layer

Flags:

- B_BUSY – buffer locked
- B_VALID – buffer has been read from disk
- B_DIRTY – buffer was modified and should be written to disk

Interface:

- binit() - called by main
- bread() - to read buffer from block on disk
- bwrite()- to write buf to disk
- brelse()- to release buf when done and move it to the head

Inode API

Kernel keeps inode in-memory until reference $\neq 0$

ialloc() - allocate inode

ilock() - and iunlock sync access to inode

iget() - returns the inode struct and inc ref count

iput() - dec the ref count and frees if ref = 0

iupdate() - copy modified inode to the disk

itrunc() - discard inode contents

Inode data blocks API



`readi()` – read data block from inode

`writei()` – write data block to inode

Inode API Usage

```
ip = iget(dev, inum)
ilock(ip)
... examine and
modify ip->xxx ...
iunlock(ip)
iput(ip)
```


ialloc concurrency

Concurrent calls to ialloc?

Will they get the same inode?

- note bread / write / brelse in ialloc
- bread locks the block, perhaps waiting, and reads from disk
- brelse unlocks the block

Now, how do we get data-blocks to associate data with our inode?

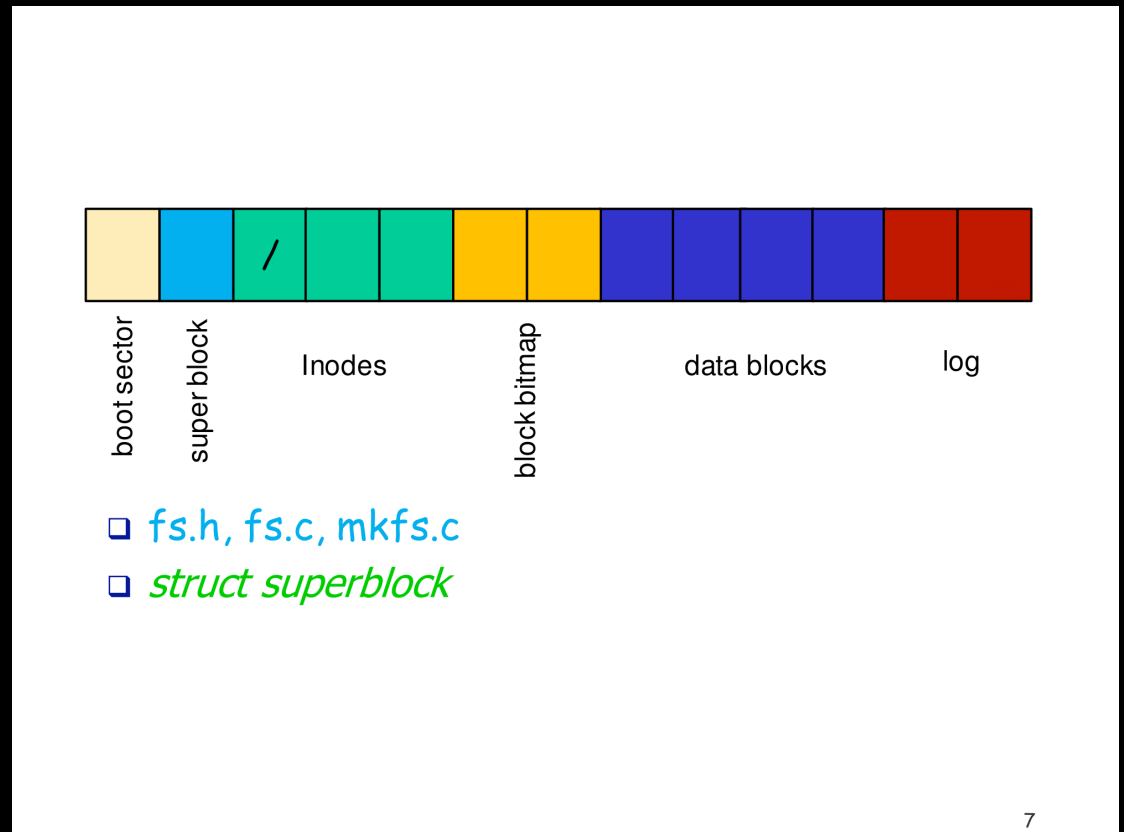
Block Allocator

xv6 maintain free bitmap on disk – one bit per block (sb->bmapstart)

- 0 means block is free, 1 means block in use

Checking if a block is free if you know block number

- `buf[blockNum/8] & (0x1 << (blockNum % 8))`



Block Allocator API



`balloc()` - allocates new disk block

`readsb()` - reads superblock

`bfree()` - clear the relevant bit

File system operations



Now, let's look at file system operations

- Illustrate on-disk data structures via how updated

FILE SYSTEMS

Q: How does xv6 create a file?

xv6 supports logging which we will discuss later

- log_write replaces bwrite()

Q: what's in block 34?

- look at create() in sysfile.c

Q: why *two* writes to block 34?

Q: what is in block 59?

```
$ echo > a
write 34 ialloc (from create
sysfile.c; mark it non-free)
write 34 iupdate (from create;
initialize nlink &c)
write 59 writei (from dirlink
fs.c, from create)
```

FILE SYSTEMS

Xv6: Write data to a file

Q: what's in block 58, block 613?

- look at writei call to bmap
- look at bmap call to balloc

```
$ echo x > a
write 58 balloc- (from bmap, from
writei)
write 613 bzero
write 613 writei (from filewrite file.c)
write 34 iupdate- (from writei)
write 613 writei
write 34 iupdate
```

FILE SYSTEMS

Delete a file

```
$ rm a  
write 59 writei (from sys_unlink; directory content)  
write 34 iupdate (from sys_unlink; link count of file)  
write 58 bfree- (from itrunc, from iput)  
write 34 iupdate (from itrunc)  
write 34 iupdate (from iput)
```

Up Next



- How do we manage persistency in the instance of crashes within our filesystem?

ORDERING, ATOMICITY, AND FILE-SYSTEMS

ALEXEY TUMANOV

CREATING THE NEXT®

Administrativa

- Lab4 bonus structure:
 - Possible to earn up to 1% towards the final grade, if  **BONUS**
 - [**threshold**] you must pass Tests 1-9 on lab4-net
 - => failing any of these tests yields a zero, even if you pass everything else
 - [**score**] assuming threshold, bonus calculation formula:
 - $\text{autograder points earned} / \text{maximum possible autograder points} * 1\%$
- No bonus for lab4-fs (Reason: net lab is ~1.5x harder than fs)
- Lab4 canvas score = max (lab4fs , lab4net). Can do both.
- Ref: <https://github.gatech.edu/cs3210-spring2026/xv6/tree/lab4-net>

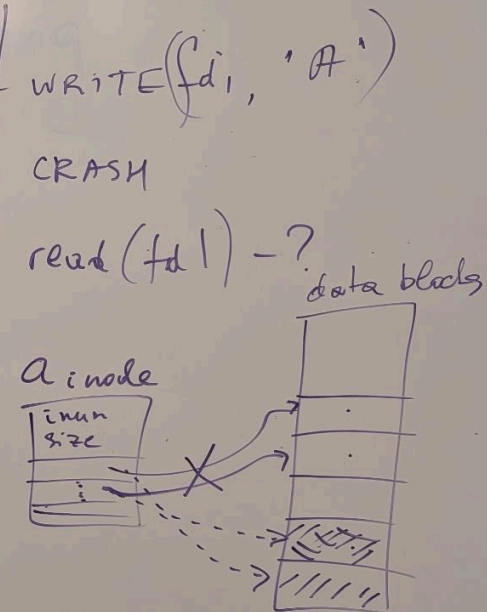
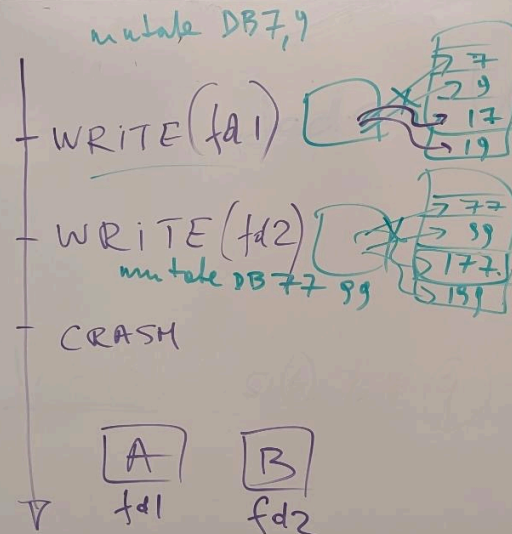
Ordering? Atomicity? What?

- With respect to the FS: What happens when my machine *crashes*, shuts down, or stops *unexpectedly*?
- How do I know my data *persisted* to disk?
 - How do I (as a user) know my data was persisted to disk?
 - How do I (as the FS) ensure the FS metadata / data wasn't **corrupted**?
 - **Ordering** – File updates happen **in order**
 - **Atomicity** – File updates happen or they don't, but not partially

I need ordering and atomicity to keep a sane system!

Crash Scenarios

write()
CRASH
recover
read()?
t↓



Motivating Examples

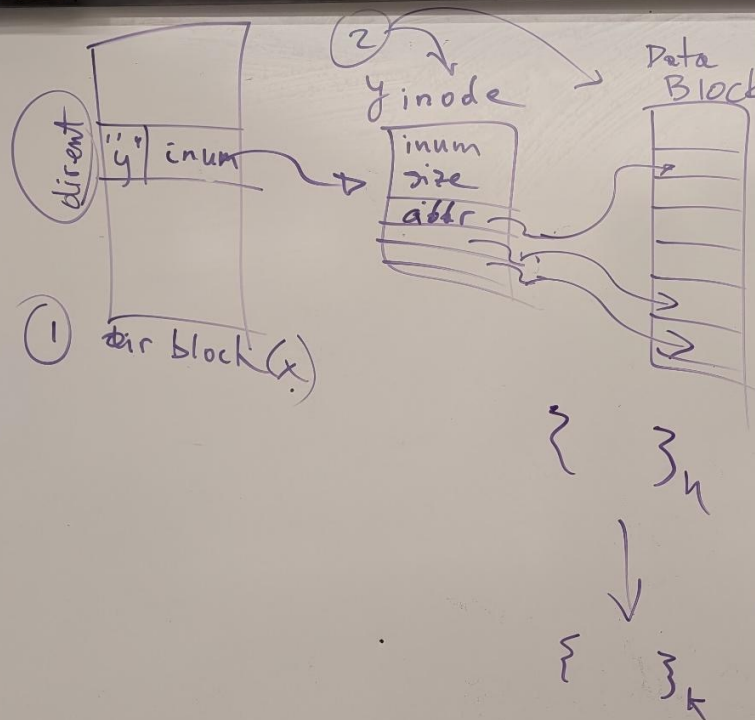
What happens if mkdir is
synced to disk after open?

```
mkdir("./x");  
open("./x/y", O_CREAT...);
```

What happens if metadata
is corrupted for y?

What happens if x direntry
is written before y's
inode/data?

Reordering Scenarios: Directory+File



Directory representation

Directory contents

Directory much like a file

- but user can't directly write

Content is array of dirents

Dirent:

- inum
- 14-byte file name
- dirent is free if inum is zero

Offset	Field	Use
0	inum	Inode number
2	name	Inode name



Disk Interface

- What interface does our disk give us?
 - Read block, write block
 - May not happen in order, may buffer data, not atomic
 - Flush
 - Waits until all pending read/writes are synced to disk.
- But what about in the instance of a crash? What happens if an operation has only partially finished?
 - Can have indeterminate data – partially written blocks!
 - Disk head can go haywire after poweroff – can get **incorrect data** on the block(s) I'm writing.

Disk Interface

On disk:
Ordering is achieved with
flush operations!



What are the ordering behaviors of different read/write operations?

Write block 47

Write block 2

Write block 48

flush

Write block 49

Space of ordering possibilities



- The happens before relation is the only primitive that reduces the cardinality of the set of orderings
- Example without ordering:
 - Write 47 2 48 49
 - *How many orderings?* $4!$
- Example with ordering (using flush):
 - Write 47 2 48 FLUSH write 49
 - *How many orderings?* $3! * 1!$
- **General formulation:**
 - two groups of writes (n and k each) separated by FLUSH
 - *How many orderings?* $n! * k!$
- Observation (without proof): $(n+k)! > n! * k!$
 - **Takeaway:** imposing partial order shrinks the space of options

Proof for cardinality reduction

Elegant proof that partial order reduces space of possibilities

Binomial coefficients , general case $C(n,k) = n! / ((n-k)! * k!)$

Replace n with $n+k \Rightarrow C(n+k, k) = (n+k)! / (n! * k!) \in \mathbb{N}$ (Pascal Triangle)

$C(n+k, k) \in \mathbb{N}$ (Pascal Triangle) $\Rightarrow \geq 1$

$\Rightarrow (n+k)! = n! * k! * (\text{some positive integer})$

$\Rightarrow (n+k)! \geq n! * k!$

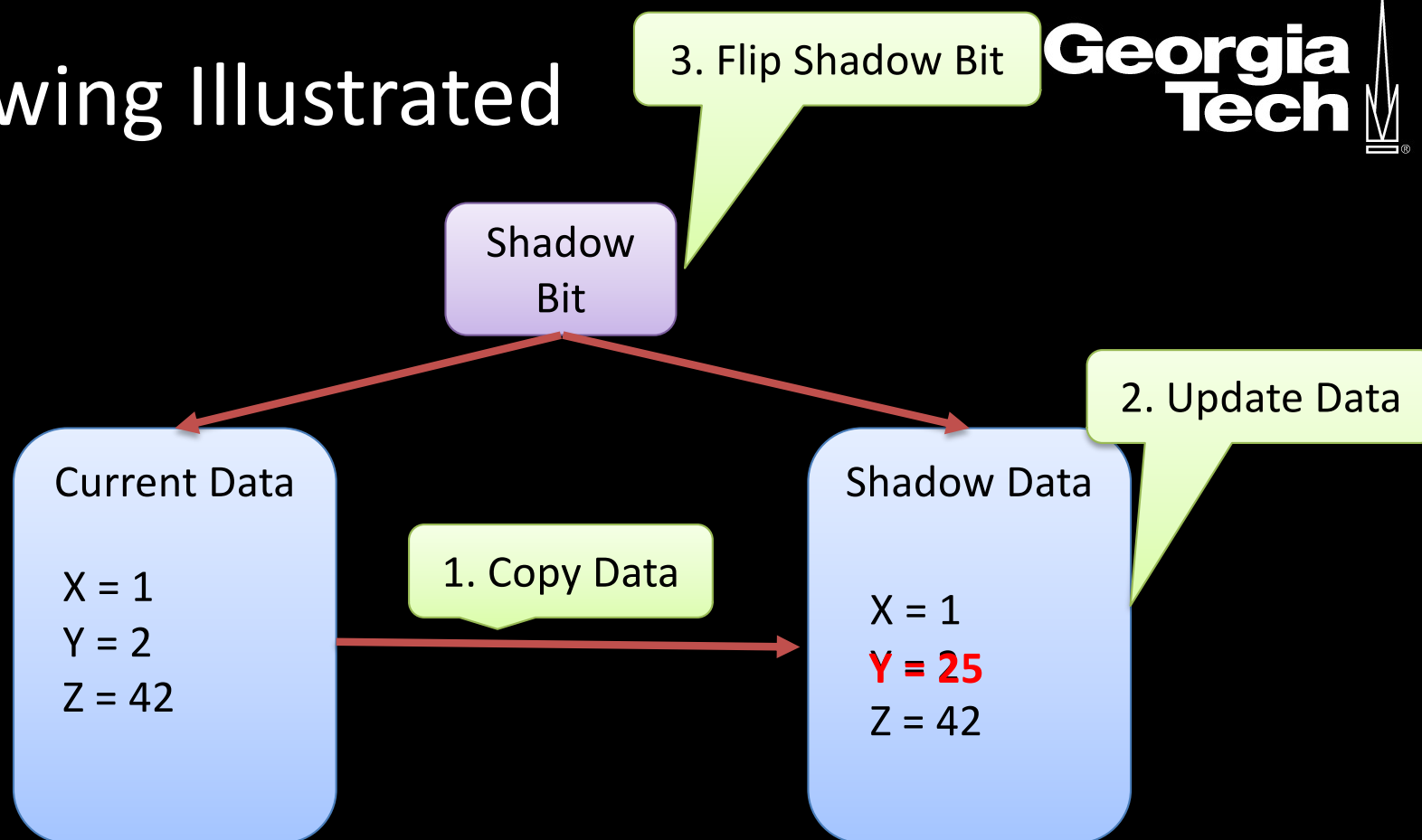
Atomicity

- How do I turn a partial disk-sector write interface (with flushes) into the atomic multi- block and inode update our FS needs?
 - Cannot update-in-place – disk interface doesn't allow it
 - Instead, make a copy, and atomically “commit” that copy.
- Two primary methods:
 - Shadowing
 - Logging (aka Journaling)

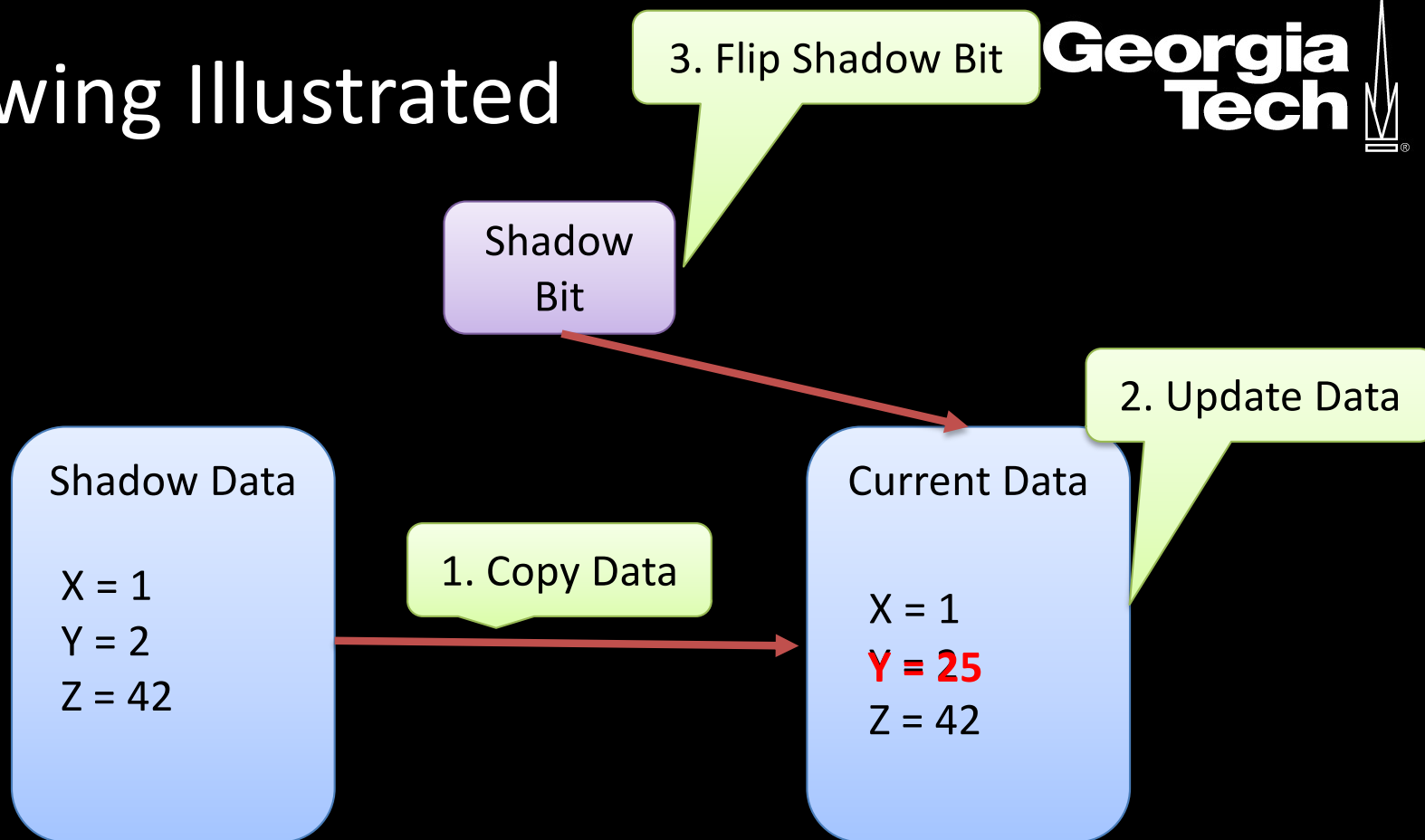
Shadowing

- **Idea:** Let's try to make the actual update of my data an operation that must be done atomically – a single Boolean bit flip.
 - Create a two copies of data – one active and one shadow
 - Create a “shadow bit” that chooses which version is active
- **Insight:**
 - Proxy large data block updates with a single bit flip

Shadowing Illustrated



Shadowing Illustrated



Shadowing

- Idea: Let's try to make the actual update of my data an atomic write – one Boolean decision that chooses between two versions of our data
- Keep two versions of data, our “current” version and our “shadow” version.
 - Also, keep a “shadow bit” that chooses which version is current.
 - On update, copy live version into “shadow” version
 - Then, update shadow version
 - When done, flip shadow bit, atomically switching from the old to the new version.

Shadow

Q: The bit flip?

3. Flip Shadow Bit

Shadow
Bit

Q: How about the
update?

2. Update Data

Current Data

X = 1
Y = 2
Z = 42

1. Copy Data

Shadow Data

X = 1
Y = 25
Z = 42

Q: Does the copy
have to be atomic?

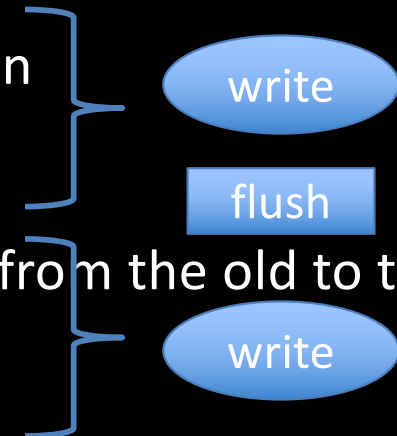
1,2

flush

3

Shadowing

- Keep two versions of data, our “current” version and our “shadow” version.
 - Also, keep a “shadow bit” that chooses which version is current.
 - On update, copy live version into “shadow” version
 - Then, update shadow version
 - When done, flip shadow bit, atomically switching from the old to the new version.



2 writes + a flush

Shadowing Illustrated

Q: What happens if we crash
at each step in the process?
What data is recovered?

Shadow
Bit

3. Flip Shadow Bit

Current Data

X = 1
Y = 2
Z = 42

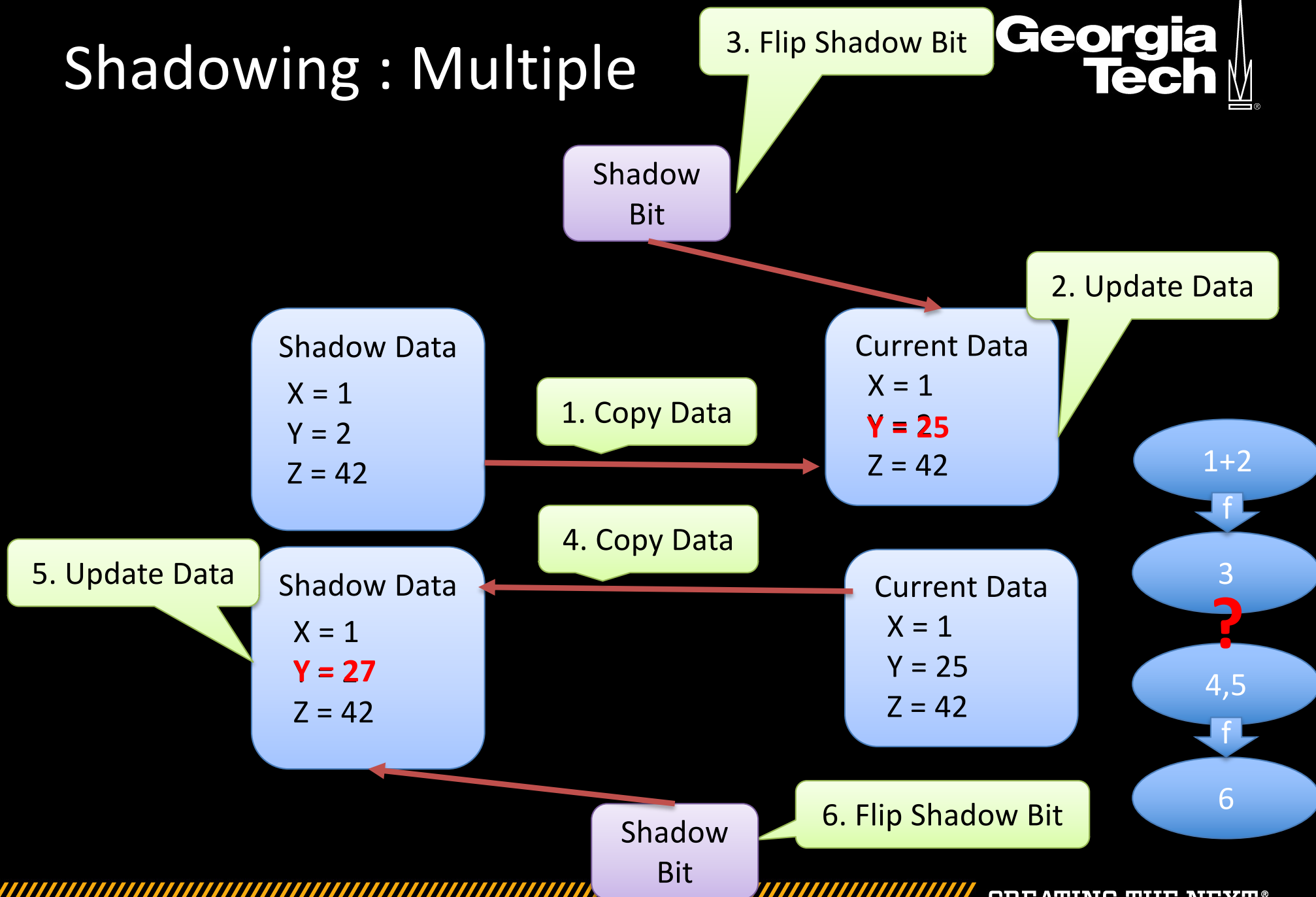
1. Copy Data

Shadow Data

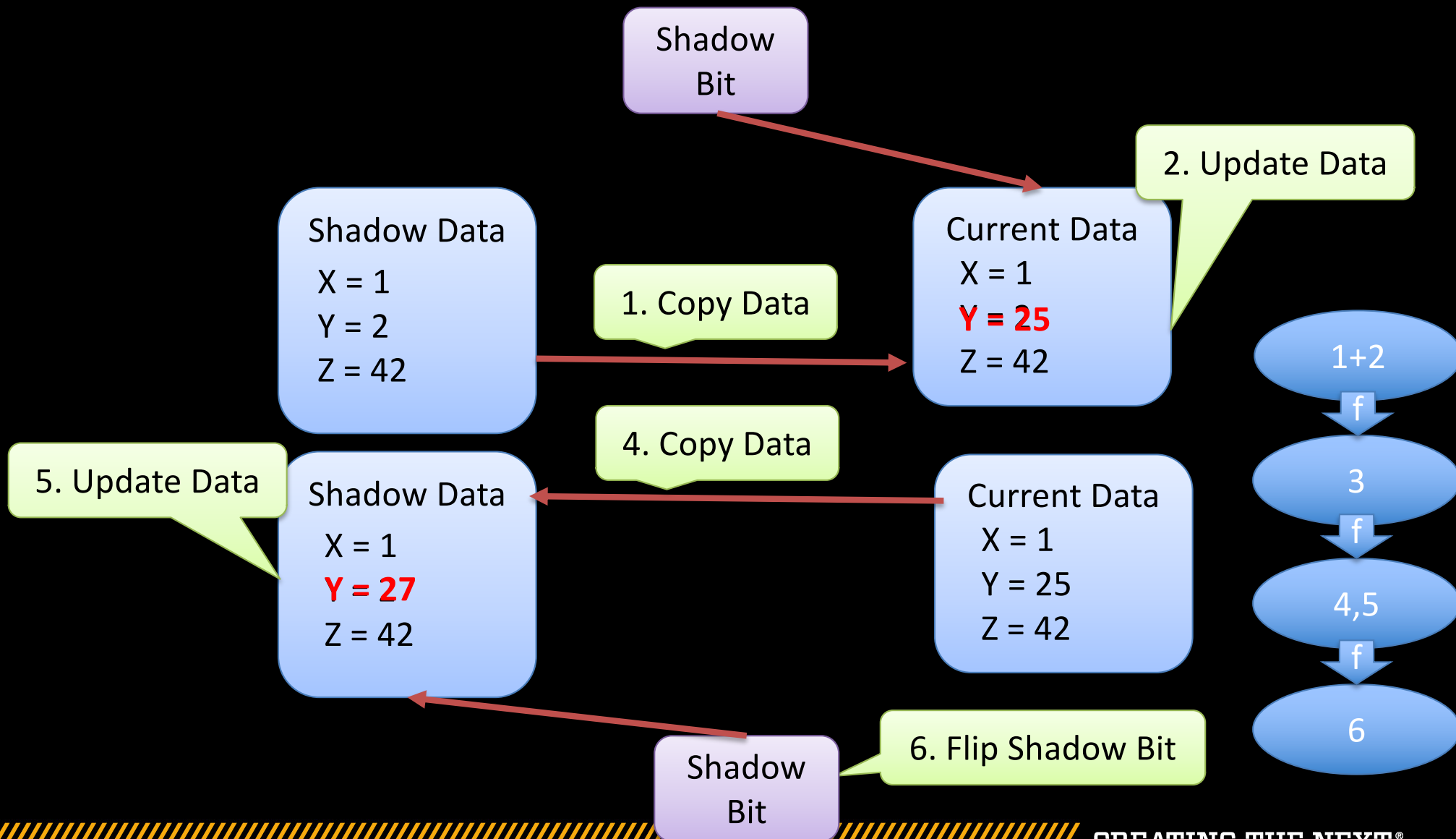
X = 1
Y = 25
Z = 42

2. Update Data

Shadowing : Multiple



Shadowing : Multiple



Shadowing breakdown

- Is shadowing atomic?
 - Yes, the Boolean “shadow bit” chooses between two versions. It cannot be partial, it’s a single Boolean decision!
- What are the costs of shadowing?
 - Need much more (at least 2x) the space ☹️
 - Requires write to a shadow bit on every version update (at least 2 disk writes – copy & update, shadow)
 - Note: Shadow must be a separate bit, meaning it must be on a separate disk block
 - Often results in large amounts of wasted space.

+ shadow bit block

Shadowing in FS

- Does it make sense to use shadowing as our *primary* method of atomicity in the FS?
 - Why or why not?
- What alternative can we create?

Logging (also called Journaling)



- Let's instead keep a log (or a journal) of data.
 - Keep a record of what the new data should be
 - Atomically “commit” records in the log
- For any “committed” log record, the log supersedes the actual data block (or metadata) in the file system.
 - If it's in the log, we take the log data
 - Otherwise, we go to the data block that's actually in the FS

ORDERING, ATOMICITY, AND FILE SYSTEMS: PART 2

ALEXEY TUMANOV

School of Computer Science
cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

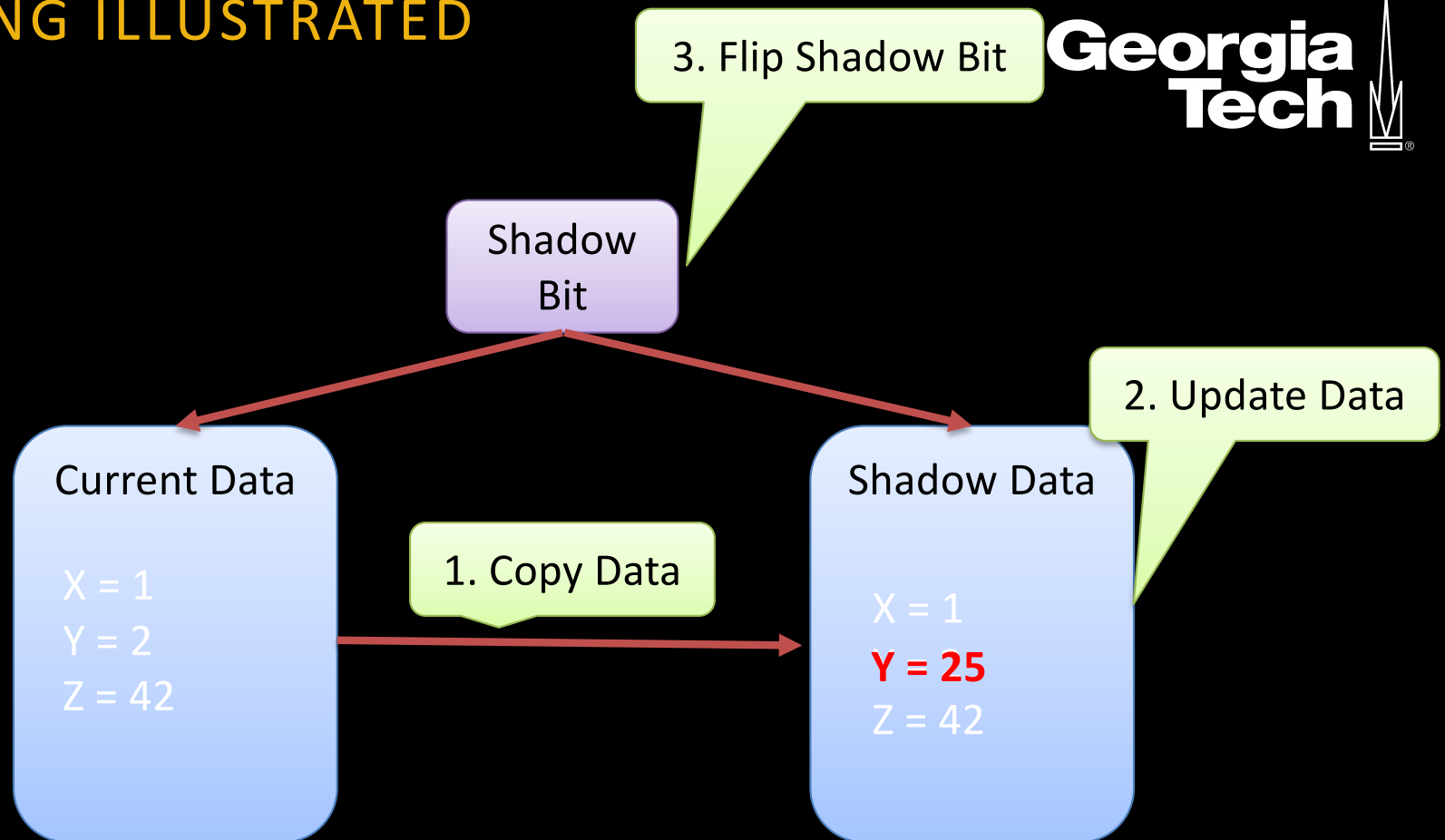
- How do I turn a partial disk-sector write interface (with flushes) into the atomic multi- block data and inode update our FS needs?
 - Cannot update-in-place – disk interface doesn't allow it
 - Instead, make a copy, and atomically “commit” that copy.
- Two primary methods:
 - Shadowing
 - Logging (aka Journaling)

Atomicity and the FS.

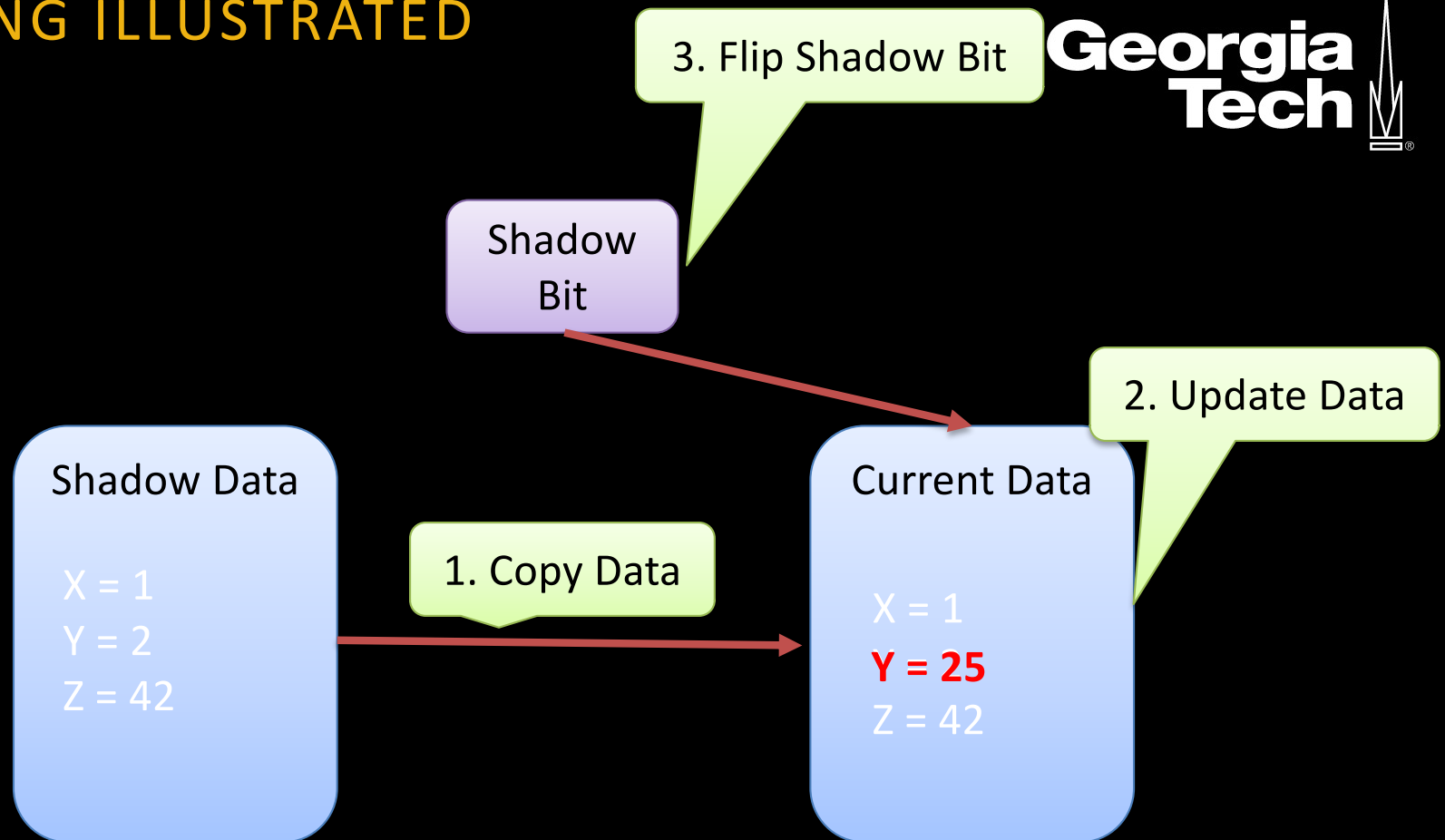
Shadowing:

- 1) Maintain two copies of the data
- 2) Make a “shadow bit” that shows which is “current”
- 3) When reading, read the “*current*” copy
- 4) When writing, update the “*shadow*” copy
- 5) THEN (after writing), *flip* the shadow bit

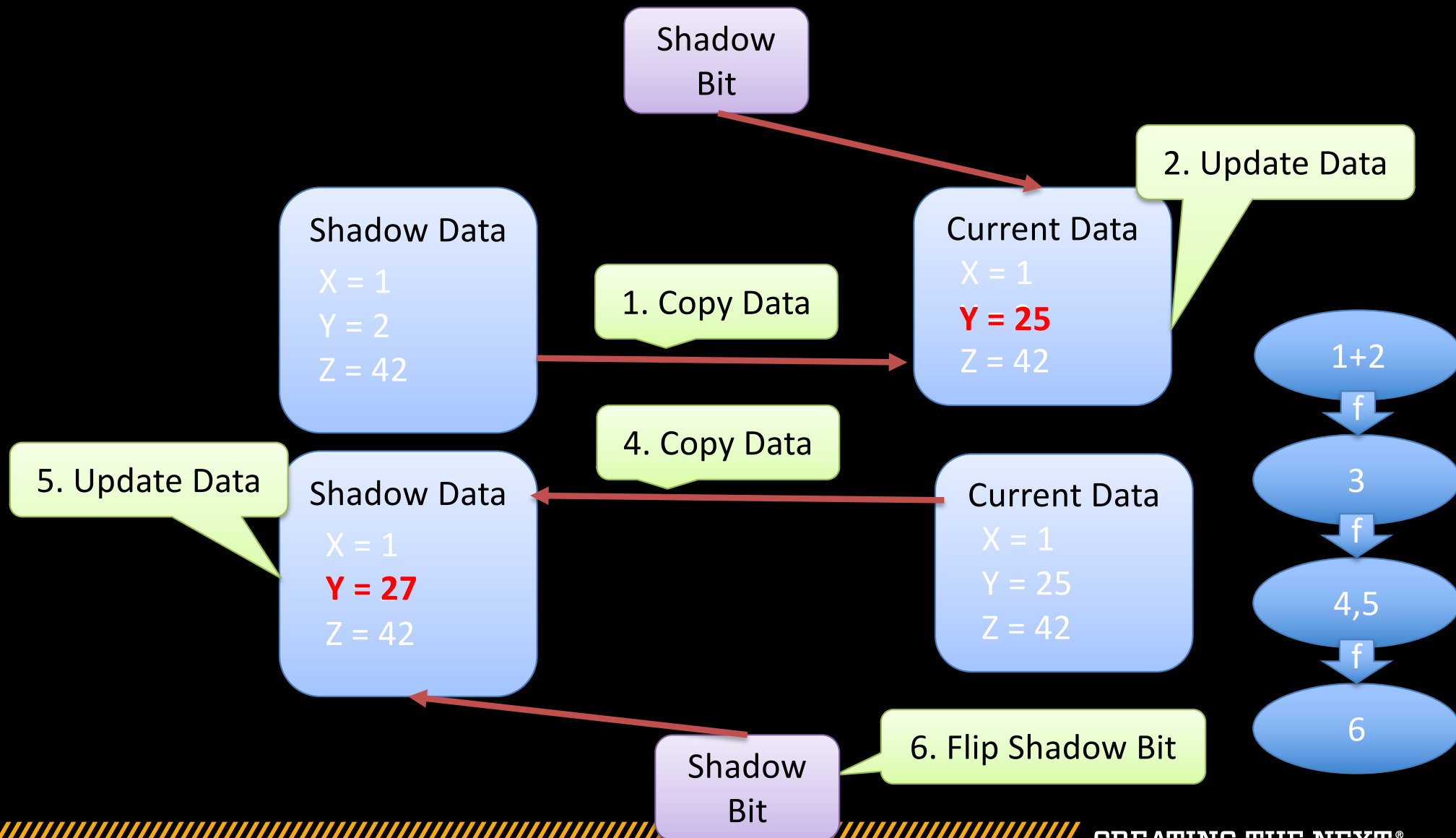
SHADOWING ILLUSTRATED



SHADOWING ILLUSTRATED



SHADOWING : CONSECUTIVE WRITES



Atomicity and the FS.

Option 2:

Journaling

We're going to keep a log of changes to the data in a "journal".

- 1) Write the change to the journal
- 2) Write a journal "commit" to atomically commit the section.

LOGGING (ALSO CALLED JOURNALING)



- Let's instead keep a log (or a journal) of data.
 - Keep a record of what the new data should be
 - Atomically “commit” records in the log
- For any “committed” log record, the log supersedes the actual data block (or metadata) in the file system.
 - If it's in the log, we take the log data
 - Otherwise, we go to the data block that's actually on disk

LOGGING ILLUSTRATED

If an entry is uncommitted, we don't consider its data valid.

Log

Entry

COMMIT

Entry

COMMIT

Entry

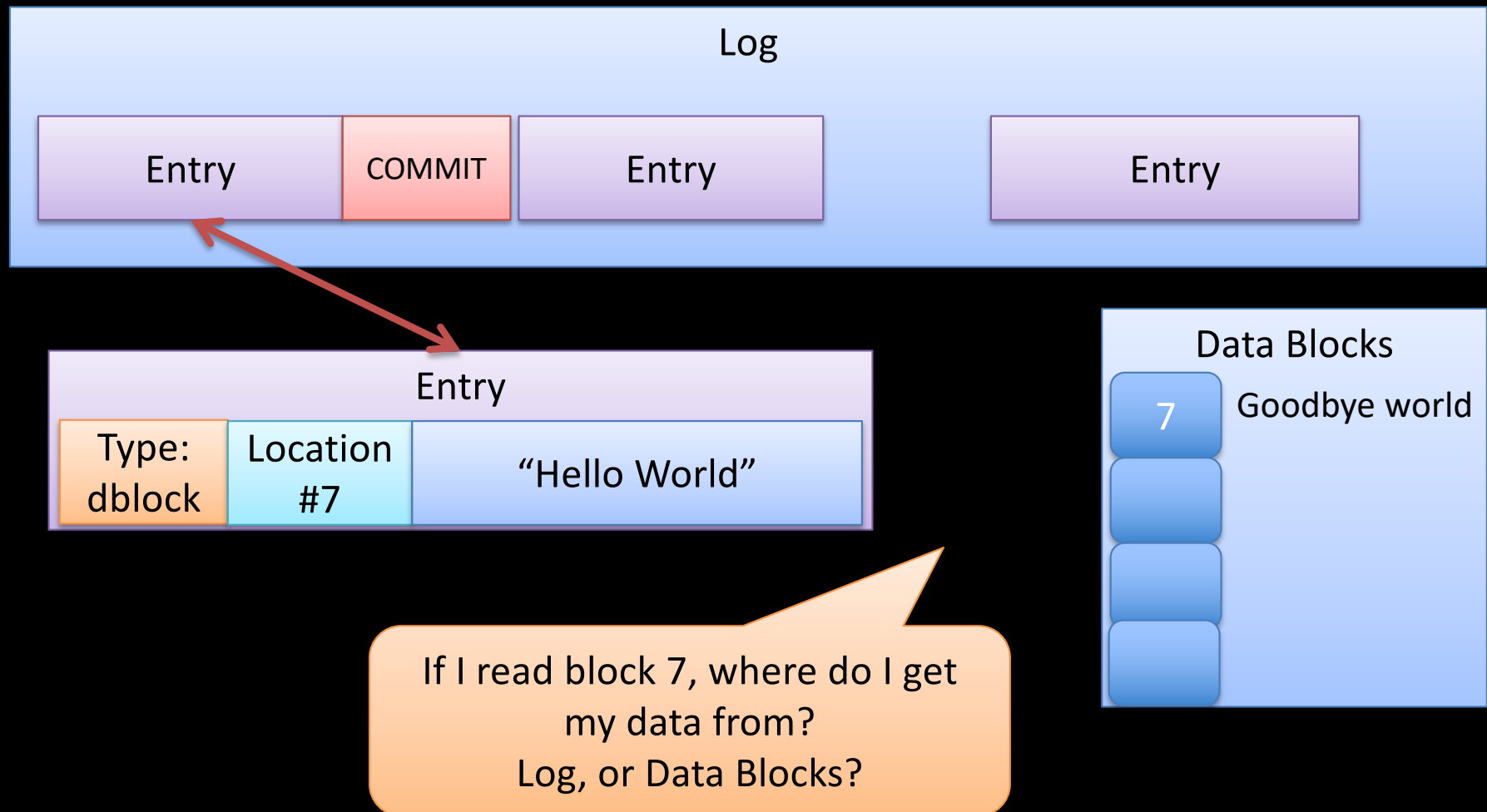
Entry

Type

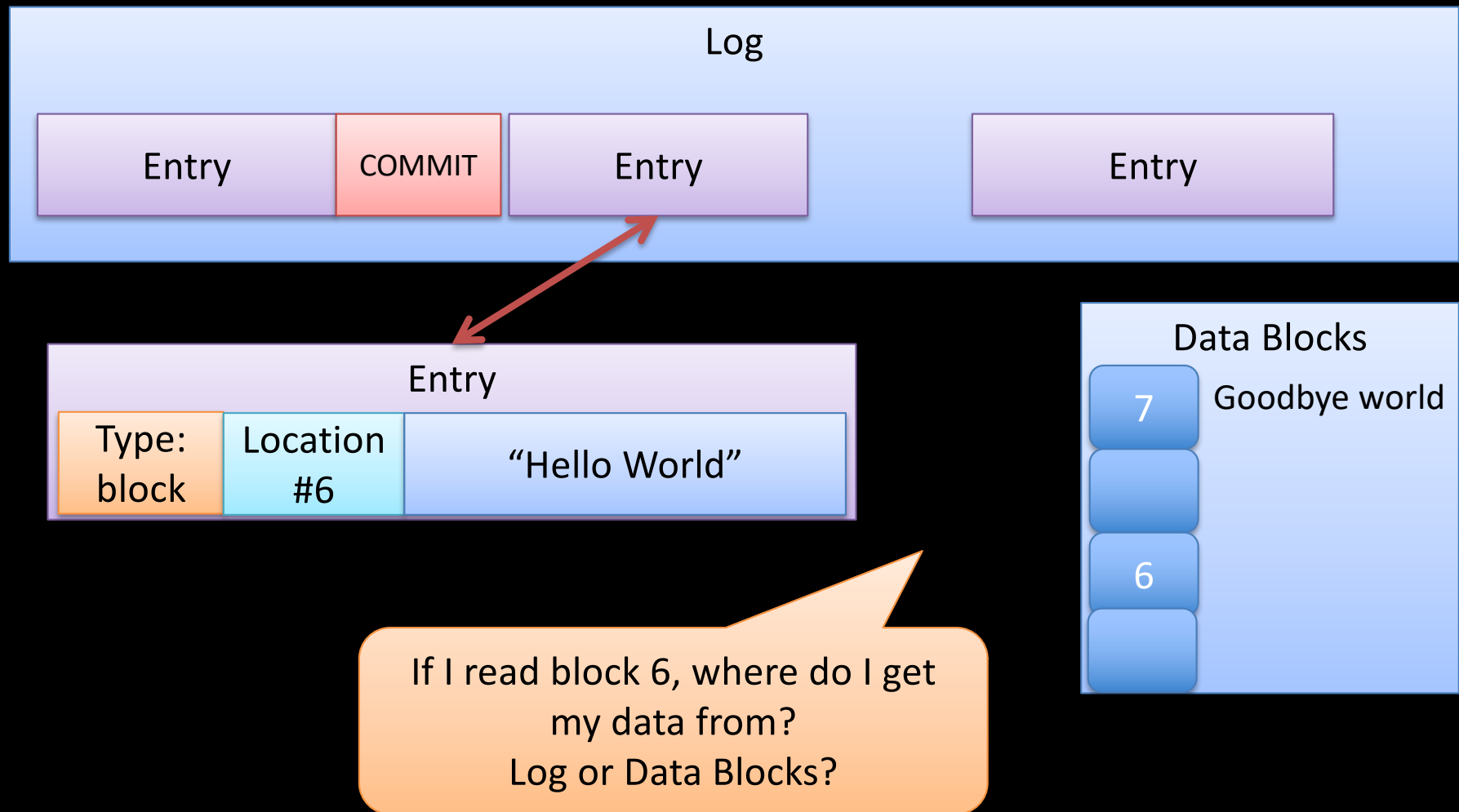
Location

Data

I/O WITH LOGGING : COMMITTED

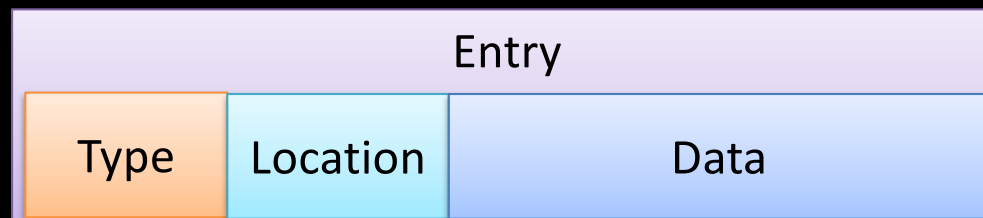
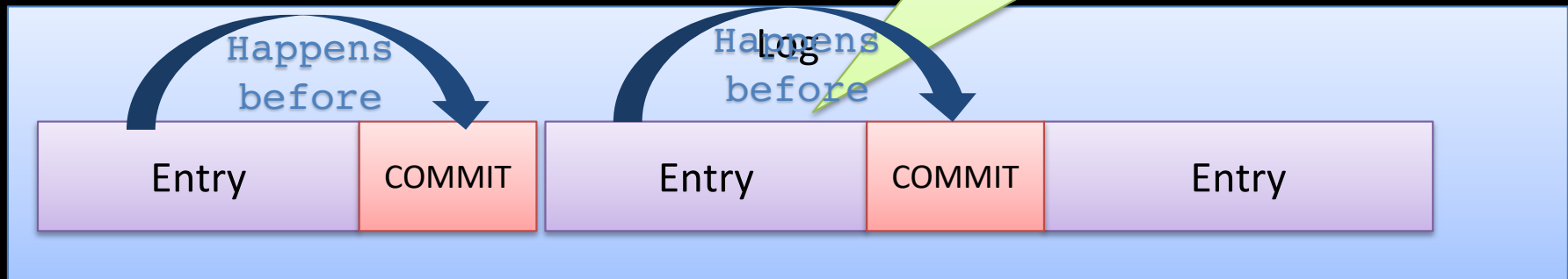


I/O WITH LOGGING : UNCOMMITTED



LOGGING ILLUSTRATED

If an entry is uncommitted, we don't consider its data valid.

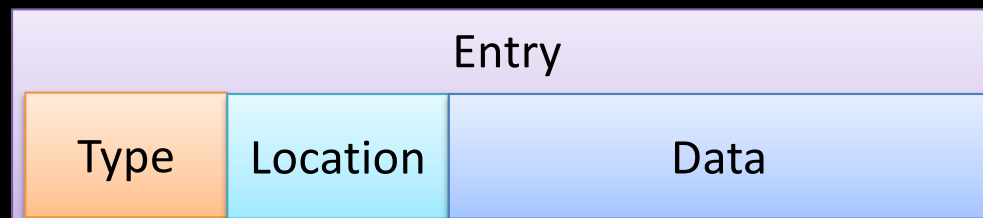
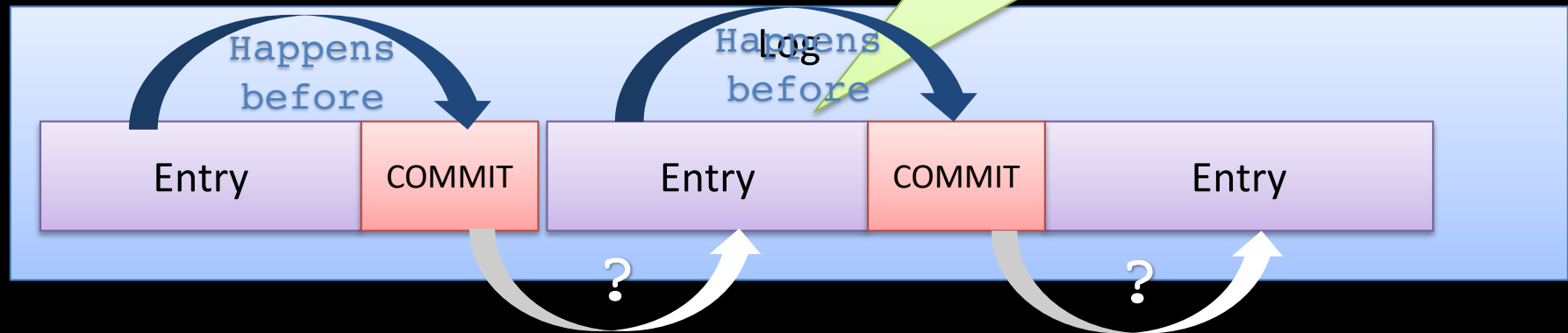


Properties of the LOG :

- Implicit ***ordering*** of operations
 - Append-only semantics on write
 - Sequential, ordered reads on recovery
 - Logs $\rightarrow$ ordering, e.g. Entry_i $\rightarrow$ happens before $\rightarrow$ Entry_{i+1}
- ***Atomicity?***
 - No, the log does not automatically provide atomicity
 - Atomicity is implemented by commit records in the log

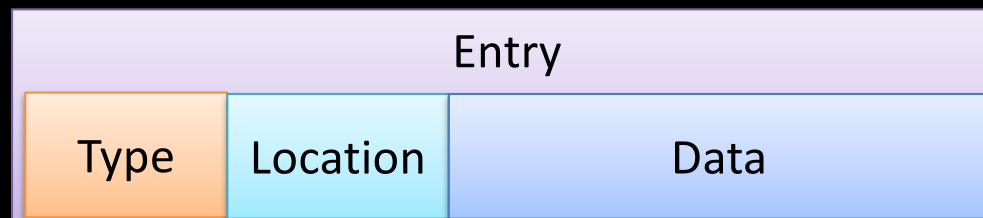
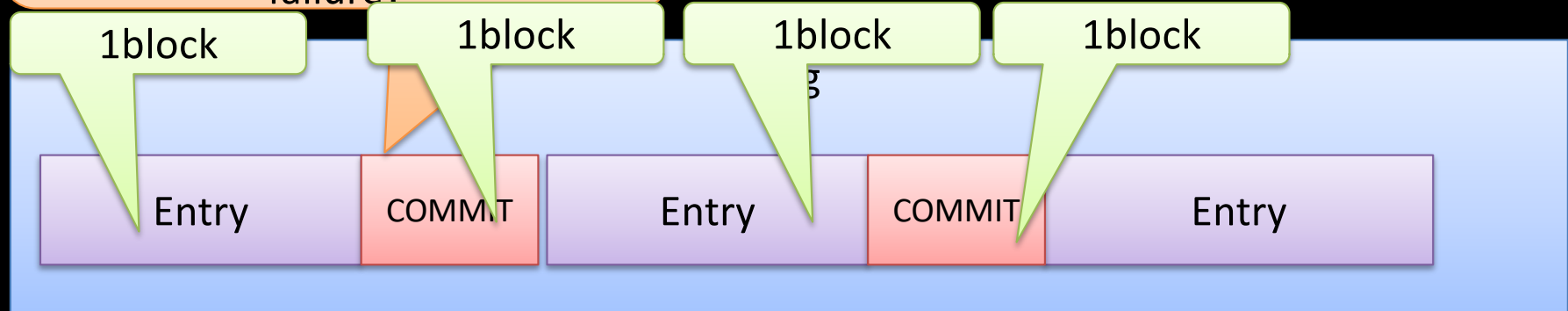
LOGGING ILLUSTRATED

If an entry is uncommitted, we don't consider its data valid.

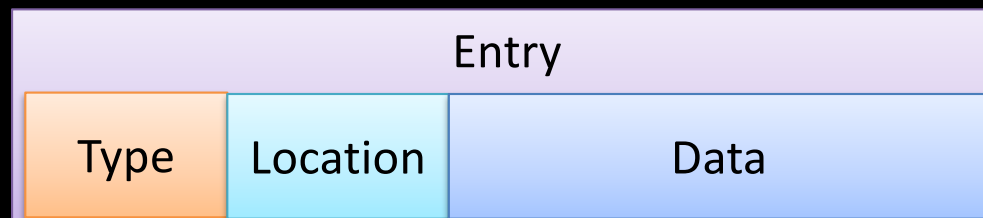
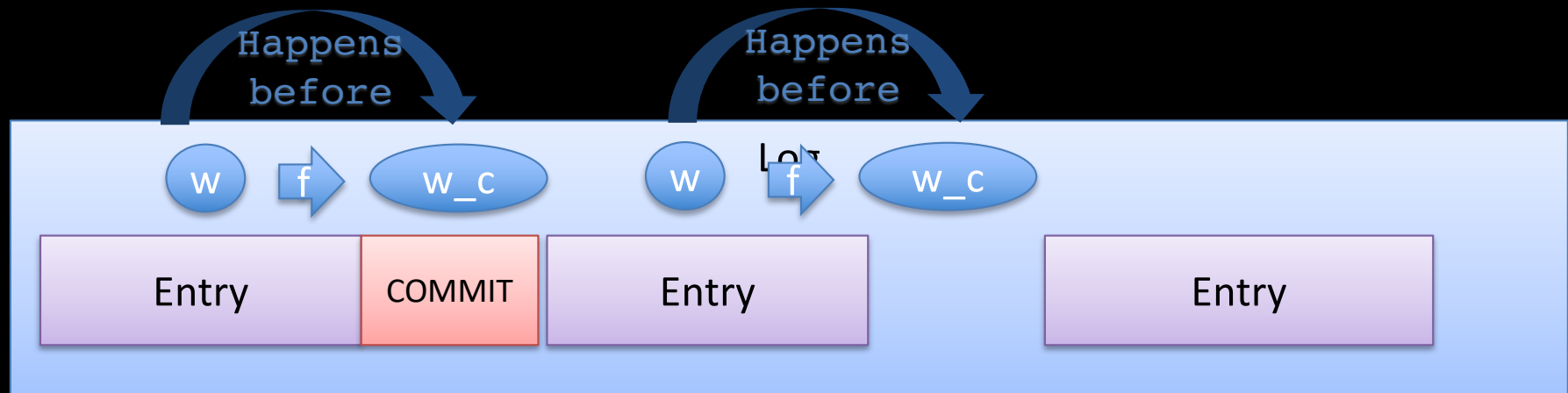


COMMIT MECHANISM 1

How can we commit an entry atomically in the instance of arbitrary disk behavior on failure?



COMMIT MECHANISM 1



CHECKPOINT IN THE DESIGN SPACE



- We've arrived at a point in the design space where
- We're getting ordering
- We're getting atomicity
- We're getting space AND time improvements over checksum
 - Space? -> no more shadow area
 - Time? -> fewer flush operations to ensure ordering
- Are we done? Can we do better?

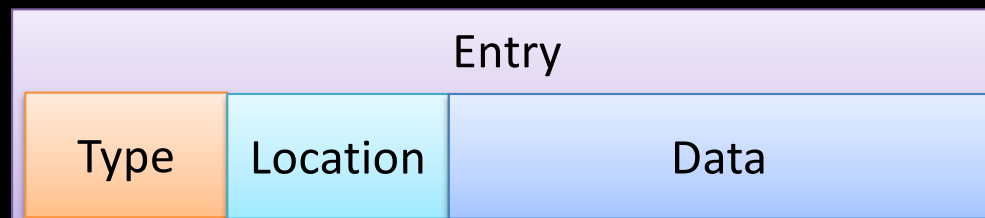
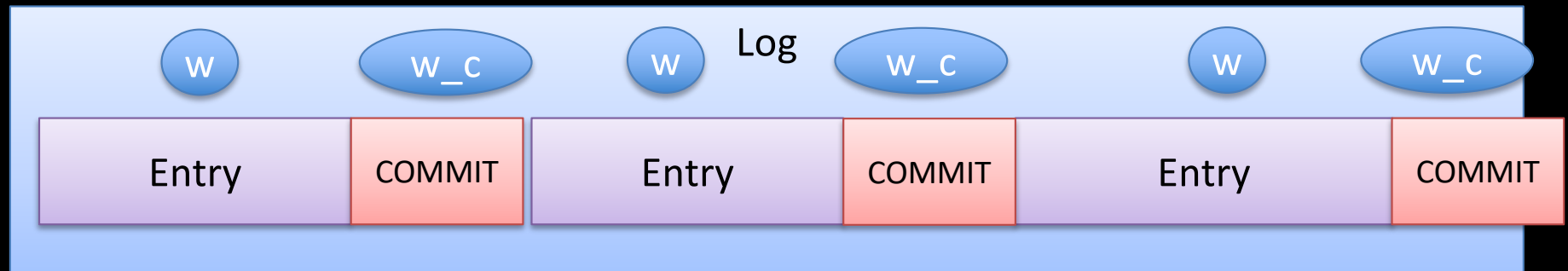
COMMIT MECHANISM 2: CHECKSUM



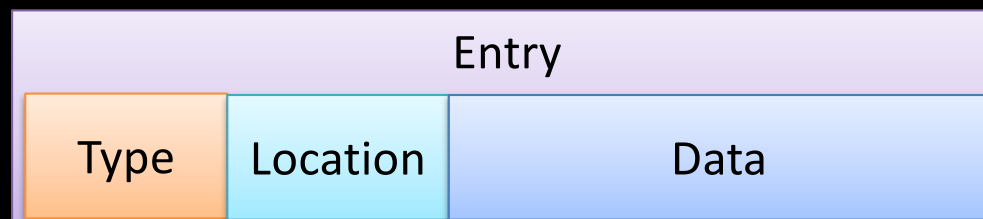
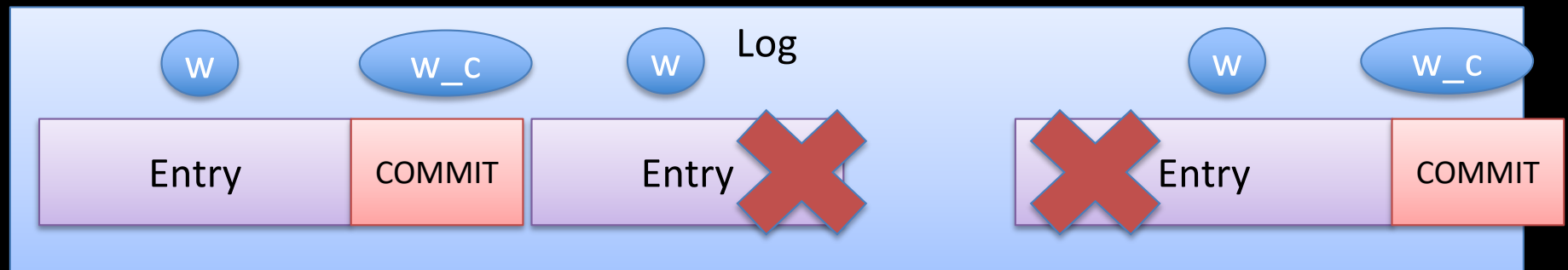
- Atomic “commit”
 - To commit an entry we use a *checksum* of the entry (handles partial writes)
 - Multiple entries on the same disk block
 - Write them all in one operation
 - Commit/checksum write : in order to “commit”, write checksum
 - If checksum doesn’t match : not committed
 - Also, an entry is only “committed” if all entries before it are “committed”
(handles out-of-order writes)

Insight: piggyback on commit functionality to validate writes

LOGGING WITH CHECKSUM COMMIT

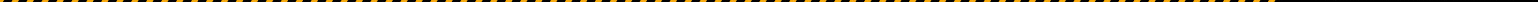


LOGGING WITH CHECKSUM COMMIT



- What do we do when **the log gets full**?
 - **Log merge** operation
 - Iterate all the **committed entries** of the log
 - Foreach entry, overwrite the “normal” version of the data with the log version
 - Once the last committed entry is processed, clear the log.
- What happens if we crash in the middle of a log merge?
 - Are we immune to that?

**Georgia
Tech**

- 
- CREATING THE NEXT®**

- What happens if we don't atomically *clear the log*?
 - How can we make log clearing atomic?
 - Well, the log only goes to the first uncommitted entry, so we can just clear out (invalidate?) the first entry.

RECOVERY



- Whenever we start up, merge the log (stopping at the first uncommitted entry).
 - Guarantee that all data is newest data
- Then, clear the log, the FS is now ready to go.

LOGGING ADVANTAGES



- ***Benefits?***

- [performance] Logs have sequential accesses! (Very disk efficient, especially for writes)
- [correctness] Provides a general form of ***Atomicity!***
- [correctness] Also provide ***Ordering!***
 - log entries are committed in **order**, preserving intended order of operations!
- [performance] Very few syncs (significant sync reduction)

- ***Costs?***

Predictability?

- ~Double the writes (once into the log, once from the log into the actual FS data when the log gets full)
- Must periodically merge log into actual data (e.g. when its full)
- May have to get latest version from log

WHAT DO REAL, MODERN FSS DO?



- Modern OS FS options:
 - Can turn *off* logging (usually a bad idea)
 - Can do *metadata only* logging (no data logging)
 - Can do “*ordered*” logging (data isn’t logged, but it's written before metadata – usually pretty safe) ****Linux Default for most Distro**
 - Can do “*full*” logging (data is logged – but is more expensive)

Atomicity and the FS.

**POSIX (the system-calls standards) –
provides very weak semantics**

What happens on a crash? Potentially bad things!

data corruption

Metadata corruption

**Modern FS's – “We need to at least protect our
metadata!”**

Atomicity and the FS.

What does ext4 provide (by default)?

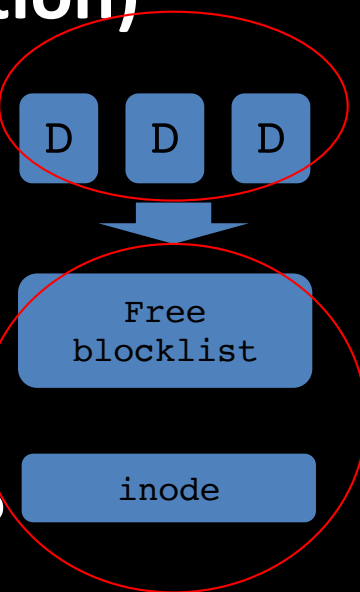
Metadata is always journaled (no FS corruption)

Data writes *ordered* before metadata

happens before

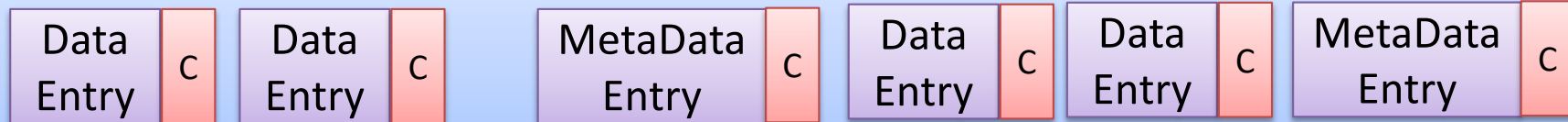
BUT, you can still get poor behavior.

What bad things can happen with data?

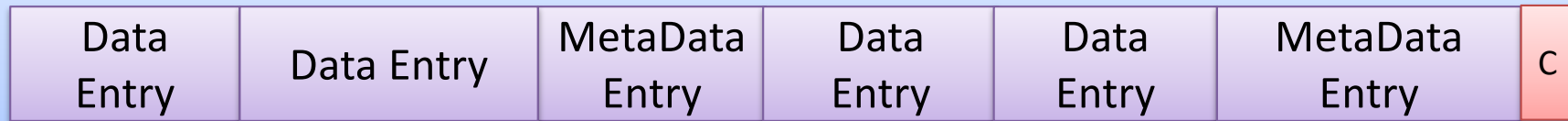


FULL JOURNALING

Log



Log



Entry



FULL JOURNALING: DISADVANTAGES



What's the problem with writing data to the log?

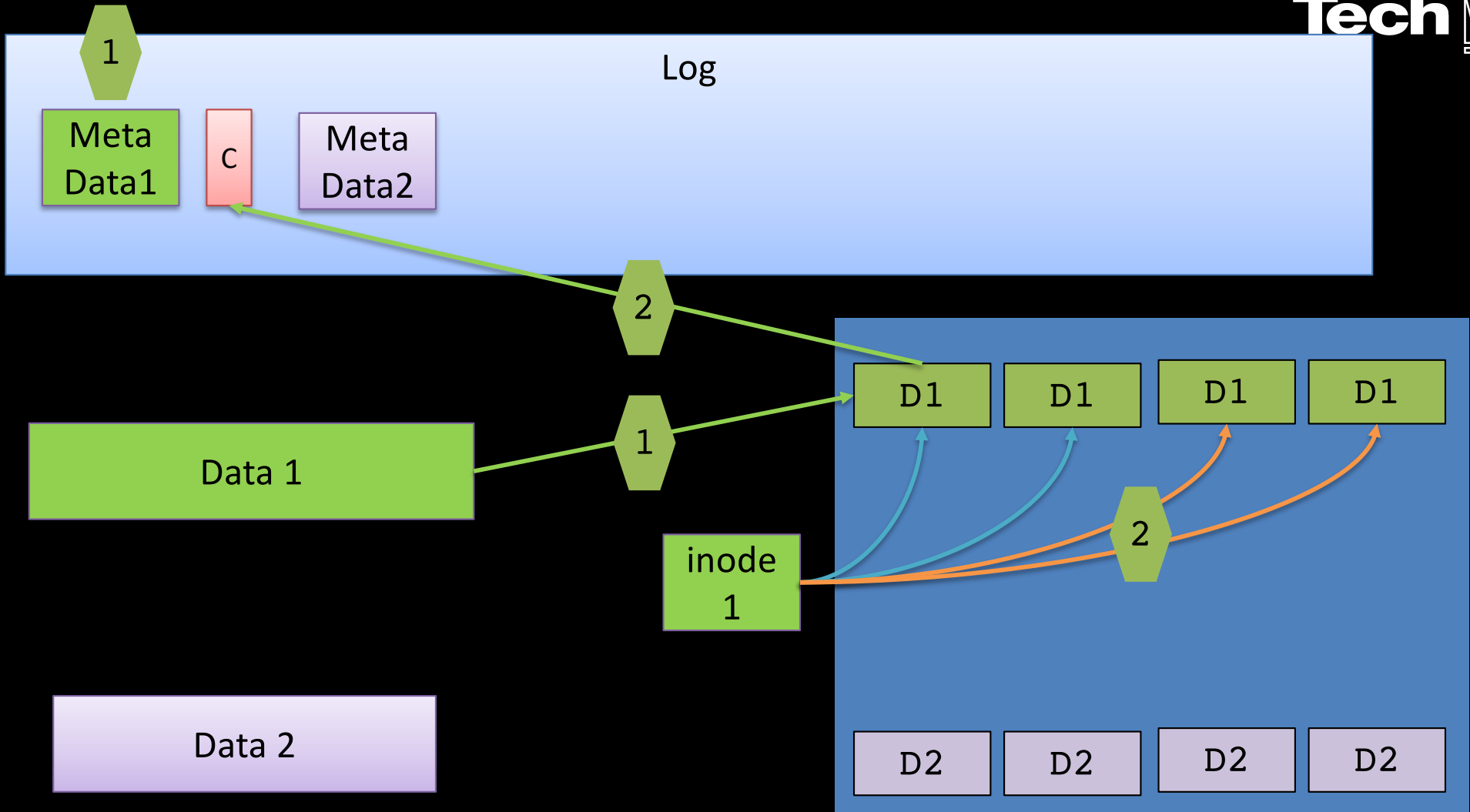
- E.g. 2GB data written, followed by a 64B inode update
- Doubling the cost of all writes, including expensive data
- ***Log capacity*** is diminished:
 - More frequent log merge operations
 - Expensive recovery
 - Expensive reads:
 - Why? (A: reading from log first)
 - Worst case perf: log usually large, traversing probabilistically large log file on every read!
 - Increases the unpredictability of read operations
 - How to quantify? (A: latency CDF!)
 - Reason about the tail latency behavior of full journaling vs others

LOGGING DATA: PROBLEMS?



- Expensive
 - Inode : 32 bytes/64 bytes?
 - Data: 2GB reasonable
- ***Head of line(HOL) blocking*** for metadata updates
 - Responsiveness of metadata updates (e.g. gcc perf)
 - Probability of failure growing proportionally to time to disk
- Ordered journaling guarantee (solution):
 - Journal metadata only

ORDERED MODE



Ordered Mode breakdown

- *Intuition*: think from the perspective of externally visible state
- *Goal*: at any given moment when inode update is visible it MUST be true that the corresponding data has been written
 - To comply with consistency of externally visible state!
- *Mechanism* (to achieve the goal):
 - Conceptually: enforce a happens before relationship between data & metadata updates
 - Practically: {data_w , metadata_w} → flush → commit_w
- *Result*: commit_w visible => metadata_w visible => data_w visible. NO stale data possible.

Atomicity and the FS.

What does ext4 provide?

Data ordering in 3 modes:

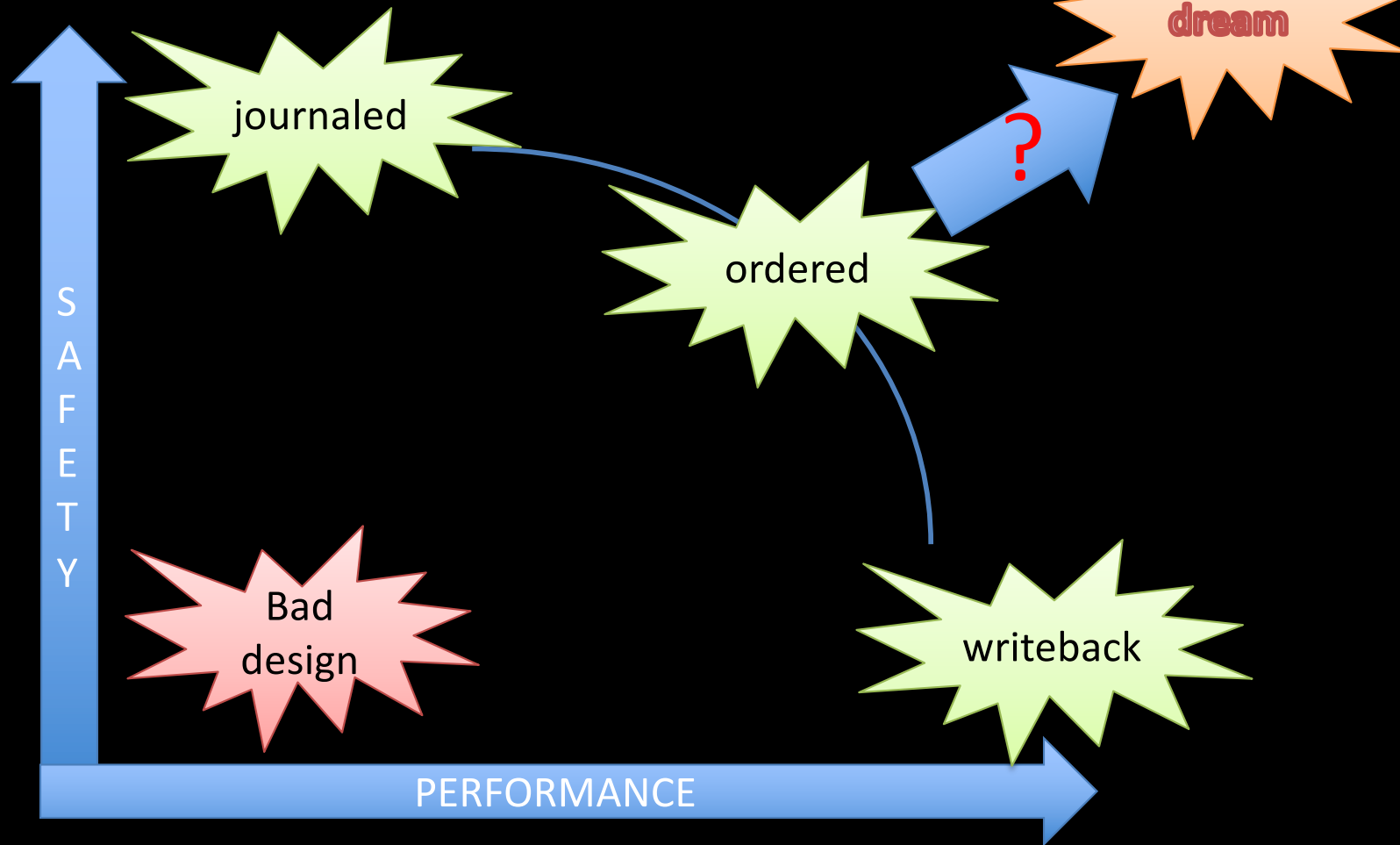
Writeback – No ordering guarantees

Ordered – Data written BEFORE metadata

Journal – Data committed in journal (atomic with metadata)

What are the implications of these modes?

JOURNALING DESIGN TRADEOFF



POSIX FS INTERFACE



- Standard Disk IO:
 - Read / Write
 - Mkdir / Create / Unlink
 - chmod / chown
 - Stat
- File Control:
 - fcntl
 - flock / funlock
- Persistency
 - **fsync – syncs a file to disk (makes the changes persistent)**

POSIX FS INTERFACE



- Persistency
 - `fsync` – syncs a file to disk (makes the changes persistent)

```
fd = open(...);
```

```
write(fd, ...data...);
```

```
// The write can be cached, we have no guarantee it's on disk
```

```
...
```

```
fsync(fd); // ~ file system ``memory barrier``
```

```
// The write is now on disk and persisted!
```

Some thoughts:

What does an fsync do within the kernel?

How do our ordering / atomicity mechanisms interact with fsync?

INTERACTION BETWEEN FSYNC AND LOG

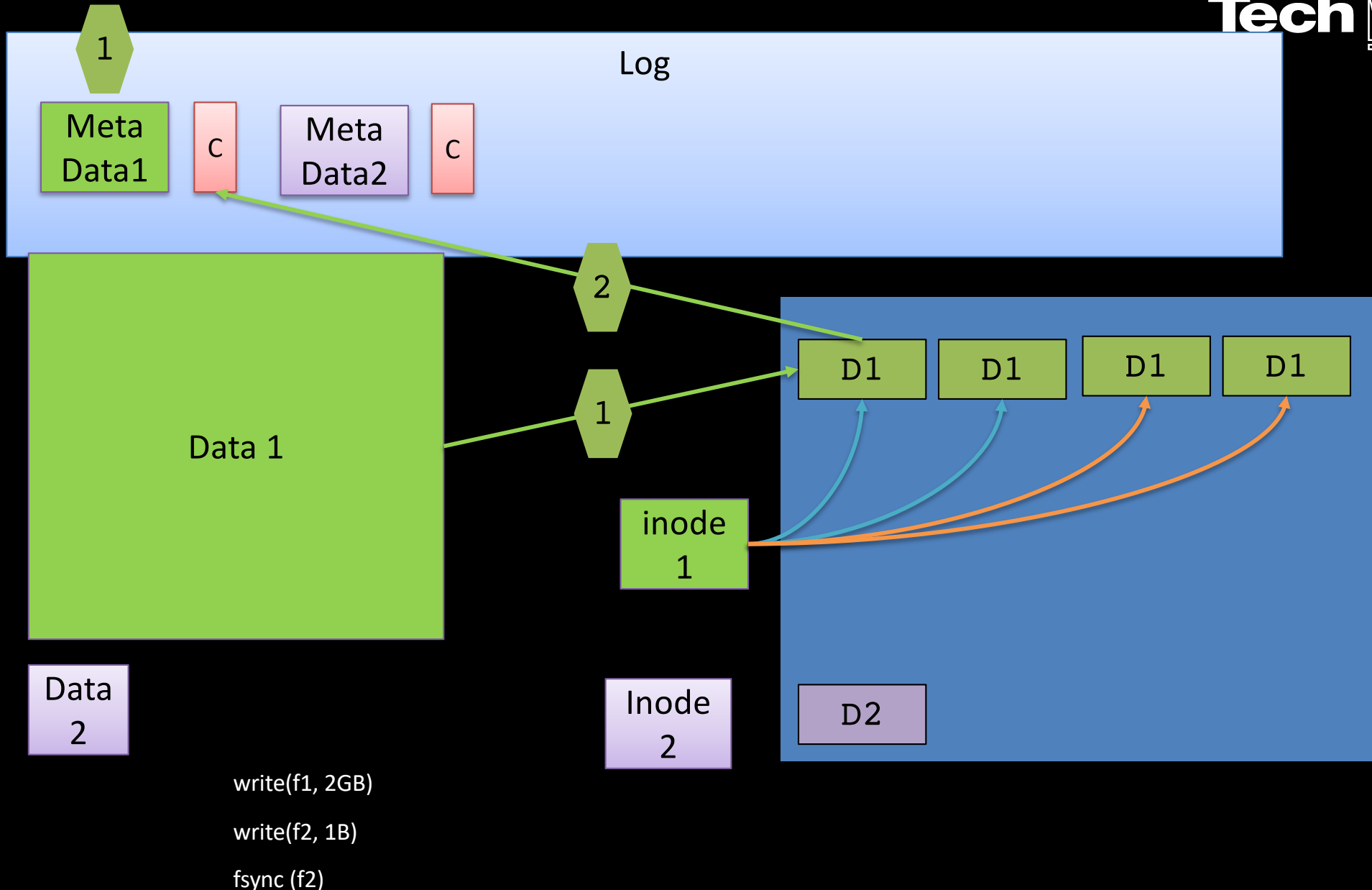


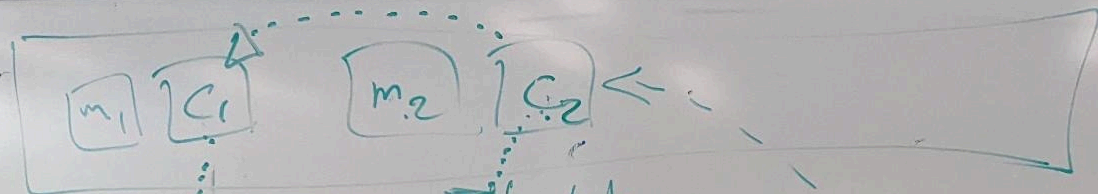
“Logging, flushing, and data confusion”

Problem with fsync : sequential logging

Writes to files can bottleneck other writes .

INTERACTION BETWEEN FSYNC AND LOG





f_1 data

f_2 data

Data Block

Data Block f_1

$W W$

f_2

$W W$

$fsync(f_2)$

f_1
 $W W$

f_2
 $W W$ $fsync$

"depends on" relation

The downside of logging:

Combine all FS data into one log

Places total ordering on data –

Flushing any file, flushes whole log!

Makes flush operations expensive!

Takeaway: fsync operations REALLY expensive

Today:

Atomic user/app-level writes (POSIX level)

How do we do them? (*mechanism*)

Why these are slow?!? (*analysis*)

How to make them better?

(Including latest research on the topic)

Today:

**Let's assume we wanted to modify a file in a crash-safe
way**

.

How would we do this via the Posix standard?

What promises does the **POSIX** standard give me?

- Remember, POSIX must provide promises consistent with all reasonable filesystems
 - Very weak guarantees
- After an `fsync`, my data will be persisted to disk
 - Before the sync, no guarantees about what's in the file – could be corrupt

Modify a file

Assume we have a file “foo” in directory “bar”:

How do we modify it atomically according to POSIX standard?

- 1. Make a log file**
- 2. Store updates in log**
- 3. Update actual file**
- 4. Delete log**

Modify a file atomically

Create(log)

Write(log, "offset", size, data, checksum)

```
Fsync(log)
```

```
Fsync("." / " ")
```

Write(file, data)

```
Fsync(file)
```

Unlink(log)

```
Fsync("." / " ")
```

ORDERING AND STUFF



Modify a file w/ logging

Generally:

```
Create(log);  
write(log, "offset, size, data, cksum")  
fsync(log)  
fsync("./")  
write(file, ...update data...)  
fsync(file)  
unlink(log)  
fsync("./") – otherwise log isn't updated!
```

PHEW!

Modify a file w/ logging

Why do I need so many syncs?

Is it because I need my data *persisted* to disk?

OR

Do I care about the data *ordering*?

Modify a file w/ logging

Generally:

Create(log);

write(log, "offset, size, data, cksum")

fsync(log)

fsync("./")

write(file, ...update data...)

fsync(file)

unlink(log)

fsync("./") – otherwise log isn't updated!

PHEW!

Modify a file w/ logging

Why do I need so many syncs? Is it because I need my data persisted to disk?

No!

I'm syncing because I want **Ordering**.

*The issue here is that ordering and persistency are the same operation in Posix: **fsync***

Research work:

“Optimistic Crash Consistency” SOSP 2013

“The issue is, we conflate ordering and persistency”

osync

dsync

We often need ordering for *correctness* (think of our logging example),

but we don’t need *persistency* now.

Fsync provides both, even though we don’t always need both!

Research work:

**“Optimistic Crash Consistency”
SOSP 2013**

“The issue is, we conflate ordering and persistency”

- **Split fsync into two operations:**
 - **osync – ordering primitive**
 - **dsync – durability primitive**

Modify a file atomically

Create(log)

Write(log, "offset", size, data, checksum)

```
osync(log)
```

```
osync(" ./")
```

Write(file, data)

```
osync(file)
```

Unlink(log)

```
Fsync(" ./")
```

ORDERING AND STUFF



This is the end of our FS lectures

Up next:

Exploring challenges in “Distributed Systems”

DISTRIBUTED SYSTEMS

ALEXEY TUMANOV

School of Computer Science
cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

Administrativa

Lab4-net – DUE THIS Friday , 4/24

Exam 2: TWO options:

Tue, Apr 28, 6:30-8:30pm

- Locations: your supervised lab location + time

Thur, April 30 (2:40pm – 4:40pm) (Section A)

- Howey Physics L1

Choose on [canvas](#) (due 4/21 midnight)

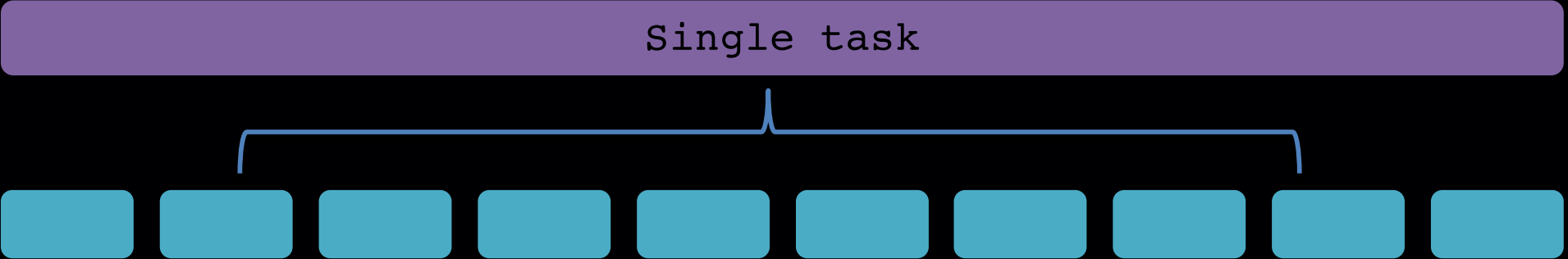
What is a distributed system?

What comes to mind when you think of distrib. Systems?

Many nodes

Working together

On a single task



Single task

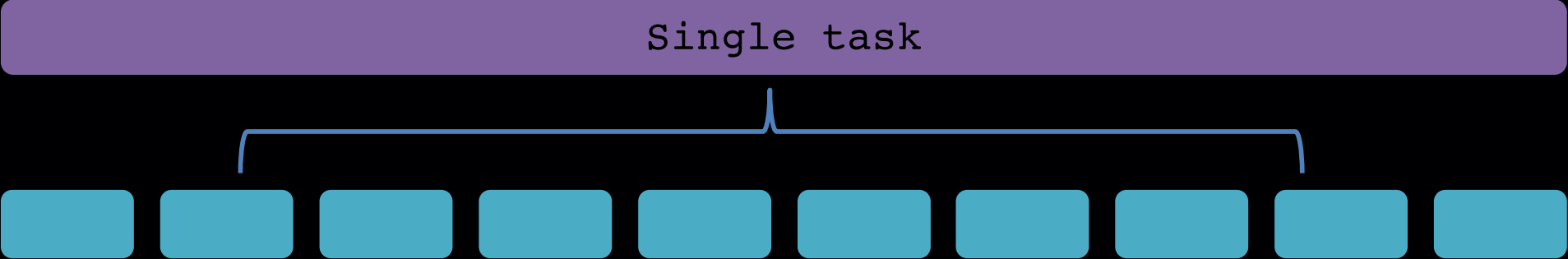
The diagram illustrates a distributed system architecture. At the top, a long purple rounded rectangle is labeled 'Single task'. A blue line with a central vertical hook extends downwards from the bottom center of this rectangle. This line branches into two horizontal lines that connect to the top of a row of ten light blue rounded rectangles. These rectangles are arranged horizontally and represent individual nodes in the system. The entire diagram is set against a dark blue background.

What is a distributed system?

A distributed system is a collection of processors that do not share memory or a clock.

Operating System
Concepts,
Silberschatz et al

Single task



The diagram illustrates a single task being distributed across multiple processors. A purple bar at the top is labeled 'Single task'. A blue line with a central hook connects this bar to a row of ten blue squares below it, representing individual processors. The entire diagram is set against a dark blue background with a yellow and black striped border at the bottom.

Why do we want distributed systems?

Why do we want distributed *software* systems?

Resource sharing

Computation
Speedup

Reliability

How does it relate to an operating system?

Multiple
resources

Working
together

To act like one

- **Similar *properties + goals***
- **Different *challenges*:**
 - **primarily coordination over unreliable comm media**
- **Similar challenges: ordering, atomicity, concurrency**

Distributed systems aim to make many computers look like one.

Does this sound familiar?

It provides similar abstractions to those we see within the OS, its essentially a multi-PC OS!

What are those abstractions?

DISTRIBUTED SYSTEMS: DIFFICULTY



What makes a distributed system *hard*?
Well, let's formalize the problem more:
We want –
two computers to agree on something.

They communicate over an unreliable network,
messages can be dropped, and get delayed.

Operating
Systems

R. Stockton Gaines
Editor

Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport
Massachusetts Computer Associates, Inc.

The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events. A distributed algorithm is given for synchronizing a system of logical clocks which can be used to totally order the events. The use of the total ordering is illustrated with a method for solving synchronization problems. The algorithm is then specialized for synchronizing physical clocks, and a bound is derived on how far out of synchrony the clocks can become.

Key Words and Phrases: distributed systems, computer networks, clock synchronization, multiprocess systems

CR Categories: 4.32, 5.29

Introduction

A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system. A single computer can also be viewed as a distributed system in which the central control unit, the memory units, and the input-output channels are separate processes. A system is distributed if the message transmission delay is not negligible compared to the time between events in a single process.

We will concern ourselves primarily with systems of spatially separated computers. However, many of our remarks will apply more generally. In particular, a multiprocessing system on a single computer involves problems similar to those of a distributed system because of the unpredictable order in which certain events can occur.

In a distributed system, it is sometimes impossible to say that one of two events occurred first. The relation "happened before" is therefore only a partial ordering of the events in the system. We have found that problems often arise because people are not fully aware of this fact and its implications.

In this paper, we discuss the partial ordering defined by the "happened before" relation, and give a distributed algorithm for extending it to a consistent total ordering of all the events. This algorithm can provide a useful mechanism for implementing a distributed system. We illustrate its use with a simple method for solving synchronization problems. Unexpected, anomalous behavior can occur if the ordering obtained by this algorithm differs from that perceived by the user. This can be avoided by introducing real, physical clocks. We describe a simple method for synchronizing these clocks, and derive an upper bound on how far out of synchrony they

DISTRIBUTED SYSTEMS PROPERTIES



Single Node OS:

- no failures
- Single clock source

Distributed Systems:

- Failures
- Independent clock source

What else is different?

- Mindset/Design approach:

- 
- ***Monolithic, global*** information mindset
 - Relying on shared memory for coordination
 - ***“think like a vertex”*** mindset
 - Operating with ***partial*** information
 - Relying on message passing for coordination

- Source of time: causality dependencies

DISTRIBUTED SYSTEMS: 2 GENERALS



Classic problem: Two generals problem:

The Mongol hoard is attacking through the mountain pass

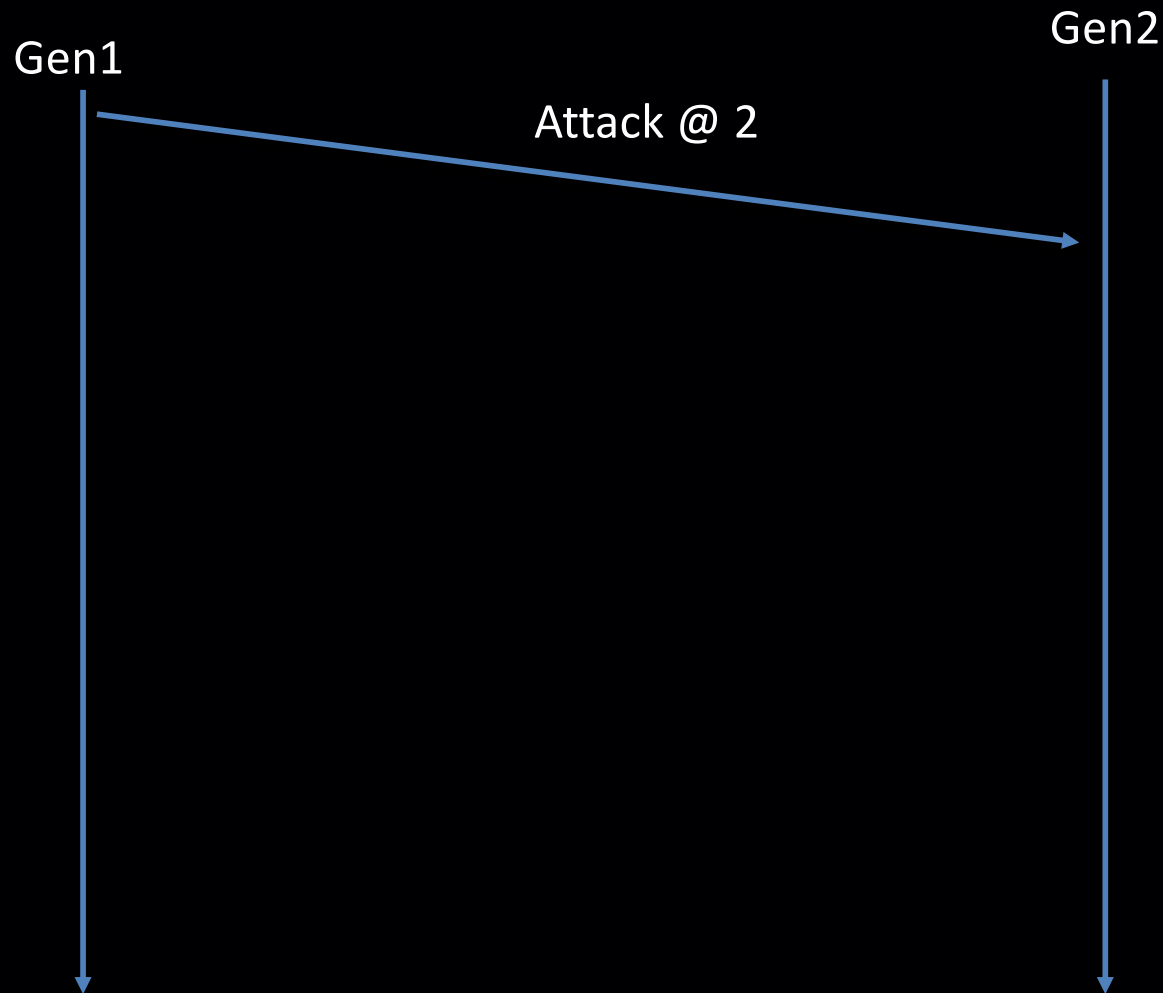
The two generals must attack together to beat the hoard (if they attack alone they'll die)

They send messengers to communicate

How are they sure they can attack the hoard together?

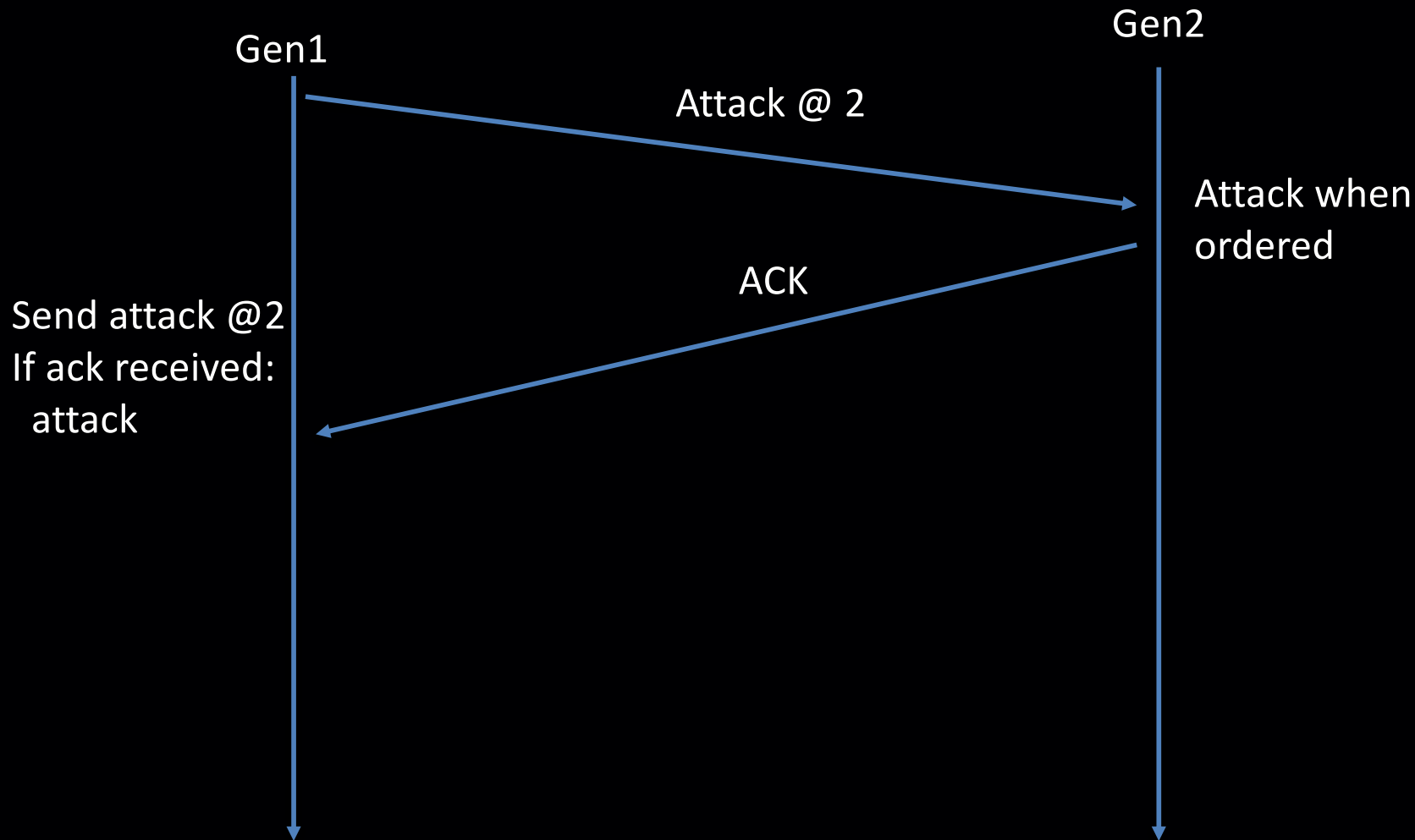
DISTRIBUTED SYSTEMS: 2 GENERALS

Classic problem: Two generals problem:



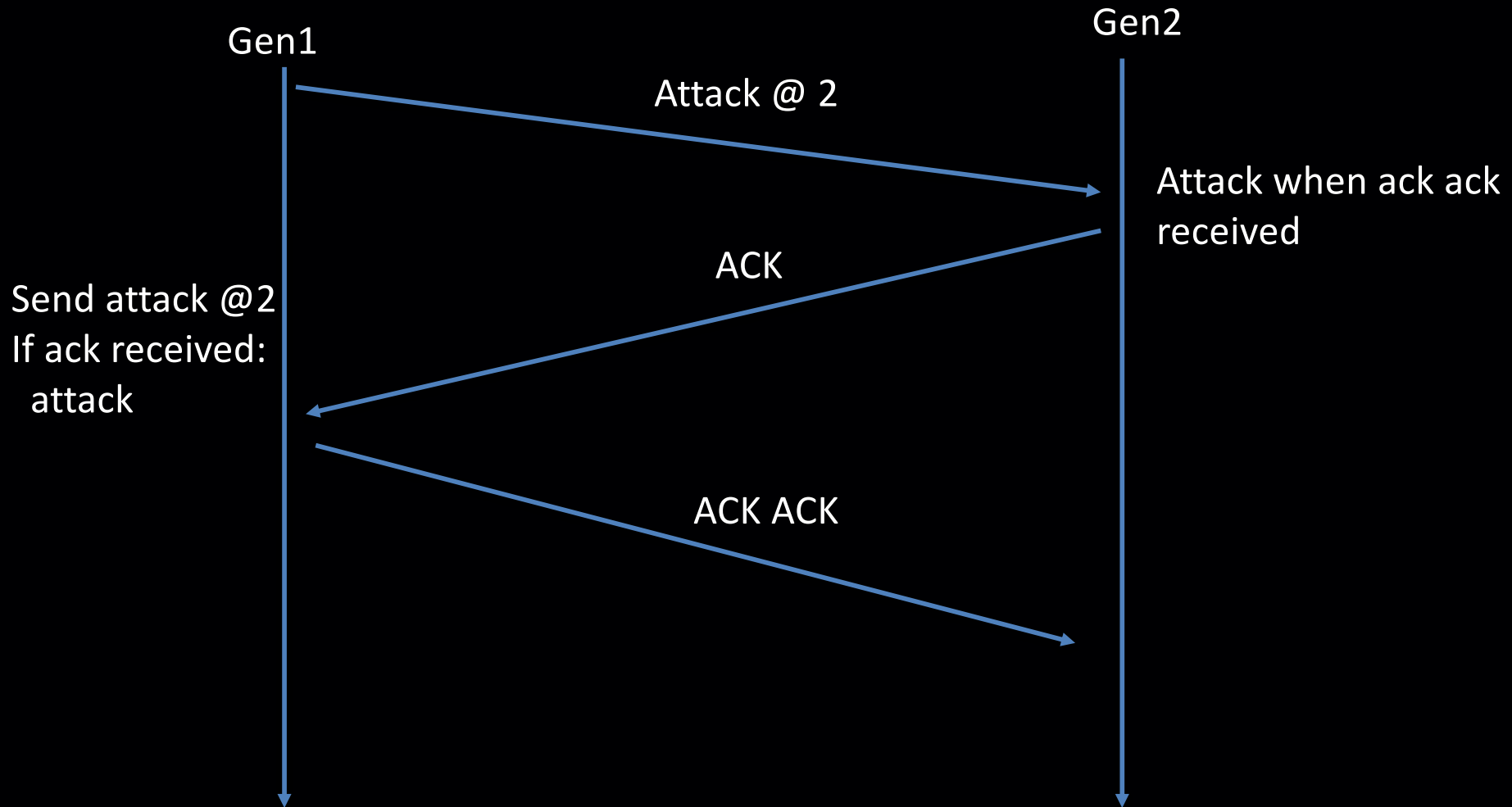
DISTRIBUTED SYSTEMS: 2 GENERALS

Classic problem: Two generals problem:



DISTRIBUTED SYSTEMS: 2 GENERALS

Classic problem: Two generals problem:



2 GENERALS : IMPOSSIBILITY



Classic problem: Two generals problem:

It turns out, this is impossible!

Can you construct a proof?

2 GENERALS: PROOF OF IMPOSSIBILITY



Classic problem: Two generals problem:

It turns out, this is impossible!

Can you construct a proof?

Imagine you could solve the problem in N messages.

If so, then message $N-1$ could be dropped

If message $N-1$ could be dropped, you could solve it in $N-1$ messages.

Since you cannot solve it in 0 messages this is impossible.

It gets worse!

FLP Theorem:

In an **asynchronous** network where messages **may be delayed but not lost**, there is **no consensus algorithm** that is **guaranteed to terminate** in every execution for all starting conditions, if at least one node may fail-stop.

Fail-Stop Failure: Fail-stop failure is a type of failures that cause the component of a system experiencing this type of failure to stop operating

DISTRIBUTED SYSTEMS: IMPOSSIBILITY



To recap, we want to have 1000s to 10s of 1000s of computers working together to solve the same problem

But, its impossible for two computers to come to consensus on anything without prior knowledge, and for any computers to come to consensus under even weaker constraints!

How do we do this?

Are we doomed?

It gets worse!

FLP Theorem:

In an **asynchronous** network where messages **may be delayed but not lost**, there is **no consensus algorithm** that is **guaranteed to terminate** in every execution for all starting conditions, if at least one node may fail-stop.

Fail-Stop Failure: Fail-stop failure is a type of failures that cause the component of a system experiencing this type of failure to stop operating

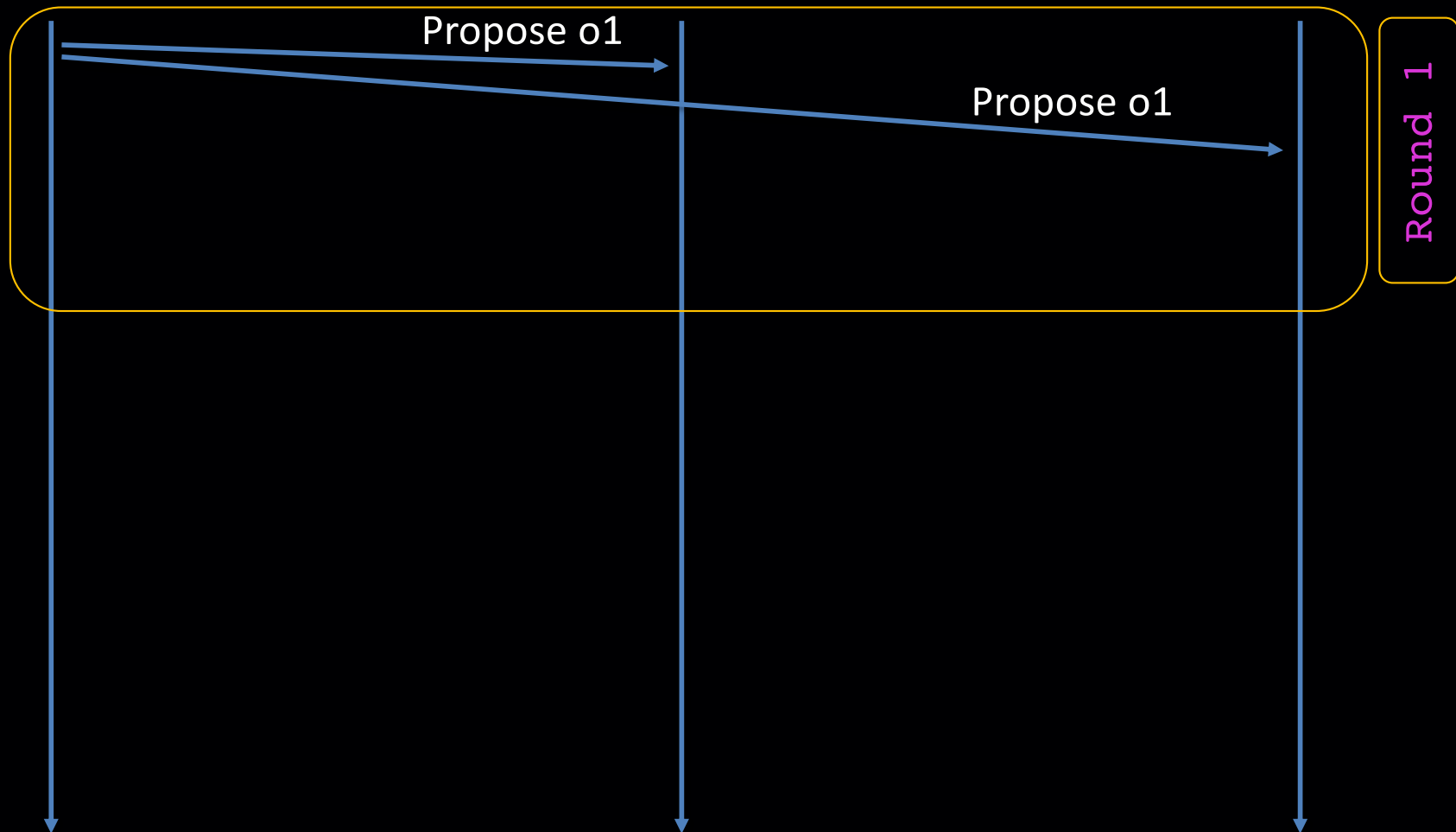
Option 1:

Use timeouts.

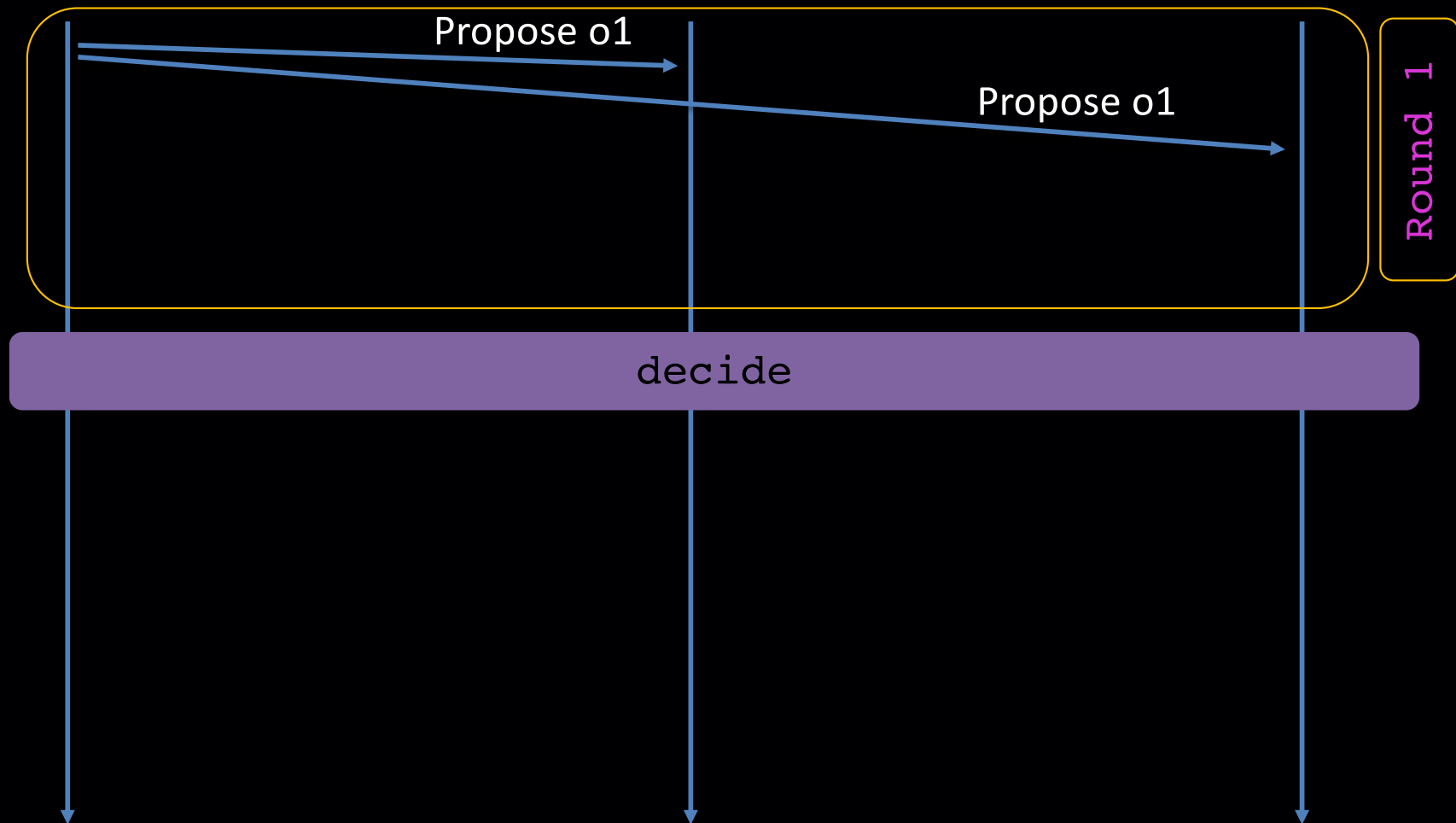
**Can we construct a consensus protocol that uses
timeouts?**

Assume if we have $2f+1$ nodes, f may fail.

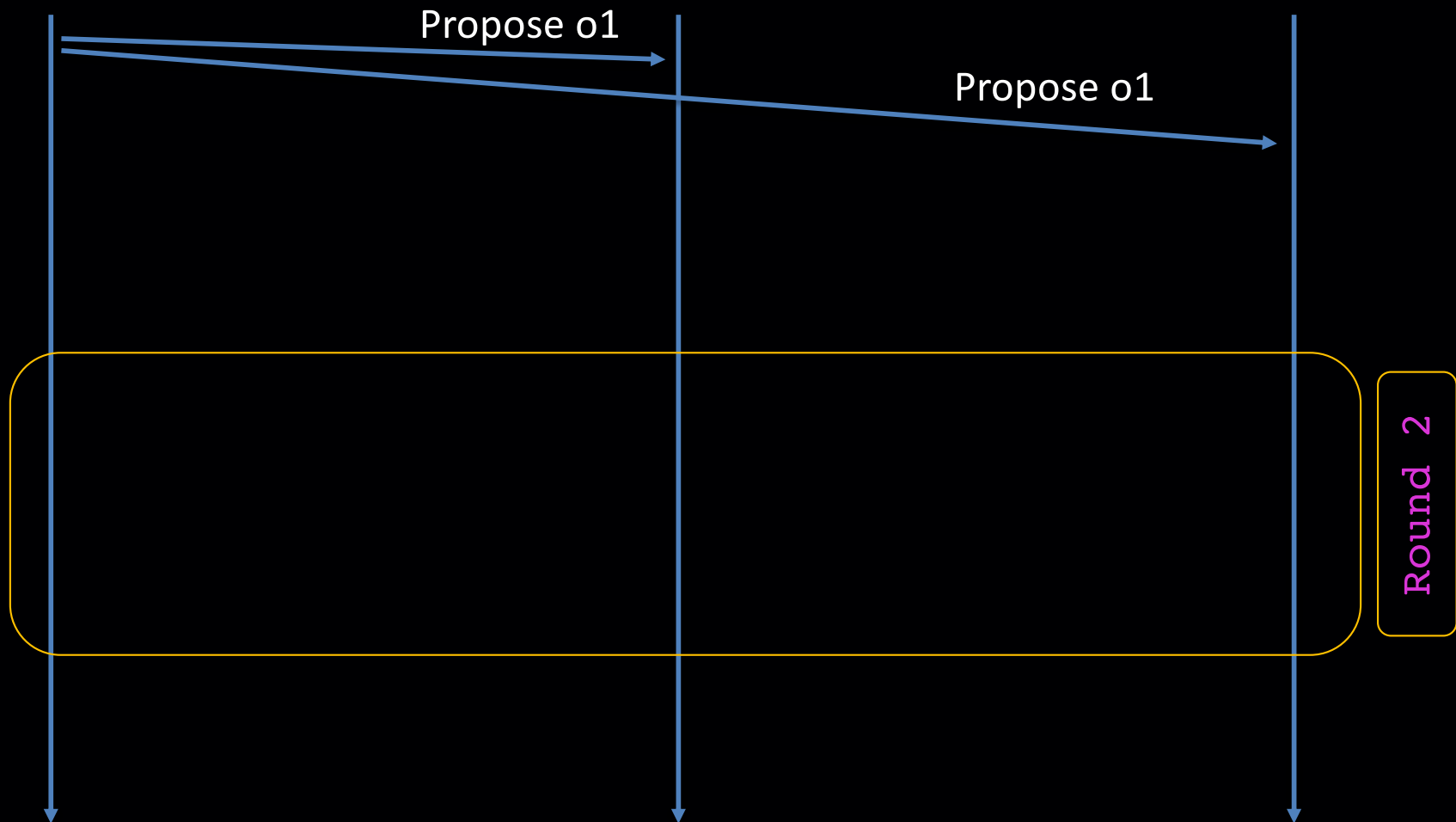
DISTRIBUTED SYSTEMS: TIMEOUTS

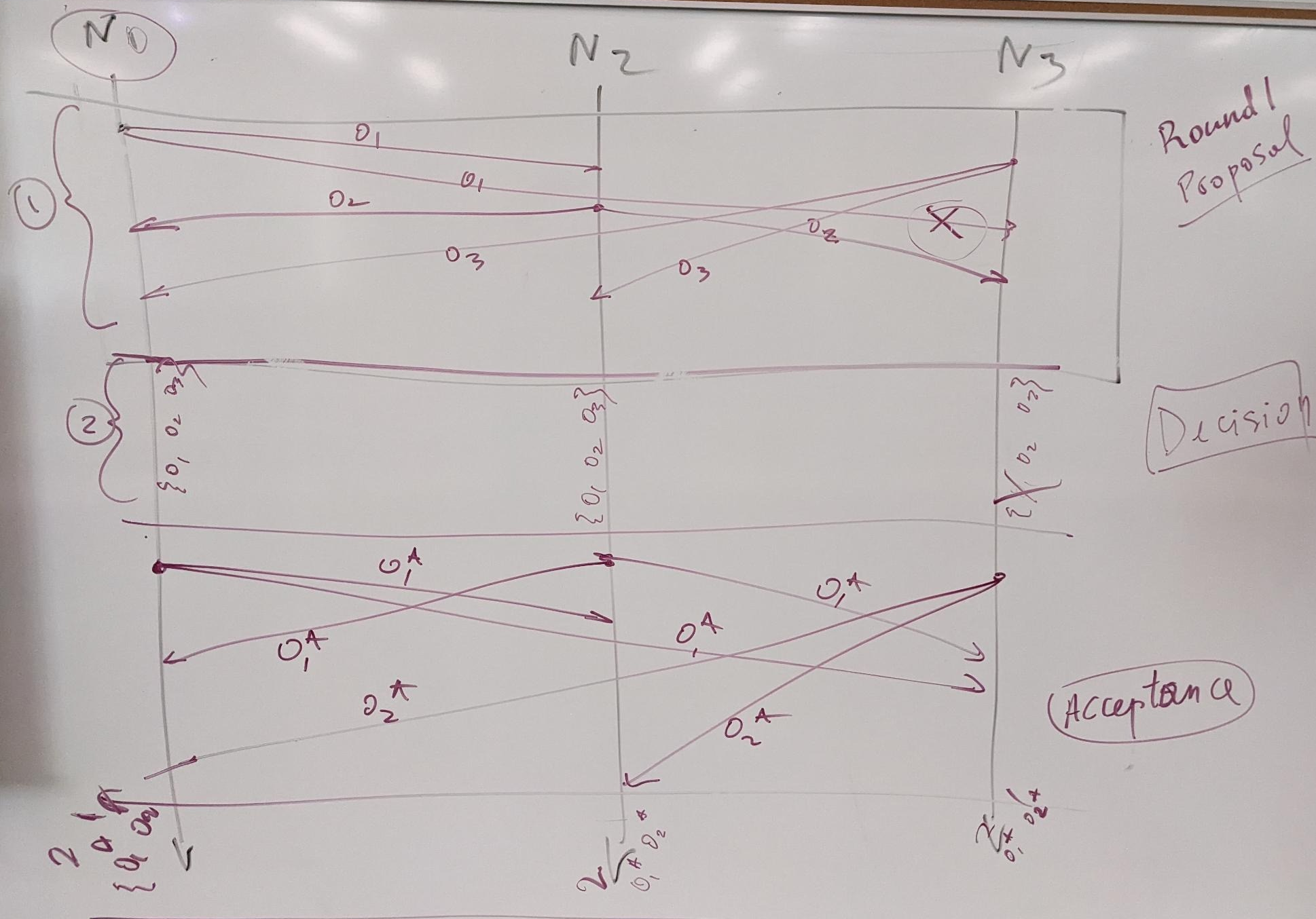


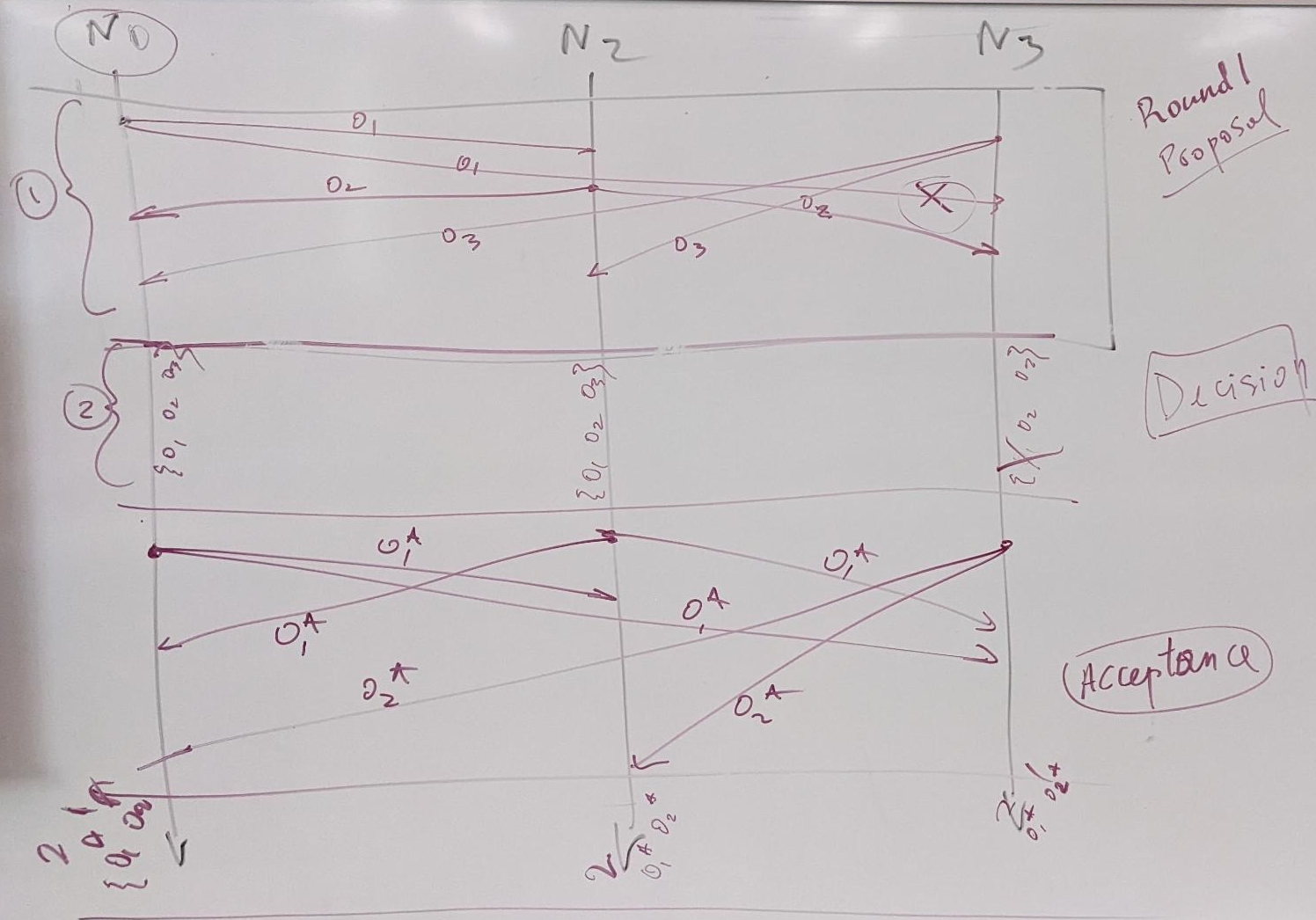
DISTRIBUTED SYSTEMS: TIMEOUTS



DISTRIBUTED SYSTEMS: TIMEOUTS







$$f=1$$

$$2f+1=3 = \boxed{f} + \boxed{f+1}$$

$$f \rightarrow \text{fail}$$

SYNCHRONOUS ROUNDS: PROBLEMS



Round = a reasonable amount of time that a message can take from point a to point b

- Has to be a conservative over-estimate
- It's slow to build algorithms on top

Option 2:

Paxos

From Leslie Lamport

**Original paper (only for the very brave: “The Part-Time
Parliament”)**

Simpler version: “Paxos Made Simple”

Option 2:

PAXOS

Complex protocol, involves leaders, election, and multi-phase commit

Important things to know:

****mostly** asynchronous (no waiting), uses leaders to coordinate**

BUT, can't be fully asynchronous (remember FLP?) – uses synchronous protocol for leader election and failure

PAXOS: HIGH LEVEL OVERVIEW



Paxos: $2f+1$ nodes where f may fail

Request is submitted to the leader,

The leader will broadcast to the nodes

The nodes come back to the leader

There's also a listener that checks leader liveness

The listener responds to the client

Up to f nodes can die.

Fully asynchronous consensus with $\leq f$ failures given
synchronous leader election

PAXOS: PROBLEMS



But, a Paxos cluster drops things when the leader dies...

Yes! Paxos loses “availability” when the leader dies.

For someone like “Amazon” who needs ~100% availability, this is likely not okay! What can they do?

Properties of a distributed system:

Consistency – All machines have the same view of the data

Availability – Stays live in the instance of machine failures

Partition Tolerance – You cannot get subsets of machines working independently

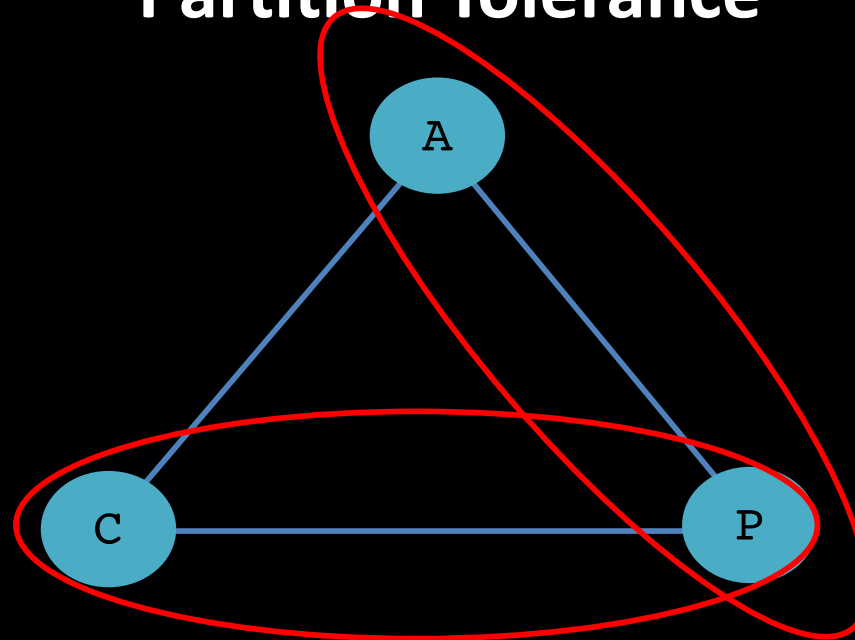
Paxos provides **consistency** and **partition tolerance**

Can we also provide availability?

Once again, its impossible!!!

CAP Theorem

You can only have 2 of Consistency, Availability, and Partition Tolerance



DISTRIBUTED SYSTEMS: A+P IN CAP



Once again, its impossible!!!

There are other systems that make other tradeoffs:

Amazon's Dynamo – provides *Availability* and *Partition Tolerance*, but *weak* consistency



Even worse, what about malfunctioning machines?

Byzantine Generals problem!

Now, your machines (or generals) can be actively trying to sabotage your consensus protocol.

Even worse, what about malfunctioning machines?

Byzantine Generals problem!

Now, your machines (or generals) can be actively trying to sabotage your consensus protocol.

Can solve with $3f+1$ machines for reasonable failure state, but well outside the scope of this lecture

DISTRIBUTED SYSTEMS



End of distributed systems lecture